

Università degli Studi di Padova
DIPARTIMENTO DI MATEMATICA “TULLIO LEVI-CIVITA”
CORSO DI LAUREA IN INFORMATICA



**Evoluzione e ottimizzazione di un firmware embedded per
sistema di telemetria basato su ESP32**

Tesi di Laurea

Relatore
Prof. Zanella Marco

Laureando
Laoud Zakaria
Matricola 2113196

إِنَّ مَعَ الْعُسْرِ يُسْرًا

“In verità, con la difficoltà giunge la facilità.”

- Corano (96 - 6)

Ringraziamenti

Desidero esprimere la mia sincera gratitudine al professor Zanella Marco, mio relatore, per la disponibilità, i preziosi consigli e il sostegno costante offertimi durante l'intero percorso di stesura di questo elaborato.

Un ringraziamento speciale va a mia madre, mio padre e mio fratello, che mi sono stati vicini in ogni momento di questo percorso. Il loro sostegno costante e la fiducia che hanno sempre riposto in me mi hanno spinto a non accontentarmi mai del primo risultato positivo, ma a cercare sempre di dare il meglio di me stesso.

Ringrazio inoltre il professor Filippo Cavallin, che durante gli anni delle scuole superiori mi ha insegnato a non porre limiti a me stesso e alle mie capacità, incoraggiandomi a puntare sempre più in alto.

Ringrazio in fine i ragazzi di Eggon, in particolare a Nicolò Castellin e Nadia Ilinca, per avermi seguito e supportato durante le fasi più impegnative di debugging, di individuazione e risoluzione degli errori.

Padova, Settembre 2026

Laoud Zakaria

Sommario

Il presente lavoro di tesi riguarda l'evoluzione e l'ottimizzazione di un firmware embedded sviluppato su piattaforma ESP32 per un sistema di telemetria applicato al settore motorsport. L'obiettivo principale è il miglioramento dell'efficienza, dell'affidabilità e delle funzionalità del sistema attraverso interventi sulla gestione dei dati, sull'acquisizione delle informazioni e sull'ottimizzazione delle risorse hardware disponibili.

Il lavoro si concentra sull'analisi del firmware esistente e sull'introduzione di miglioramenti volti a rendere il sistema più performante e adattabile alle esigenze di acquisizione dati in ambito motociclistico. Verranno inoltre integrate nuove funzionalità per ampliare le capacità del dispositivo e migliorare la qualità delle informazioni raccolte durante l'utilizzo.

Indice

1	Introduzione	1
1.1	L'azienda	1
1.2	Il progetto	1
1.3	Motivazioni della scelta del progetto	2
1.4	Convenzioni tipografiche	3
2	Descrizione dello stage	6
2.1	Competenze e obiettivi	6
2.1.1	Competenze da acquisire	6
2.1.2	Obiettivi	7
2.1.2.1	Obiettivi obbligatori	7
2.1.2.2	Obiettivi desiderabili	8
2.1.2.3	Obiettivi facoltativi	8
2.2	Pianificazione	8
2.3	Organizzazione del lavoro	10
2.4	Analisi preventiva dei rischi	10
3	Analisi dei requisiti	12
3.1	Stato attuale del firmware	12
3.2	Analisi degli utenti	12
3.3	Lista degli <i>use case</i>	12
3.3.1	UC1-F - Acquisizione binaria dei dati sensore	13
3.3.2	UC2-F - Registrazione dei dati binari dei sensori su SD	14
3.3.3	UC3-F - Calcolo dello spazio occupato dai dati registrati	15
3.3.4	UC4-A - Visualizzazione dello spazio di archiviazione occupato	16
3.3.5	UC5-F - Gestione della capacità di registrazione e blocco automatico	17
3.3.6	UC6-A - Notifica di sessione bloccata per spazio GPS esaurito	17
3.3.7	UC7-F - Recupero della sessione dopo rimozione SD	18
3.3.8	UC8-F - Notifica di rimozione della scheda microSD	18
3.3.9	UC9-A - Gestione della rimozione SD durante la sessione	19
3.3.10	UC10-A - Gestione dell'assenza della SD all'avvio sessione	20
3.3.11	UC11-S - Acquisizione dei dati dal bus CAN e trasmissione in broadcast BLE	20
3.3.12	UC11-F - Ricezione dei dati di telemetria CAN via BLE broadcast	21
3.3.13	UC12-F - Gestione dello stato del firmware tramite LED_G	22
3.3.13.1	UC12.1-F - Segnalazione di scheda microSD mancante	23
3.3.13.2	UC12.2-F - Segnalazione di errore generico	24
3.3.13.3	UC12.3-F - Segnalazione di pairing Wi-Fi	24
3.3.13.4	UC12.4-F - Segnalazione di pairing BLE	25

3.3.13.5	UC12.5-F - Segnalazione di recupero dati per calibrazione .	25
3.3.13.6	UC12.6-F - Segnalazione di calibrazione in corso	26
3.3.13.7	UC12.7-F - Segnalazione di download dati	26
3.3.13.8	UC12.8-F - Segnalazione di eliminazione dati	27
3.3.13.9	UC12.9-F - Segnalazione di sospensione per inattività . . .	27
3.3.13.10	UC12.10-F - Segnalazione di logging attivo	28
3.3.13.11	UC12.11-F - Segnalazione di dispositivo avviato in attesa di sessione	28
3.3.14	UC13-F - Sospensione dell'acquisizione sensori in stato di inattività	29
3.3.15	UC14-A - Configurazione del tempo di inattività	30
3.3.16	UC15-F - Avvio e interruzione sessione tramite comandi BLE	30
3.3.17	UC16-F - Calibrazione dell'IMU	31
3.3.18	UC17-A - Sequenza guidata di calibrazione dell'IMU	32
3.3.19	UC18-A - Visualizzazione dei dati calibrati	32
3.3.20	UC19-A - Visualizzazione dei movimenti della moto in tempo reale .	33
3.3.21	UC20-F - Calibrazione automatica dell'altitudine a inizio sessione . .	34
3.3.22	UC21-F - Salvataggio delle calibrazioni collegate alla sessione	34
3.4	Tracciamento dei requisiti	35
3.4.1	Requisiti funzionali	35
3.4.2	Requisiti non funzionali	38
3.4.3	Requisiti di vincolo	39
3.4.4	Tracciamento Fonti - Requisiti	40
3.4.5	Riepilogo dei requisiti	41
4	Tecnologie utilizzate	42
4.1	ESP32-S3-DevKitC-1	42
4.2	ESP-WROOM-32	43
4.3	C++20	44
4.4	ESP32 Arduino Core	44
4.5	FreeRTOS	45
4.5.1	Sistemi real-time e concorrenza	45
4.5.2	Scheduler e priorità	45
4.5.3	Task	46
4.5.4	Stati dei task	46
4.5.5	Race condition e sincronizzazione	47
4.5.6	Mutex	48
4.5.7	Comunicazione tra task	48
4.6	Expo & React Native	49
4.7	TypeScript	49
4.8	Git	50
4.9	PlatformIO	50
4.10	L ^A T _E X	51
4.11	PlantUML	51
4.12	C4 Model	51

5	Progettazione	52
5.1	Situazione di partenza	52
5.1.1	Firmware	52
5.1.1.1	Architettura	52
5.1.1.2	Scelte progettuali	53
5.1.2	Applicazione mobile	53
5.2	Sistema di persistenza a superfile	57
5.2.1	Architettura	57
5.2.2	Scelte progettuali	58
5.2.3	Formato su disco: l'header	58
5.2.4	Capacità del file	59
5.2.5	Scrittura e <i>wraparound</i>	59
5.2.6	Sincronizzazione dell'header	60
5.2.7	Finestra di azzeramento	61
5.2.8	Recovery dopo interruzioni impreviste	62
5.2.9	Comunicazione dello stato di occupazione	62
5.3	Stato globale del dispositivo	64
5.3.1	Architettura	64
5.3.2	Scelte progettuali	64
5.4	Segnalazione di stato tramite LED	65
5.4.1	Architettura	65
5.4.2	Scelte progettuali	65
5.5	Gestione dell'inattività dei sensori	67
5.5.1	Architettura	67
5.5.2	Scelte progettuali	67
5.5.3	Configurazione e comportamento ai bordi	67
5.6	Protocollo di avvio e arresto della sessione	69
5.6.1	Architettura	69
5.6.2	Scelte progettuali	70
5.6.3	Avvio sessione	70
5.6.4	Arresto sessione	71
5.7	Calibrazione dell'altitudine	73
5.7.1	Architettura	73
5.7.2	Algoritmo di calibrazione	73
5.8	Calibrazione dell'IMU	75
5.8.1	Architettura	75
5.8.2	Scelte progettuali	75
5.8.2.1	Algoritmo di calibrazione	76
5.8.2.1.1	Step 1 - Posizione dritta	76
5.8.2.1.2	Step 2 - Inclinazione a destra	76
5.8.2.1.3	Step 3 - Asse longitudinale e salvataggio	76
5.8.2.1.4	Gestione degli esiti negativi	76
5.9	Valutazione tecnica sniffer CAN	78
5.9.1	Hardware	78
5.9.1.1	Opzione 1 - protezione passiva	78

5.9.1.2	Opzione 2 - modulo esterno con adattatore logico	78
5.9.1.3	Opzione 3 - isolamento galvanico completo	78
5.9.2	Scelta dell'Opzione 1 e schema elettrico del prototipo	79
5.9.2.1	Componenti dello schema	79
5.9.2.2	Schema elettrico	79
5.9.2.3	Analisi dei rischi	80
5.10	Architettura dello sniffer CAN	81
5.10.1	Architettura	81
5.10.2	Scelte progettuali	82
5.10.3	Trasporto dei dati: BLE advertising broadcast	83
5.11	Visualizzazione 3D del veicolo	85
5.11.1	Architettura	85
5.11.2	Scelte progettuali	85
5.12	Segnalazione di microSD mancante durante la sessione	86
5.12.1	Architettura	86
5.12.2	Scelte progettuali	86
5.13	Configurazione dei parametri del dispositivo	88
5.13.1	Architettura	88
5.13.2	Scelte progettuali	88
6	Implementazione	89
6.1	Sistema di persistenza a superfile	89
6.1.1	Creazione e calcolo della capacità	91
6.1.2	Scrittura e wraparound	92
6.1.3	Sincronizzazione e finestra di azzeramento	92
6.1.4	Recovery su più livelli	93
6.1.5	Comunicazione dello stato di occupazione	93
6.1.6	Problematiche riscontrate e soluzioni	93
6.2	Stato globale del dispositivo	95
6.2.1	Accesso senza lock	95
6.2.2	Un caso concreto: due task sullo stesso indicatore	96
6.3	Segnalazione di stato tramite LED	97
6.3.1	Livello decisionale	97
6.3.2	Livello di rendering	98
6.3.3	Livello hardware	98
6.4	Gestione dell'inattività dei sensori	99
6.4.1	Sorgente autorevole	99
6.4.2	Debounce sul segnale GPS	99
6.4.3	Soglie e parametri di temporizzazione	100
6.4.4	Attivazione mirata del fallback IMU	100
6.4.5	Sospensione esclude il GPS	101
6.4.6	Sospensione disabilitata	101
6.5	Protocollo di avvio e arresto della sessione	102
6.5.1	Un prerequisito: producer in grado di dormire	102
6.5.2	Comandi come valori instradati centralmente	102

6.5.3	Avvio sessione	103
6.5.4	Arresto sessione	103
6.5.5	Notifica periodica dello stato di sessione	103
6.5.6	Comando RESUME: risincronizzazione dopo la riconnessione BLE	104
6.5.7	Un caso particolare: l'arresto per spazio esaurito	104
6.6	Calibrazione dell'altitudine	105
6.6.1	Raccolta campioni e filtro duplicati	105
6.6.2	Mediana, deviazione standard e accettazione del gruppo	105
6.6.3	Formula di conversione e scelta del dato da conservare	106
6.7	Calibrazione dell'IMU	107
6.7.1	Fan-out non invasivo sul flusso dati	107
6.7.2	Macchina a stati e instradamento dei comandi	108
6.7.3	Soglie di accettazione degli step	108
6.7.4	Uscita anticipata basata sulla stabilità del segnale	108
6.7.5	Step 1 - Posizione dritta	109
6.7.6	Step 2 - Inclinazione a destra	109
6.7.7	Step 3 - Asse longitudinale e salvataggio	109
6.7.8	Gestione degli esiti negativi	110
6.7.9	Problematiche riscontrate e soluzioni	110
6.8	Architettura dello sniffer CAN	111
6.8.1	Il caso d'uso: orchestrazione senza conoscenza degli adattatori	111
6.8.2	Adattatore del bus CAN: ascolto passivo	113
6.8.3	Selezione del profilo veicolo	113
6.8.4	Decodifica specifica del veicolo	113
6.8.5	Trasporto dati: advertising BLE broadcast-only	114
6.9	Visualizzazione 3D del veicolo	115
6.9.1	Aggiornamento del riferimento, non dello stato React	115
6.9.2	Lettura e interpolazione nel ciclo di rendering 3D	115
6.9.3	Elementi testuali a frequenza ridotta	116
6.10	Segnalazione di microSD mancante durante la sessione	117
6.10.1	Ripartizione dell'occupazione per sensore	117
6.10.2	Stesso componente, azione differente per contesto	118
6.10.3	Impossibilità di chiusura senza azione	119
6.10.4	Scomparsa automatica al reinserimento della microSD	119
6.11	Configurazione dei parametri del dispositivo	121
6.11.1	Un controllo dedicato per un insieme discreto di valori	121
6.11.2	Codifica esplicita immediatamente prima della scrittura	121
7	Verifica e Validazione	123
7.1	Introduzione	123
7.2	Test	123
7.2.1	Metodologia dei test su campo	123
7.2.2	Test su campo 1 - Calibrazione e posizionamento del sensore	124
7.2.3	Test su campo 2 - Arrotondamento dei dati di accelerazione	125
7.2.4	Test su campo 3 - Validazione finale	125

7.2.5	Test su banco 1 - Sospensione e ripresa del logging con fix GPS . . .	125
7.2.6	Test su banco 2 - <i>Fallback</i> sull'IMU senza fix GPS	125
7.2.7	Sintesi dei test	125
7.3	Benchmark prestazionali	127
7.3.1	Metodologia	127
7.3.2	Tempo di scrittura su microSD	127
7.3.2.0.1	Il problema della cluster size (superato)	127
7.3.2.0.2	Confronto misurato: un file per chunk vs superfile	127
7.3.3	Occupazione di CPU	128
7.3.4	Occupazione della RAM	128
7.3.4.0.1	Calcolo della RAM occupata	128
7.3.4.0.2	Confronto per sensore	128
7.3.4.0.3	Variazione percentuale tra le due versioni	129
7.3.4.0.4	Il ruolo del formato di serializzazione	129
7.3.4.0.5	Un compromesso tra RAM e frequenza di scrittura	130
7.3.4.0.6	Disabilitazione del magnetometro	130
7.3.4.0.7	Il momento dell'allocazione conta più della dimen- sione	130
8	Conclusioni	132
8.1	Consuntivo finale	132
8.2	Raggiungimento degli obiettivi	132
8.2.1	Obiettivi obbligatori	132
8.2.2	Obiettivi desiderabili	133
8.2.3	Obiettivi facoltativi	133
8.2.4	Requisiti raggiunti	133
8.3	Conoscenze acquisite	134
8.4	Valutazione personale	134
	Glossario	i
	Sitografia	vii

Elenco delle figure

1.1	Logo dell'azienda Eggon Srl	1
1.2	Legenda dei diagrammi di flusso	4
1.3	Legenda dei diagrammi di sequenza	5
3.1	UC1-F - Acquisizione binaria dei dati sensore	13
3.2	Inclusioni UC2-F - Registrazione dei dati sensore su SD	14
3.3	UC2-F - Registrazione dei dati sensore su SD	15
3.4	UC3-F - Calcolo dello spazio occupato dai dati registrati	15
3.5	UC4-A - Visualizzazione dello spazio di archiviazione occupato	16
3.6	UC5-F - Gestione della capacità di registrazione e blocco automatico	17
3.7	UC6-A - Notifica di sessione bloccata per spazio GPS esaurito	17
3.8	UC7-F - Recupero della sessione dopo rimozione SD	18
3.9	UC8-F - Notifica di rimozione della scheda microSD	18
3.10	UC9-A - Gestione della rimozione SD durante la sessione	19
3.11	UC10-A - Gestione dell'assenza della SD all'avvio sessione	20
3.12	UC11-S - Acquisizione dei dati dal bus CAN e trasmissione in broadcast BLE	20
3.13	UC11-F - Ricezione dei dati di telemetria CAN via BLE broadcast	21
3.14	UC12-F - Gestione dello stato del firmware tramite LED	22
3.15	UC12.1-F - Segnalazione di scheda microSD mancante	23
3.16	UC12.2-F - Segnalazione di errore generico	24
3.17	UC12.3-F - Segnalazione di pairing Wi-Fi	24
3.18	UC12.4-F - Segnalazione di pairing BLE	25
3.19	UC12.5-F - Segnalazione di recupero dati per calibrazione	25
3.20	UC12.6-F - Segnalazione di calibrazione in corso	26
3.21	UC12.7-F - Segnalazione di download dati	26
3.22	UC12.8-F - Segnalazione di eliminazione dati	27
3.23	UC12.9-F - Segnalazione di sospensione per inattività	27
3.24	UC12.10-F - Segnalazione di logging attivo	28
3.25	UC12.11-F - Segnalazione di dispositivo avviato in attesa di sessione	28
3.26	UC13-F - Sospensione dell'acquisizione sensori in stato di inattività	29
3.27	UC14-A - Configurazione del tempo di inattività	30
3.28	UC15-F - Avvio e interruzione sessione tramite comandi BLE	30
3.29	UC16-F - Calibrazione dell'IMU	31
3.30	UC17-A - Sequenza guidata di calibrazione dell'IMU	32
3.31	UC18-A - Visualizzazione dei dati calibrati	32
3.32	UC19-A - Visualizzazione dei movimenti della moto in tempo reale	33
3.33	UC20-F - Calibrazione automatica dell'altitudine a inizio sessione	34
3.34	UC21-F - Salvataggio delle calibrazioni collegate alla sessione	34

4.1	Scheda di sviluppo ESP32-S3-DevKitC-1	42
4.2	Modulo ESP-WROOM-32	43
4.3	Logo del linguaggio C++	44
4.4	FreeRTOS	45
4.5	Logo di Expo & React Native	49
4.6	Git	50
4.7	PlatformIO	50
4.8	Esempio di diagramma generato con PlantUML	51
5.1	Architettura producer-consumer preesistente	53
5.2	Diagramma di contesto del sistema EggLog	55
5.3	Diagramma dei container del sistema EggLog	56
5.4	Diagramma dei componenti del firmware EggLog: persistenza e stato con- diviso	57
5.5	Campi logici dell'header del superfile	59
5.6	Diagramma di flusso della scrittura di un insieme di record	61
5.7	Layout su disco del superfile: area dati valida, finestra di azzeramento e spazio libero	62
5.8	Flusso di recovery di un superfile dopo un'interruzione imprevista	63
5.9	Diagramma dei componenti del firmware EggLog: segnalazione e monito- raggio sensori	65
5.10	Transizioni alla sospensione guidate da GPS con fallback IMU	68
5.11	Diagramma dei componenti del firmware EggLog: sessione e calibrazioni	69
5.12	Diagramma di sequenza del comando START	71
5.13	Diagramma di sequenza del comando STOP	72
5.14	Diagramma di flusso della calibrazione altitudine	74
5.15	Diagramma della calibrazione dell'IMU	77
5.16	Schema elettrico di EggLog CAN Sniffer.	79
5.17	Diagramma dei componenti di EggLog CAN Sniffer: architettura esagonale	82
5.18	Flusso dati dal bus CAN al payload BLE	83
5.19	I tre ruoli BLE nel sistema complessivo	83
5.20	Diagramma dei componenti dell'applicazione EggLog: schermata Record e configurazione	84
6.1	Visualizzazione 3D della moto, con assetto aggiornato in tempo reale	116
6.2	Ripartizione dell'occupazione per sensore, con indicatore di sovrascrittura	118
6.3	I due utilizzi del componente SdMissingOverlay	119

Elenco delle tabelle

3.1	Tracciamento dei requisiti funzionali.	38
3.2	Tracciamento dei requisiti non funzionali.	39
3.3	Tracciamento dei requisiti di vincolo.	40
3.4	Tracciamento Fonti - Requisiti.	41
3.5	Conteggio dei requisiti tracciati.	41
4.1	Principali parametri di configurazione di un task in FreeRTOS	46
4.2	Principali stati di una task in FreeRTOS	47
4.3	Esempio di interleaving che genera una race condition	47
4.4	Fasi di utilizzo di un mutex	48
4.5	Confronto tra queue e task notification in FreeRTOS	48
5.1	Priorità e segnalazioni visive del sottosistema LED	66
5.2	Confronto tra le soluzioni considerate per l'interfacciamento CAN	78
5.3	Rischi elettrici e relative mitigazioni	80
6.1	Corrispondenza tra i campi logici dell'header e la loro effettiva persistenza	90
6.2	Parametri concreti per sensore definiti in <code>constants.h</code>	92
6.3	Soglie di accettazione della calibrazione IMU	108
7.1	Sintesi dei test condotti sul firmware	126
7.2	Confronto dei tempi di scrittura su microSD	127
7.3	Occupazione di CPU nei diversi stati del firmware	128
7.4	RAM occupata dai buffer dei producer (ping-pong incluso)	129
7.5	Variazione percentuale della RAM occupata dai buffer dei producer	129
7.6	Confronto tra la dimensione stimata di un record in formato CSV e la dimensione del record binario attuale	129
7.7	Numero stimato di campioni contenuti in un singolo buffer, prima e dopo il passaggio al formato binario	130
8.1	Ripartizione delle ore di stage per fase	132
8.2	Raggiungimento degli obiettivi obbligatori	133
8.3	Raggiungimento degli obiettivi desiderabili	133
8.4	Raggiungimento degli obiettivi facoltativi	133
8.5	Requisiti raggiunti, per categoria e urgenza	134

Elenco dei codici sorgenti

1.1	Esempio di estratto di codice	3
6.1	Header effettivamente persistito su disco	89
6.2	Interfaccia del modulo superfile (<code>openAndRecover</code> condivide gli stessi parametri di <code>create</code> , omessi per brevità)	91
6.3	Calcolo della capacità del superfile	92
6.4	Split della scrittura in caso di wraparound (gestione errori omessa per brevità)	92
6.5	Riconciliazione del cursore in fase di recovery	93
6.6	Rilevamento della scheda basato sul pin DET	93
6.7	Rilevamento della scheda tramite lettura del CID	94
6.8	Stato principale di sessione	95
6.9	Interfaccia di un indicatore booleano indipendente	95
6.10	Un indicatore come variabile atomica	96
6.11	Scrittura dell'indicatore da parte del task consumer	96
6.12	Lettura dello stesso indicatore da parte del task LED	96
6.13	Value Object del pattern visivo	97
6.14	Risoluzione del pattern per priorità esplicita (troncato per brevità)	97
6.15	Aggancio della fase dell'animazione al cambio di pattern	98
6.16	Interfaccia del livello hardware	98
6.17	Interfaccia del modulo di gestione dell'inattività	99
6.18	Determinazione della sorgente autorevole	99
6.19	Debounce del segnale di movimento GPS (semplificato)	100
6.20	Campionamento mirato del fallback IMU nel producer	100
6.21	Propagazione della sospensione a tutti i producer tranne il GPS	101
6.22	Disattivazione della sospensione tramite timeout a zero	101
6.23	Interfaccia del modulo sessione	102
6.24	Attesa del producer finché non è in corso una sessione (semplificato)	102
6.25	Sequenza di avvio sessione (semplificata)	103
6.26	Sequenza di arresto sessione (semplificata)	103
6.27	Interfaccia del modulo di calibrazione dell'altitudine	105
6.28	Filtro dei campioni duplicati tramite confronto del timestamp	105
6.29	Calcolo di mediana e deviazione standard, e criterio di accettazione	106
6.30	Comandi e stati della calibrazione IMU	107
6.31	Campione grezzo instradato al task di calibrazione	107
6.32	Duplicazione del campione verso il canale di calibrazione	107
6.33	Firma della funzione comune di esecuzione di uno step	108
6.34	Uscita anticipata quando il segnale è già stabile (semplificato)	109
6.35	Le tre porte del dominio	111

6.36	Costruzione del caso d'uso tramite le sole interfacce di dominio	112
6.37	Punto di chiamata del ciclo di orchestrazione	112
6.38	Ciclo di orchestrazione del caso d'uso	112
6.39	Configurazione del controller TWAI in modalità listen-only	113
6.40	Selezione del profilo veicolo tramite Factory Pattern	113
6.41	Instradamento dei frame per identificativo nel profilo Yamaha MT-07 (semplificato)	114
6.42	Configurazione dell'advertising come non connettibile	114
6.43	Verifica statica della dimensione del pacchetto trasmesso	114
6.44	Stato di assetto mantenuto fuori dal ciclo di rendering React	115
6.45	Sottoscrizione diretta allo store, senza passare da setState React	115
6.46	Interpolazione del valore letto a ogni frame (semplificato)	115
6.47	Firma del componente presentazionale di segnalazione SD mancante	117
6.48	Ripartizione per sensore della percentuale di occupazione	117
6.49	Le due invocazioni del componente, con azioni differenti	118
6.50	Overlay bloccante senza via di chiusura alternativa	119
6.51	L'overlay scompare da solo non appena la condizione diventa falsa	120
6.52	Riconoscimento del parametro di timeout tramite l'oggetto di dominio	121
6.53	Menu a tendina per il timeout di inattività	121
6.54	Codifica del valore in un intero a 16 bit little-endian	122

1. Introduzione

Nel seguente capitolo viene presentata l'azienda ospitante, il progetto svolto e le motivazioni che hanno portato alla scelta di questo argomento per il lavoro di tesi.

1.1 L'azienda

Eggon S.r.l. è una *software house* con sede a Padova che opera in diversi ambiti tecnologici, sviluppando soluzioni software innovative e fornendo servizi personalizzati in base alle esigenze dei propri clienti. L'attività dell'azienda si concentra principalmente in settori quali la *Digital Health_G*, l'*Industria 5.0_G* con dispositivi correlati all'*Industrial Internet of Things_G*, l'*Insurtech_G* e il *Fintech_G*, oltre che nell'ambito dell'automazione logistica.



Figura 1.1: Logo dell'azienda Eggon Srl

La società nasce dall'esperienza maturata negli Stati Uniti dai suoi fondatori, Gianpaolo Ferrarin e Fulvio Menegozzo, i quali hanno successivamente introdotto nel contesto italiano metodologie di lavoro, approcci innovativi e una forte attenzione alla concretezza nello sviluppo di soluzioni tecnologiche.

1.2 Il progetto

Il tirocinio si è svolto presso Eggon Srl ed è stato dedicato allo sviluppo di **EggLog Motion Core**, un ecosistema per la telemetria applicata al motorsport motociclistico. Il sistema si articola in un dispositivo *embedded_G* per l'acquisizione dei dati a bordo del veicolo e in un'applicazione dedicata alla consultazione, alla visualizzazione e all'analisi delle informazioni raccolte durante le sessioni di guida stradale o in pista.

L'obiettivo del progetto è stato realizzare *firmware_G* affidabile per i dispositivi embedded, basati su *ESP32-S3* e *ESP-WROOM-32*, in grado di acquisire i dati provenienti dai diversi sensori installati a bordo, organizzarli in modo efficiente e memorizzarli localmente su scheda microSD, garantendo continuità di acquisizione, integrità dei dati e prestazioni adeguate ai vincoli tipici di un sistema real-time. Parallelamente, è stata sviluppata un'applicazione destinata all'interazione con il dispositivo e all'analisi dei dati acquisiti.

Il lavoro ha previsto inoltre lo studio del prototipo realizzato dal precedente gruppo di tirocinanti, con l'obiettivo di valutare le soluzioni adottate e individuare possibili margini di miglioramento, sia a livello di architettura software sia in vista di una futura revisione hardware del dispositivo.

Il progetto ha permesso di affrontare problematiche caratteristiche dello sviluppo di sistemi embedded destinati ad applicazioni reali, approfondendo aspetti relativi all'acquisizione dei dati da sensori, alla gestione efficiente delle risorse hardware e alla progettazione di firmware affidabili.

1.3 Motivazioni della scelta del progetto

Le principali motivazioni che mi hanno portato a scegliere questo progetto di tirocinio sono:

- **Interesse per l'Internet of Things e i sistemi embedded**

Fin dagli anni della scuola secondaria di secondo grado ho sviluppato un forte interesse per il mondo dell'*Internet of Things*_G, grazie alla realizzazione di alcuni progetti che mi hanno permesso di avvicinarmi ai sistemi embedded. Questo interesse mi ha spinto a ricercare un'esperienza che mi consentisse di approfondire tali tematiche in un contesto professionale.

- **Interesse per l'automotive**

Nel corso degli anni ho sviluppato un particolare interesse per il settore *automotive*_G e per le tecnologie che ne caratterizzano l'evoluzione, come i sistemi di telemetria, le reti di comunicazione veicolari e le applicazioni software dedicate all'acquisizione e all'analisi dei dati.

- **Ambiente di lavoro giovane**

Un ulteriore elemento che ha influenzato la mia scelta è stato il contesto aziendale. Essendo composta prevalentemente da un team giovane, l'azienda favorisce una comunicazione diretta, il confronto continuo e uno scambio di idee privo di barriere.

- **Approfondimento dello sviluppo firmware**

Nel corso di precedenti esperienze avevo già utilizzato piattaforme come *Arduino*_G, ma esclusivamente per applicazioni relativamente semplici. Il tirocinio rappresentava quindi un'opportunità per affrontare uno sviluppo firmware più strutturato, lavorando direttamente su un prodotto reale e acquisendo competenze più avanzate nel campo dell'embedded.

- **Crescita personale**

Gli argomenti relativi alle telecomunicazioni sono sempre stati tra quelli che ho trovato più complessi durante il percorso scolastico. Per questo motivo ho visto il progetto come una sfida personale, con l'obiettivo di consolidare le mie conoscenze e migliorare le mie competenze.

1.4 Convenzioni tipografiche

Durante la redazione del presente documento sono state adottate alcune convenzioni tipografiche con l'obiettivo di rendere più chiara e uniforme la consultazione del testo. In particolare:

- Gli acronimi, le abbreviazioni e i termini tecnici o di uso non comune vengono descritti all'interno del *glossario*, riportato nella parte finale del documento;
- Alla prima occorrenza di un termine presente nel glossario viene utilizzata la seguente nomenclatura: *Arduino_G*;
- Nel caso degli acronimi alla prima ricorrenza viene utilizzato il riferimento per esteso (*Industrial Internet of Things_G*), mentre dalla seconda in poi viene semplicemente riportato come acronimo (IIoT);
- I termini in lingua straniera non entrati nell'uso comune della lingua italiana, così come quelli appartenenti al gergo tecnico, vengono riportati in carattere *corsivo*;
- I nomi di funzioni, variabili o altri elementi appartenenti al codice sorgente vengono rappresentati mediante carattere **monospaziato**;
- I riferimenti a risorse esterne presenti nella *bibliografia* sono indicati nella seguente maniera¹;
- Gli estratti di codice sorgente vengono riportati attraverso appositi blocchi formatati, come riportato di seguito:

```
1 struct Header {  
2     uint32_t magic; // commento  
3     uint8_t version;  
4     ...  
5 };
```

Codice 1.1: Esempio di estratto di codice

- I diagrammi di flusso e di sequenza utilizzati per illustrare la logica del firmware e le interazioni tra i componenti seguono le convenzioni notazionali riportate rispettivamente in Figura 1.2 e in Figura 1.3.

¹ESP32-S3-DevKitC-1. URL: <https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/index.html>.

Legenda - Diagrammi di flusso

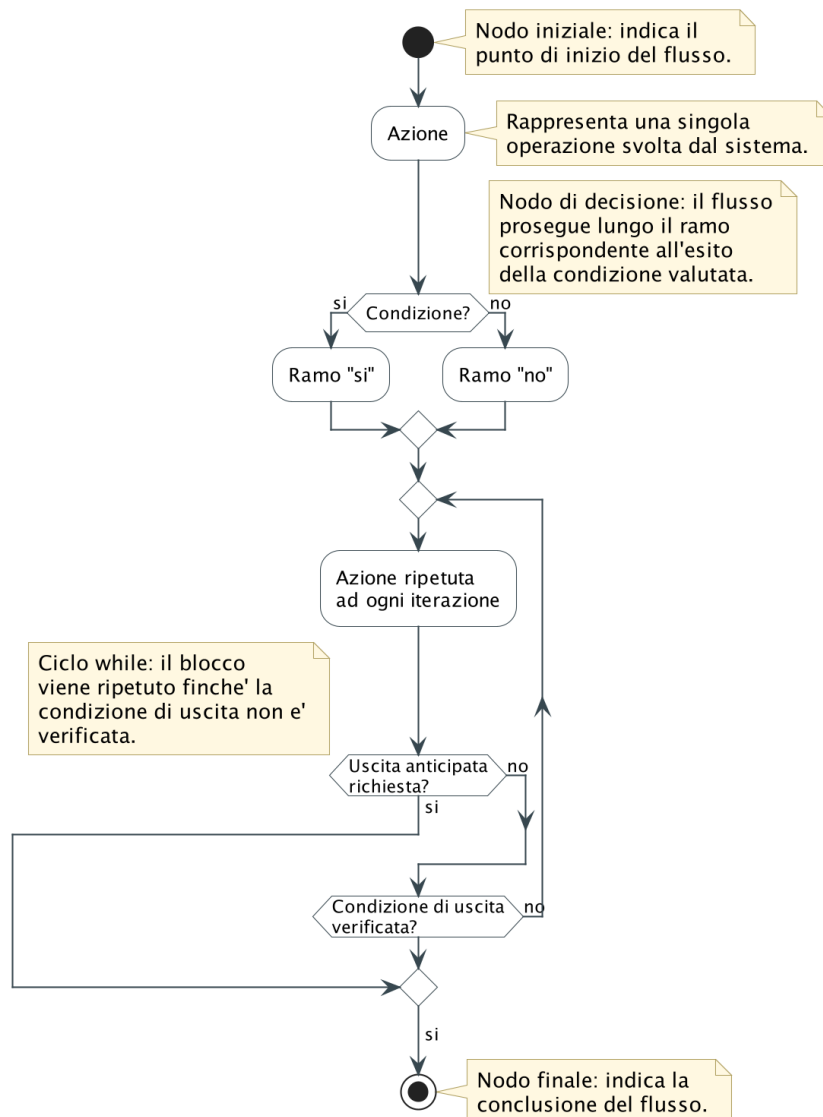


Figura 1.2: Legenda dei diagrammi di flusso

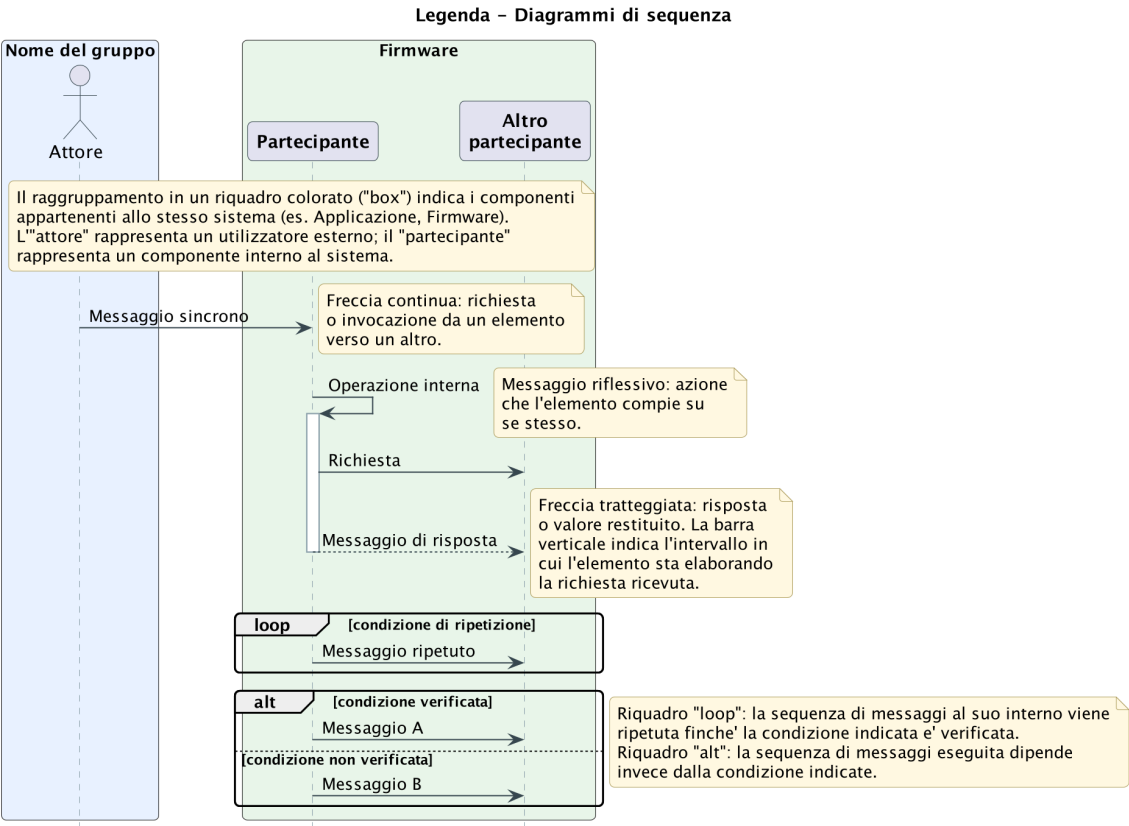


Figura 1.3: Legenda dei diagrammi di sequenza

2. Descrizione dello stage

Il presente capitolo descrive il percorso di stage svolto presso l'azienda, illustrando gli obiettivi formativi, le competenze richieste e le attività pianificate durante il periodo di tirocinio. Vengono inoltre presentate l'organizzazione del lavoro adottata, l'analisi preventiva dei principali rischi progettuali e il ruolo svolto dal tutor aziendale nel supporto alle attività di sviluppo.

2.1 Competenze e obiettivi

L'obiettivo principale dello stage è stato l'evoluzione e l'ottimizzazione del firmware embedded del sistema *EggLog Motion Core*, una piattaforma dedicata all'acquisizione e alla gestione di dati telemetrici in ambito motorsport, sviluppata su piattaforma ESP32.

Il lavoro riguarda principalmente l'analisi dell'architettura firmware esistente, il miglioramento dei componenti software già presenti, sia su firmware che nell'applicazione mobile, e l'introduzione di nuove funzionalità volte a rendere il sistema più affidabile, efficiente e facilmente estendibile.

Le attività svolte hanno coinvolto diversi aspetti del sistema, tra cui la gestione dello *storage* su memoria microSD, l'ottimizzazione dei servizi di comunicazione BLE (*Bluetooth Low Energy_G*), la gestione del trasferimento dei dati acquisiti real-time e l'analisi del traffico CAN proveniente dalla centralina elettronica (*Electronic Control Unit_G*) della moto mediante tecniche di *CAN sniffing_G*.

2.1.1 Competenze da acquisire

Durante il periodo di tirocinio erano previste l'acquisizione e il consolidamento delle seguenti competenze:

- sviluppo e manutenzione di firmware embedded in linguaggio *C++_G* su piattaforma ESP32;
- analisi, *refactoring* e ottimizzazione di codice embedded esistente;
- gestione di sistemi di memorizzazione embedded e ottimizzazione delle operazioni di lettura e scrittura su memoria SD;
- sviluppo e ottimizzazione di servizi BLE;
- analisi dei protocolli di comunicazione utilizzati in ambito automotive, con particolare riferimento al *bus CAN_G*;
- acquisizione e analisi di messaggi dal bus CAN provenienti dalla ECU di un veicolo motorizzato;

- integrazione lato applicazione mobile delle nuove funzionalità sviluppate lato firmware, attraverso l'implementazione delle relative funzionalità di comunicazione e gestione dei dati;
- utilizzo di strumenti di *debugging* e *testing* per sistemi embedded;
- produzione di documentazione tecnica relativa al firmware e ai protocolli di comunicazione analizzati.

2.1.2 Obiettivi

Gli obiettivi definiti dall'azienda sono stati classificati secondo tre livelli di priorità:

- **O**: obbligatorio;
- **D**: desiderabile;
- **F**: facoltativo.

2.1.2.1 Obiettivi obbligatori

Gli obiettivi obbligatori comprendevano principalmente attività volte al miglioramento dell'affidabilità e delle prestazioni del firmware esistente.

- **OO-1**: *Refactoring* e ottimizzazione del sistema di lettura e scrittura su scheda microSD;
- **OO-2**: Completamento del sistema di rotazione dei file mediante l'utilizzo di *buffer circolare_G*;
- **OO-3**: Implementare la scrittura e salvataggio dei dati raccolti direttamente in formato binario;
- **OO-4**: Implementazione di un sistema di *recovery* che permette di riprendere la scrittura su microSD in caso di guasti o rimozione della stessa dal suo *slot*.
- **OO-5**: Miglioramento dei servizi BLE e introduzione di nuovi comandi firmware;
- **OO-6**: Aggiunta di metriche di confronto per poter stimare il miglioramento rispetto alla versione precedente del firmware;
- **OO-7**: Implementare un protocollo di calibrazione dei sensori;
- **OO-8**: Aggiungere una logica di sessionamento, attivando i sensori solamente all'avvio della sessione e disattivandoli quando termina la sessione;
- **OO-9**: Produzione della documentazione tecnica relativa al firmware sviluppato.

2.1.2.2 Obiettivi desiderabili

Gli obiettivi desiderabili erano principalmente orientati all'introduzione di funzionalità aggiuntive e al miglioramento delle modalità di trasferimento dei dati.

- **OD-1:** Permettere la visualizzazione dei movimenti della moto in tempo reale con un modellino 3D;
- **OD-2:** Integrazione dell'analisi e gestione dei dati provenienti dalla ECU;
- **OD-3:** Implementare un sistema di rilevamento di assenza della scheda microSD.

2.1.2.3 Obiettivi facoltativi

- **OF-1:** Implementazione di un sistema di indicizzazione delle sessioni memorizzate sulla scheda microSD.
- **OF-2:** Implementazione di un sistema di *inactivity* per permettere la disattivazione dei sensori quando non viene rilevato alcun movimento per un tempo configurabile dall'utente.

2.2 Pianificazione

Le attività di stage sono state pianificate con il tutor aziendale considerando un monte complessivo di 320 ore, distribuite su otto settimane lavorative da quaranta ore ciascuna. Tali settimane sono state suddivise come segue:

1. Prima settimana - *Onboarding* e analisi del progetto

- Introduzione al sistema **EggLog Motion Core** e all'architettura generale della piattaforma;
- Configurazione dell'ambiente di sviluppo e degli strumenti di compilazione e debugging;
- Analisi della struttura del firmware, dell'app esistente e delle principali componenti software e hardware;
- Studio delle modalità di acquisizione, memorizzazione e trasferimento dei dati.

2. Seconda settimana - Studio delle tecnologie e analisi del firmware

- Approfondimento della gestione della memoria microSD e del relativo *file system*;
- Analisi dello stack BLE e dei servizi implementati;
- Studio dei protocolli di comunicazione utilizzati dal sistema;
- Individuazione delle principali criticità presenti nel firmware.

3. Terza settimana - Analisi e progettazione delle modifiche

- Analisi dell'architettura software esistente e definizione degli interventi di *refactoring*;

- Progettazione delle modifiche relative alla gestione dello storage;
- Studio dei meccanismi di recupero e gestione dei dati in caso di errore;
- Analisi preliminare del bus CAN della moto e delle informazioni trasmesse dalla ECU.

4. Quarta settimana - Sviluppo e ottimizzazione dello storage

- *Refactoring* dei moduli responsabili della gestione della memoria SD;
- Ottimizzazione delle operazioni di lettura e scrittura dei dati;
- Miglioramento del sistema di rotazione dei file;
- Implementazione e verifica dei meccanismi di *recovery*.

5. Quinta settimana - Gestione dei dati e comunicazioni

- Implementazione delle funzioni di calibrazione dei sensori;
- Miglioramento della qualità dei dati inviati via BLE, inteso come invio di dati già calibrati;
- Miglioramento dei servizi BLE;
- Implementazione di un sistema di inattività, basato su GPS con *fallback* su accelerometro.

6. Sesta settimana - Analisi tecnica CAN sniffer e integrazione firmware

- Stesura di un'analisi tecnica sulla fattibilità di uno sniffer CAN;
- Acquisizione dei messaggi presenti sul bus CAN tramite attività di *sniffing*;
- Analisi degli identificativi dei pacchetti CAN provenienti dalla ECU;
- Studio della possibile integrazione delle informazioni trasmesse nel bus CAN all'interno del sistema telemetrico;

7. Settima settimana - Testing e validazione

- Esecuzione di test funzionali sul firmware modificato;
- Raccolta di metriche sulle prestazioni del sistema;
- Esecuzione di benchmark sulle operazioni di memorizzazione e occupazione della CPU;
- Analisi e risoluzione delle problematiche emerse durante i test.

8. Ottava settimana - Documentazione finale

- Aggiornamento della documentazione tecnica del firmware;
- Descrizione delle modifiche architetturali introdotte;
- Documentazione relativa ai protocolli di comunicazione analizzati;
- Stesura della relazione finale di stage.

2.3 Organizzazione del lavoro

Le attività di sviluppo sono state organizzate seguendo una *Metodologia Agile_G*, ispirata al framework *Scrum_G*. Il lavoro è stato suddiviso in sprint della durata indicativa di due settimane, durante i quali venivano pianificate le attività da svolgere, definiti gli obiettivi da raggiungere e monitorato lo stato di avanzamento delle attività insieme al tutor aziendale.

Durante il periodo di stage sono stati inoltre svolti degli *stand-up meeting_G* con cadenza giornaliera, finalizzati al mantenimento dell'allineamento tra i membri del team. Questi incontri hanno permesso di condividere i progressi effettuati, discutere eventuali problematiche emerse durante le fasi di sviluppo e progettazione e individuare possibili soluzioni attraverso il confronto tecnico.

Il gruppo di lavoro era composto da due tirocinanti, con attività prevalenti su componenti differenti del sistema. Il sottoscritto ha operato principalmente sulla componente firmware embedded, occupandosi dell'evoluzione del software a bordo del dispositivo, e ha inoltre contribuito all'applicazione con interventi puntuali. La collega ha lavorato principalmente sulle componenti applicative, contribuendo a sua volta al firmware con interventi mirati. Nonostante la suddivisione delle responsabilità, le attività sono state svolte in maniera collaborativa, favorendo il confronto continuo tra i membri del team e garantendo una corretta integrazione tra firmware, applicazione mobile e infrastruttura software.

2.4 Analisi preventiva dei rischi

Prima dell'avvio delle attività è stata effettuata un'analisi preventiva dei principali rischi che avrebbero potuto influenzare il corretto svolgimento del progetto. Per ciascun rischio individuato si riportano:

- la probabilità di accadimento (Improbabile, Poco probabile, Probabile, Altamente probabile);
- le conseguenze in caso di accadimento (Lieve, Medio, Grave, Gravissimo);
- rimedi adottati per mitigarne la probabilità o le conseguenze.

I possibili rischi individuati sono:

- **Introduzione di regressioni nel codice esistente**

Probabilità: Probabile - il firmware esistente non dispone di test automatici, per cui le modifiche apportate al codice possono alterare comportamenti già funzionanti senza che ciò venga rilevato immediatamente.

Conseguenze: Grave - una regressione non rilevata può compromettere l'affidabilità dell'acquisizione o della scrittura dei dati, richiedendo una nuova sessione di test per essere individuata.

Rimedi: prima di intervenire su ciascun modulo è stata condotta un'analisi puntuale dell'architettura e del codice preesistente; dopo ogni modifica sono state eseguite sessioni di test manuale mirate alle funzionalità potenzialmente impattate, seguite da revisione del codice con il tutor aziendale.

- **Mancato miglioramento delle prestazioni**

Probabilità: Probabile - le modifiche introdotte per ottimizzare acquisizione, buffering e trasferimento dei dati comportano un *overhead* computazionale e di occupazione *RAM_G* che potrebbe annullare, in tutto o in parte, il guadagno prestazionale atteso, specialmente se il compromesso tra velocità e memoria non viene bilanciato correttamente.

Conseguenze: Medio - un mancato miglioramento prestazionale vanificherebbe l'obiettivo stesso delle ottimizzazioni introdotte durante il tirocinio, richiedendo ulteriori interventi correttivi.

Rimedi: per prevenire il rischio, la scelta delle strutture dati e degli algoritmi di buffering è stata guidata da una stima preventiva del loro costo computazionale e di memoria, con verifiche incrementalmente ad ogni modulo sviluppato anziché un'unica misurazione finale. Per individuare tempestivamente eventuali regressioni prestazionali sono stati inoltre implementati dei test di *benchmark* che misurano: il tempo impiegato per ogni operazione di scrittura e il livello occupazionale di *RAM_G* e *CPU_G*. Tali *benchmark* sono stati implementati anche nella versione precedente del firmware a scopo di confronto.

- **Interpretazione errata dei messaggi provenienti dal bus CAN**

Probabilità: Altamente probabile - i messaggi trasmessi sul bus CAN dalla ECU della moto non seguono una codifica standardizzata e pubblica, per cui è necessario ricostruirne il significato tramite osservazione ed ipotesi, con concreto rischio di misinterpretazione.

Conseguenze: Grave - un'errata interpretazione porta all'acquisizione di dati non corretti o non significativi, invalidando le misurazioni raccolte e richiedendo di ripetere l'analisi.

Rimedi: prima di avviare l'attività di *sniffing* è stata condotta un'analisi di realizzabilità software; l'interpretazione dei messaggi è stata poi validata in modo incrementale confrontando i valori decodificati con misurazioni di riferimento note.

- **Incompatibilità tra le componenti sviluppate in parallelo**

Probabilità: Probabile - il firmware e l'applicazione mobile sono stati sviluppati contemporaneamente da persone diverse, con il rischio che le rispettive interfacce evolvano in modo incoerente tra loro.

Conseguenze: Medio - un eventuale disallineamento richiederebbe comunque lavoro aggiuntivo di adattamento e potrebbe ritardare il completamento degli obiettivi predisposti.

Rimedi: per le funzionalità introdotte nel corso del tirocinio, le modifiche lato firmware e la relativa integrazione lato applicativo sono state implementate in modo contestuale dalla stessa persona, garantendo coerenza tra le interfacce senza necessità di sincronizzazione con altri sviluppatori; sono stati inoltre organizzati confronti periodici con il tutor aziendale per validare le scelte effettuate.

3. Analisi dei requisiti

Questo capitolo presenta gli use case rilevanti per le attività di ottimizzazione ed evoluzione svolte sul firmware embedded del dispositivo EggLog Motion Core, dell'applicazione mobile e del nodo sniffer can (d'ora in poi rispettivamente solo EggLog e EggLog App e EggLog CAN Sniffer) durante il periodo di tirocinio, insieme alla relativa analisi dei requisiti.

3.1 Stato attuale del firmware

Allo stato attuale, il firmware di EggLog è in grado di acquisire i dati provenienti dai sensori di bordo (*IMU_G*, magnetometro, barometro e modulo GPS). I dati acquisiti vengono temporaneamente memorizzati in buffer in RAM secondo uno schema di *ping-pong buffering_G* e successivamente scritti in formato testuale (*CSV_G*) su scheda microSD, organizzati in file distinti all'interno di cartelle dedicate a ciascun sensore.

Il dispositivo espone inoltre un web server locale, raggiungibile via Wi-Fi, che consente all'utente di configurare i parametri di funzionamento, scaricare i dati archiviati in un intervallo temporale specifico e rimuoverli dalla scheda microSD.

3.2 Analisi degli utenti

Prima di affrontare le fasi di progettazione e sviluppo, è opportuno individuare a chi si rivolge il prodotto finale, soprattutto quando questo non è ancora disponibile sul mercato. L'azienda ha già condotto uno studio preliminare del pubblico di riferimento, individuando tre principali categorie di utenti:

- i **centri di *coaching***, che possono impiegare i dati sensoriali raccolti come supporto didattico per il miglioramento tecnico dei piloti.
- i **team organizzatori di eventi motociclistici**, interessati a raccogliere dati telemetrici dettagliati sui partecipanti alle gare;
- i **piloti semi-professionisti**, che possono sfruttare EggLog per valutare le proprie prestazioni durante sessioni di allenamento o competizioni ufficiali;

3.3 Lista degli *use case*

Ciascun caso d'uso è identificato da un codice seguito dalla lettera **F**, **A** oppure **S**, che ne indica rispettivamente l'appartenenza al **firmware** del dispositivo EggLog, all'**applicazione mobile** EggLog App, oppure al nodo **EggLog CAN Sniffer**: un dispositivo separato, privo di connessione fisica a EggLog, dedicato alla lettura del bus CAN del veicolo e alla trasmissione dei dati telemetrici verso EggLog tramite BLE. I casi d'uso relativi ai tre componenti sono presentati in un ordine unico, in modo da mantenere adiacenti quelli tra

loro correlati anche quando appartengono a sistemi diversi. La descrizione testuale deve contenere le seguenti informazioni:

- **Attore Primario:** rappresenta il ruolo che un'entità esterna al sistema assume quando interagisce con esso per raggiungere un obiettivo.
- **Precondizioni:** condizioni che devono essere vere nello stato del sistema prima che il caso d'uso inizi la sua esecuzione.
- **Postcondizioni:** condizioni che devono essere vere nello stato del sistema dopo che il caso d'uso è terminato.
- **Flusso Principale:** interazioni tra attore e sistema che portano al raggiungimento dell'obiettivo del caso d'uso con successo.
- **Flusso Alternativo:** variazioni rispetto al flusso principale.
- **Inclusioni:** riferimenti ai casi d'uso inclusi.
- **Estensioni:** riferimenti ai casi d'uso di estensione.

3.3.1 UC1-F - Acquisizione binaria dei dati sensore

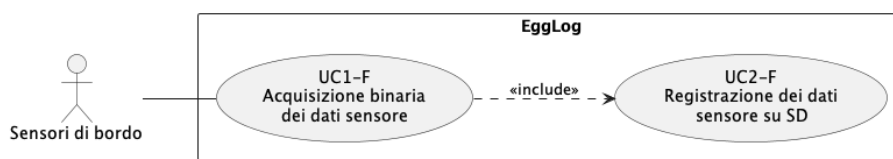


Figura 3.1: UC1-F - Acquisizione binaria dei dati sensore

- **Attore Primario:** sensori di bordo.
- **Precondizioni:**
 - I sensori sono stati inizializzati e, se necessario, calibrati (cfr. [UC16-F](#), [UC20-F](#));
 - è in corso una sessione di registrazione attiva (cfr. [UC15-F](#)).
- **Postcondizioni:** il buffer in RAM contiene i campioni acquisiti, serializzati in formato binario, pronti per essere scritti su SD.
- **Flusso Principale:**
 1. il firmware recupera un nuovo campione da ciascun sensore attivo, secondo la frequenza di campionamento configurata;
 2. ciascun sensore restituisce il valore misurato;
 3. il firmware associa a ogni campione il timestamp corrente;
 4. il firmware serializza il campione in formato binario nativo;
 5. il firmware inserisce il campione serializzato nel buffer in RAM;
 6. se il buffer ha raggiunto la soglia di scrittura, il firmware avvia [UC2-F](#).
- **Inclusioni:** [UC2-F](#).

3.3.2 UC2-F - Registrazione dei dati binari dei sensori su SD

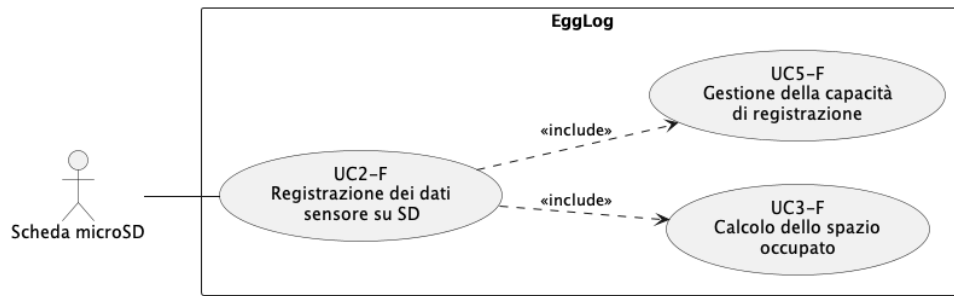


Figura 3.2: Inclusioni UC2-F - Registrazione dei dati sensore su SD

- **Attore Primario:** scheda microSD.
- **Precondizioni:**
 - il buffer di acquisizione (cfr. UC1-F) contiene dei dati binari pronti per la scrittura;
 - la scheda microSD è presente e correttamente montata;
 - è stata creata la cartella dedicata alla sessione corrente (cfr. UC21-F), contenente i file destinati a ciascun sensore.
- **Postcondizioni:** i dati sono persistiti nel file dedicato al sensore all'interno della cartella di sessione e il buffer in RAM viene svuotato.
- **Flusso Principale:**
 1. il firmware verifica che il buffer abbia raggiunto la soglia di scrittura;
 2. il firmware accede al file dedicato al sensore all'interno della cartella di sessione corrente;
 3. il firmware scrive il contenuto del buffer in coda al file;
 4. il firmware svuota il buffer in RAM;
 5. il firmware calcola lo spazio occupato dal file (cfr. UC3-F) e ne verifica la capacità (cfr. UC5-F);
 6. il firmware conferma l'avvenuta scrittura.
- **Flusso Alternativo:**
 - se la scheda microSD non è presente, la scrittura viene rimandata in attesa del reinserimento della scheda, mentre il firmware ne segnala l'assenza via BLE all'applicazione (cfr. UC8-F);
 - se la scrittura fallisce (SD piena o guasta), il firmware segnala l'errore (cfr. UC12.2-F).
- **Inclusioni:**
 - UC3-F;

– UC5-F.

- **Estensioni:**

– UC8-F;

– UC12.2-F.

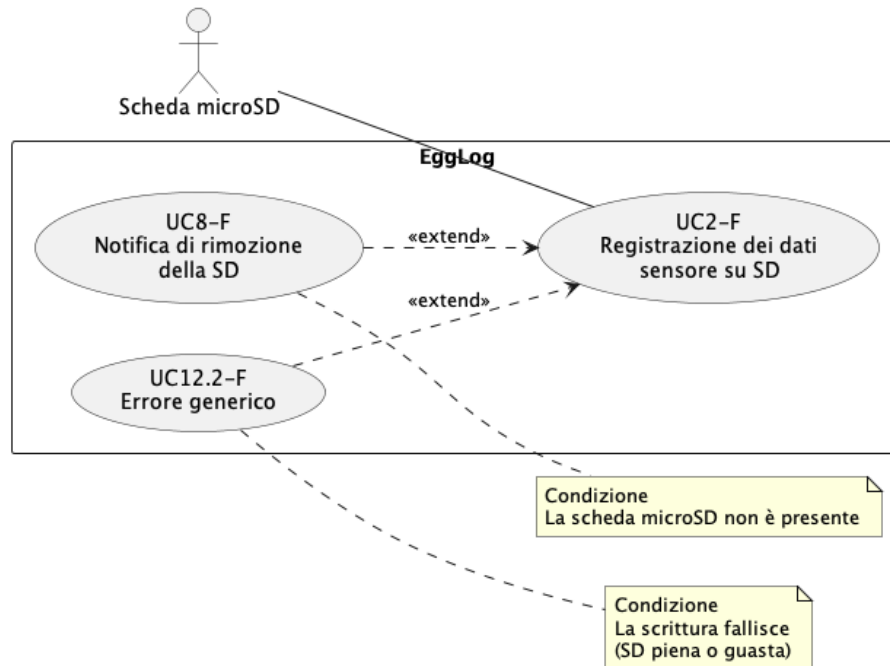


Figura 3.3: UC2-F - Registrazione dei dati sensore su SD

3.3.3 UC3-F - Calcolo dello spazio occupato dai dati registrati

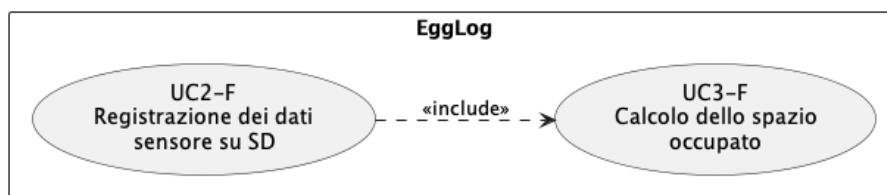


Figura 3.4: UC3-F - Calcolo dello spazio occupato dai dati registrati

- **Attore Primario:** nessuno; il caso d'uso è innescato internamente da UC2-F.
- **Precondizioni:** è stata completata una scrittura di dati sul file dedicato a un sensore.
- **Postcondizioni:** lo spazio occupato dai dati all'interno del file relativo al sensore viene calcolato in modo da poterlo inviare all'applicazione via BLE (cfr. UC4-A) e per la verifica dei limiti di capacità (cfr. UC5-F).
- **Flusso Principale:**

1. il firmware calcola la quantità spazio occupato dai dati scritti nel file relativo al sensore e lo converte in percentuale;
2. il firmware invia all'applicazione la percentuale di spazio occupato associato al sensore.

3.3.4 UC4-A - Visualizzazione dello spazio di archiviazione occupato

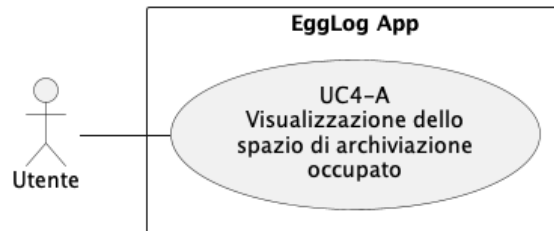


Figura 3.5: UC4-A - Visualizzazione dello spazio di archiviazione occupato

- **Attore Primario:** utente.
- **Precondizioni:**
 - è in corso una sessione di registrazione attiva;
 - il firmware ha calcolato lo spazio occupato per ciascun sensore (cfr. [UC3-F](#)).
- **Postcondizioni:** l'utente visualizza la percentuale di spazio occupato da ogni sensore all'interno del proprio file.
- **Flusso Principale:**
 1. l'applicazione riceve periodicamente lo stato di occupazione dello spazio dal firmware;
 2. l'applicazione effettua un controllo sui limiti delle percentuali di spazio occupato;
 3. l'applicazione mostra all'utente la percentuale di spazio occupato.
- **Flusso Alternativo:**
 - se per un sensore diverso dal GPS lo spazio disponibile ha raggiunto il limite e sono in corso sovrascritture dei dati meno recenti, l'applicazione mostra, sotto la percentuale, l'indicazione *"rewriting"*.

3.3.5 UC5-F - Gestione della capacità di registrazione e blocco automatico

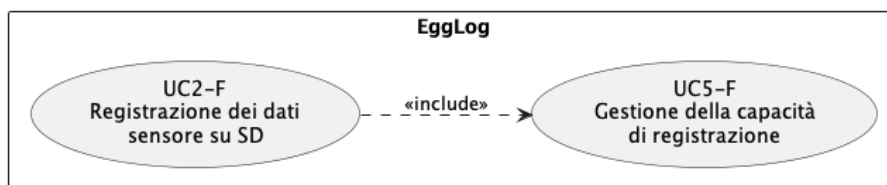


Figura 3.6: UC5-F - Gestione della capacità di registrazione e blocco automatico

- **Attore Primario:** nessuno; il caso d'uso è innescato internamente da [UC2-F](#).
- **Precondizioni:** lo spazio occupato dal file relativo a un sensore è stato calcolato (cfr. [UC3-F](#)).
- **Postcondizioni:** la sessione risulta interrotta oppure i dati meno recenti del sensore risultano sovrascritti, in base al fatto che il GPS abbia o meno raggiunto la capacità massima di archiviazione dentro al proprio file.
- **Flusso Principale:**
 1. il firmware verifica se lo spazio occupato dal file del sensore ha raggiunto il limite previsto per quel sensore;
 2. se il sensore è il modulo GPS, il firmware interrompe automaticamente la sessione di registrazione e lo notifica all'applicazione (cfr. [UC6-A](#));
 3. se il sensore invece è l'IMU, il barometro oppure il magnetometro, il firmware prosegue la registrazione sovrascrivendo i dati meno recenti presenti nel file.

3.3.6 UC6-A - Notifica di sessione bloccata per spazio GPS esaurito

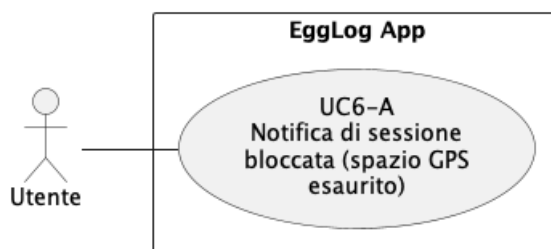


Figura 3.7: UC6-A - Notifica di sessione bloccata per spazio GPS esaurito

- **Attore Primario:** utente.
- **Precondizioni:** il file dedicato al modulo GPS ha raggiunto il limite massimo di spazio previsto (cfr. [UC5-F](#)).
- **Postcondizioni:** la sessione di registrazione risulta interrotta e l'utente è stato informato del motivo dell'interruzione.

- **Flusso Principale:**

1. il firmware notifica all'applicazione l'interruzione della sessione;
2. l'applicazione mostra all'utente un messaggio che indica il raggiungimento della capacità massima prevista per i dati GPS.

3.3.7 UC7-F - Recupero della sessione dopo rimozione SD

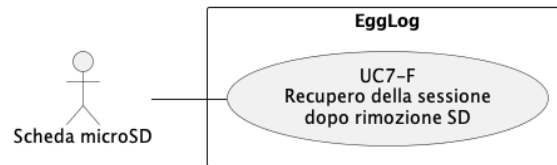


Figura 3.8: UC7-F - Recupero della sessione dopo rimozione SD

- **Attore Primario:** scheda microSD.
- **Precondizioni:** era in corso una sessione di registrazione, interrotta in modo imprevisto a causa della rimozione della scheda microSD.
- **Postcondizioni:** la registrazione riprende dal punto corretto all'interno dei file di sessione, senza perdita né sovrascrittura dei dati già acquisiti.
- **Flusso Principale:**
 1. al reinserimento della scheda microSD, il firmware rileva la presenza di una sessione precedentemente interrotta;
 2. il firmware individua, per ciascun file di sessione, l'ultima posizione di scrittura valida;
 3. il firmware ripristina lo stato di scrittura a partire da tale posizione;
 4. il firmware riprende la registrazione dei dati in coda ai file esistenti.

3.3.8 UC8-F - Notifica di rimozione della scheda microSD

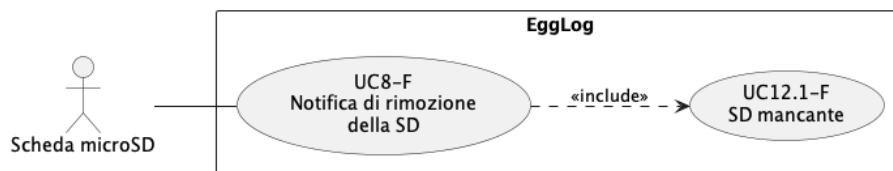


Figura 3.9: UC8-F - Notifica di rimozione della scheda microSD

- **Attore Primario:** scheda microSD.
- **Precondizioni:** è in corso, oppure si sta tentando di avviare, una sessione di registrazione.
- **Postcondizioni:** l'applicazione è stata informata dell'assenza della scheda microSD.

- **Flusso Principale:**

1. il firmware rileva l'assenza della scheda microSD;
2. il firmware notifica lo stato di assenza della scheda all'applicazione;
3. il firmware applica il pattern LED corrispondente (cfr. [UC12.1-F](#)).

- **Inclusioni:** [UC12.1-F](#).

3.3.9 UC9-A - Gestione della rimozione SD durante la sessione

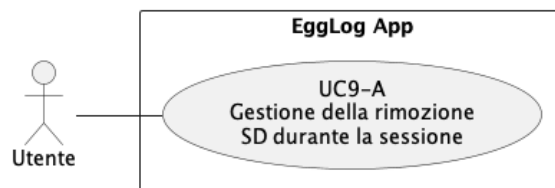


Figura 3.10: UC9-A - Gestione della rimozione SD durante la sessione

- **Attore Primario:** utente.

- **Precondizioni:** è in corso una sessione di registrazione attiva e il firmware ha notificato l'assenza della scheda microSD (cfr. [UC8-F](#)).

- **Postcondizioni:** la sessione risulta sospesa oppure interrotta, in base alla scelta dell'utente o al successivo reinserimento della scheda.

- **Flusso Principale:**

1. l'applicazione riceve la notifica di assenza della scheda microSD;
2. l'applicazione sospende la sessione e mostra il messaggio di scheda microSD mancante, offrendo la possibilità di interromperla manualmente;
3. l'utente reinserisce la scheda microSD;
4. l'applicazione rileva il ripristino della scheda, chiude il messaggio e la sessione riprende normalmente.

- **Flusso Alternativo:**

- se l'utente preme il tasto per interrompere la sessione, la sessione viene terminata definitivamente.

3.3.10 UC10-A - Gestione dell'assenza della SD all'avvio sessione

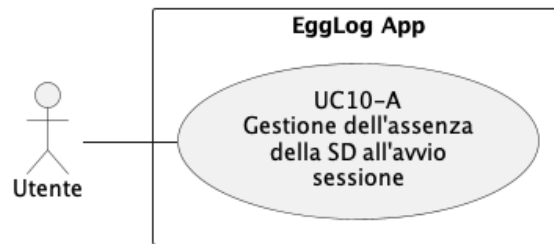


Figura 3.11: UC10-A - Gestione dell'assenza della SD all'avvio sessione

- **Attore Primario:** utente.
- **Precondizioni:** l'utente ha richiesto l'avvio di una nuova sessione di registrazione e il firmware ha notificato l'assenza della scheda microSD (cfr. [UC8-F](#)).
- **Postcondizioni:** la sessione viene avviata solo dopo che la scheda microSD risulta correttamente inserita.
- **Flusso Principale:**
 1. l'utente richiede l'avvio di una nuova sessione;
 2. l'applicazione riceve la notifica di assenza della scheda microSD;
 3. l'applicazione mostra il messaggio di scheda microSD mancante con il pulsante per ritentare l'avvio della sessione;
 4. l'utente inserisce la scheda microSD e ritenta l'avvio della sessione;
 5. il firmware verifica la presenza della scheda e, in parallelo con l'applicazione, avvia la sessione (cfr. [UC15-F](#)).
- **Flusso Alternativo:**
 - se la scheda risulta ancora assente, l'applicazione mostra nuovamente il messaggio di scheda microSD mancante con il pulsante per ritentare l'avvio della sessione.

3.3.11 UC11-S - Acquisizione dei dati dal bus CAN e trasmissione in broadcast BLE

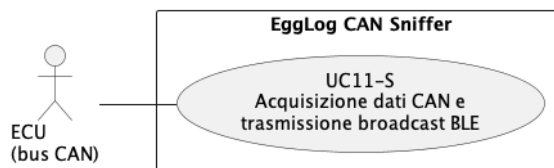


Figura 3.12: UC11-S - Acquisizione dei dati dal bus CAN e trasmissione in broadcast BLE

- **Attore Primario:** ECU del veicolo connesso mediante bus CAN.

- **Precondizioni:** il nodo EggLog CAN Sniffer è collegato alla presa diagnostica del veicolo e alimentato.
- **Postcondizioni:** i parametri del veicolo decodificati sono pubblicati in broadcast su BLE.
- **Flusso Principale:**
 1. il nodo EggLog CAN Sniffer si mette in ascolto sul bus CAN;
 2. la ECU trasmette periodicamente i messaggi contenenti i parametri motore;
 3. il nodo ne decodifica i valori e li inserisce in un pacchetto di advertising BLE;
 4. il nodo pubblica il pacchetto in broadcast su BLE.
- **Flusso Alternativo:**
 - se il frame CAN ricevuto non è riconosciuto o risulta malformato, viene scartato senza alcun effetto sullo stato pubblicato;

3.3.12 UC11-F - Ricezione dei dati di telemetria CAN via BLE broadcast

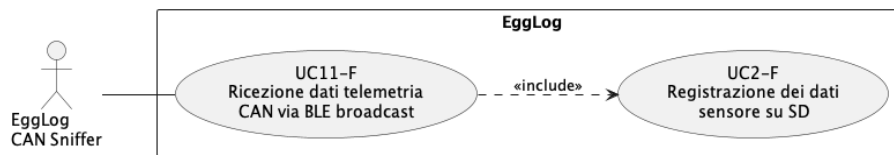


Figura 3.13: UC11-F - Ricezione dei dati di telemetria CAN via BLE broadcast

- **Attore Primario:** nodo EggLog CAN Sniffer.
- **Precondizioni:**
 - il nodo EggLog CAN Sniffer è alimentato, collegato alla presa diagnostica del veicolo e sta trasmettendo in broadcast BLE i pacchetti di telemetria (cfr. [UC11-S](#));
 - il firmware è avviato ed è in corso una sessione di registrazione attiva.
- **Postcondizioni:** i dati di telemetria motore ricevuti sono resi disponibili alla registrazione su scheda microSD (cfr. [UC2-F](#)).
- **Flusso Principale:**
 1. il firmware esegue una scansione BLE passiva continua;
 2. il firmware individua, tra i pacchetti di advertising ricevuti, quello corrispondente al nodo EggLog CAN Sniffer;
 3. il firmware associa al pacchetto la marca temporale di ricezione;
 4. il firmware rende disponibili i dati CAN per la registrazione su scheda microSD.

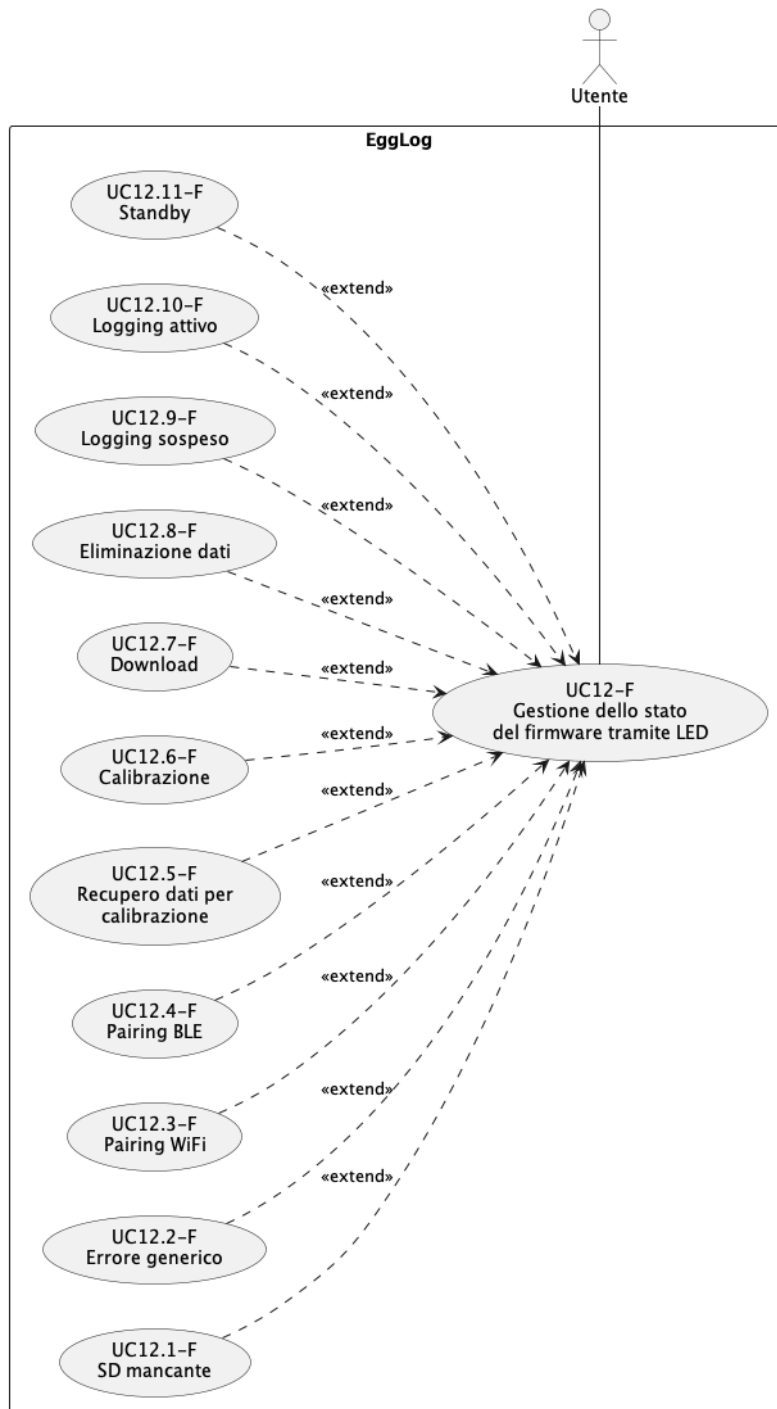
3.3.13 UC12-F - Gestione dello stato del firmware tramite *LED_G*

Figura 3.14: UC12-F - Gestione dello stato del firmware tramite LED

- **Attore Primario:** utente.
- **Precondizioni:** il dispositivo è alimentato e il firmware è in esecuzione.
- **Postcondizioni:** il LED riflette lo stato operativo corrente del dispositivo, secondo un ordine di priorità tra le condizioni concorrenti.
- **Flusso Principale:**

1. il firmware rileva un cambiamento nello stato operativo del dispositivo;
 2. il firmware seleziona il pattern di colore/lampeggio associato alla condizione a priorità più alta tra quelle attive;
 3. il firmware applica il pattern al LED;
 4. l'utente osserva il LED e ne interpreta lo stato del dispositivo.
- **Flusso Alternativo:**
 - se si verificano più condizioni contemporaneamente, il firmware seleziona il pattern relativo alla condizione a priorità più alta.
 - **Estensioni:**
 - UC12.1-F;
 - UC12.2-F;
 - UC12.3-F;
 - UC12.4-F;
 - UC12.5-F;
 - UC12.6-F;
 - UC12.7-F;
 - UC12.8-F;
 - UC12.9-F;
 - UC12.10-F;
 - UC12.11-F.

3.3.13.1 UC12.1-F - Segnalazione di scheda microSD mancante

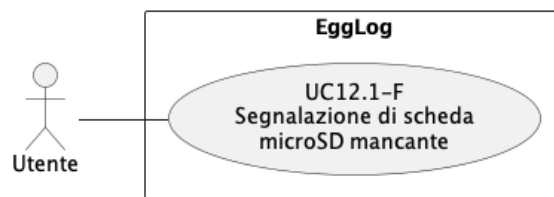
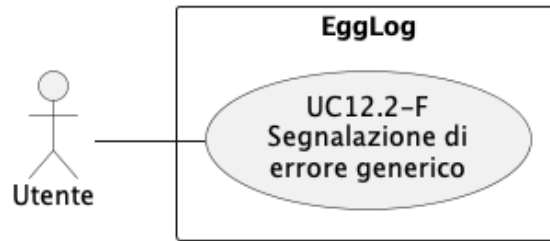
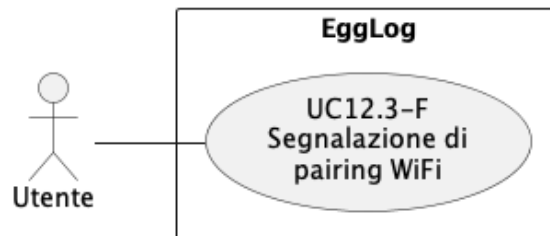


Figura 3.15: UC12.1-F - Segnalazione di scheda microSD mancante

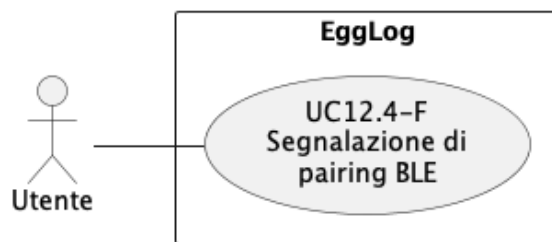
- **Attore Primario:** utente.
- **Precondizioni:** la scheda microSD risulta assente (cfr. UC8-F) e si sta tentando di avviare una sessione oppure si è in piena sessione di registrazione.
- **Postcondizioni:** il LED mostra il pattern rosso con lampeggio veloce, con priorità massima su ogni altro stato.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l'assenza della scheda microSD;
 2. il modulo applica il pattern rosso con lampeggio veloce;
 3. il pattern resta attivo finché la scheda non viene rilevata correttamente.

3.3.13.2 UC12.2-F - Segnalazione di errore generico**Figura 3.16:** UC12.2-F - Segnalazione di errore generico

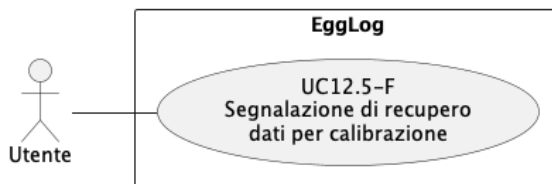
- **Attore Primario:** utente.
- **Precondizioni:** si verifica una condizione di errore o malfunzionamento generica.
- **Postcondizioni:** il LED mostra il pattern rosso fisso, con priorità su ogni stato successivo.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva la condizione di errore;
 2. il modulo applica il pattern rosso fisso;
 3. il pattern resta attivo finché l'errore non viene risolto o il dispositivo riavviato.

3.3.13.3 UC12.3-F - Segnalazione di pairing Wi-Fi**Figura 3.17:** UC12.3-F - Segnalazione di pairing Wi-Fi

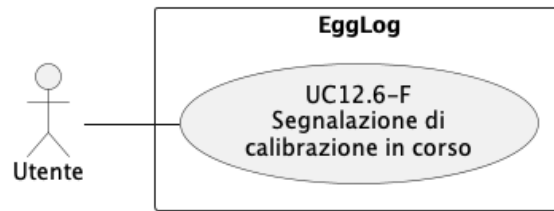
- **Attore Primario:** utente.
- **Precondizioni:** è in corso una procedura di associazione *pairing* con una rete Wi-Fi.
- **Postcondizioni:** il LED mostra il pattern blu con lampeggio veloce.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l'avvio della procedura di pairing Wi-Fi;
 2. il modulo applica il pattern blu con lampeggio veloce;
 3. il pattern rimane attivo per l'intera durata del pairing.

3.3.13.4 UC12.4-F - Segnalazione di pairing BLE**Figura 3.18:** UC12.4-F - Segnalazione di pairing BLE

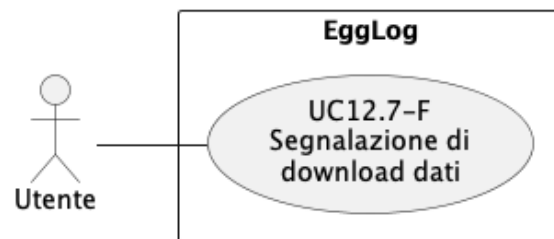
- **Attore Primario:** utente.
- **Precondizioni:** è in corso una procedura di associazione (*pairing*) con un dispositivo tramite BLE.
- **Postcondizioni:** il LED mostra il pattern blu con lampeggio lento.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l'avvio della procedura di pairing BLE;
 2. il modulo applica il pattern blu con lampeggio lento;
 3. il pattern rimane attivo per l'intera durata del pairing.

3.3.13.5 UC12.5-F - Segnalazione di recupero dati per calibrazione**Figura 3.19:** UC12.5-F - Segnalazione di recupero dati per calibrazione

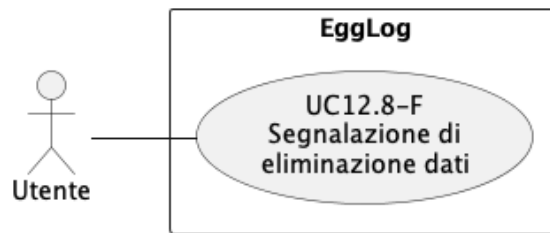
- **Attore Primario:** utente.
- **Precondizioni:** è in corso l'acquisizione dei campioni necessari a una procedura di calibrazione (cfr. [UC16-F](#)).
- **Postcondizioni:** il LED mostra il pattern giallo fisso.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l'avvio dell'acquisizione dei campioni di calibrazione;
 2. il modulo applica il pattern giallo fisso;
 3. il pattern rimane attivo per l'intera durata della fase di acquisizione dei dati di calibrazione.

3.3.13.6 UC12.6-F - Segnalazione di calibrazione in corso**Figura 3.20:** UC12.6-F - Segnalazione di calibrazione in corso

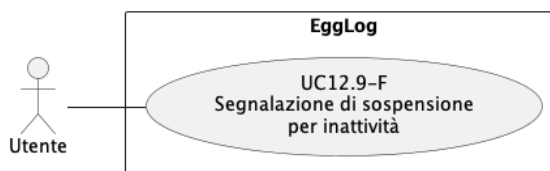
- **Attore Primario:** utente.
- **Precondizioni:** è in corso l'elaborazione dei dati di calibrazione.
- **Postcondizioni:** il LED mostra il pattern giallo pulsante (effetto *breathing*).
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l'avvio dell'elaborazione dei dati di calibrazione;
 2. il modulo applica il pattern giallo pulsante;
 3. il pattern rimane attivo finché la calibrazione non è completata.

3.3.13.7 UC12.7-F - Segnalazione di download dati**Figura 3.21:** UC12.7-F - Segnalazione di download dati

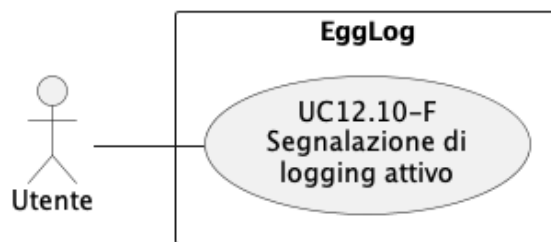
- **Attore Primario:** utente.
- **Precondizioni:** è in corso il download dei dati registrati verso l'applicazione.
- **Postcondizioni:** il LED mostra il pattern viola fisso.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l'avvio del download dei dati;
 2. il modulo applica il pattern viola fisso;
 3. il pattern rimane attivo per l'intera durata del download.

3.3.13.8 UC12.8-F - Segnalazione di eliminazione dati**Figura 3.22:** UC12.8-F - Segnalazione di eliminazione dati

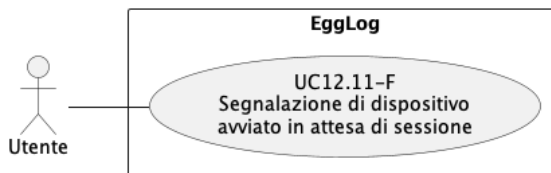
- **Attore Primario:** utente.
- **Precondizioni:** è in corso l'eliminazione dei dati registrati dalla scheda microSD.
- **Postcondizioni:** il LED mostra il pattern viola con lampeggio regolare.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l'avvio dell'eliminazione dei dati;
 2. il modulo applica il pattern viola con lampeggio regolare;
 3. il pattern rimane attivo per l'intera durata dell'eliminazione.

3.3.13.9 UC12.9-F - Segnalazione di sospensione per inattività**Figura 3.23:** UC12.9-F - Segnalazione di sospensione per inattività

- **Attore Primario:** utente.
- **Precondizioni:** si verifica la sospensione automatica dell'acquisizione sensori descritta in [UC13-F](#).
- **Postcondizioni:** il LED mostra il pattern verde pulsante lento (effetto *breathing*).
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva la sospensione dell'acquisizione sensori;
 2. il modulo applica il pattern verde pulsante lento;
 3. il pattern rimane attivo finché l'acquisizione sensori resta sospesa.

3.3.13.10 UC12.10-F - Segnalazione di logging attivo**Figura 3.24:** UC12.10-F - Segnalazione di logging attivo

- **Attore Primario:** utente.
- **Precondizioni:** è in corso una regolare sessione di registrazione dati (cfr. [UC15-F](#)).
- **Postcondizioni:** il LED mostra il pattern verde fisso.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l'avvio, o la ripresa, della sessione di registrazione;
 2. il modulo applica il pattern verde fisso;
 3. il pattern rimane attivo per l'intera durata della sessione.

3.3.13.11 UC12.11-F - Segnalazione di dispositivo avviato in attesa di sessione**Figura 3.25:** UC12.11-F - Segnalazione di dispositivo avviato in attesa di sessione

- **Attore Primario:** utente.
- **Precondizioni:** il dispositivo è stato avviato correttamente, oppure una sessione precedente è appena terminata (cfr. [UC15-F](#)), e non è attiva alcuna delle altre condizioni.
- **Postcondizioni:** il LED mostra il pattern bianco fisso.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l'assenza di ogni altra condizione a priorità più alta;
 2. il modulo applica il pattern bianco fisso;
 3. il pattern resta attivo finché non si presenta una condizione a priorità più alta.

3.3.14 UC13-F - Sospensione dell'acquisizione sensori in stato di inattività

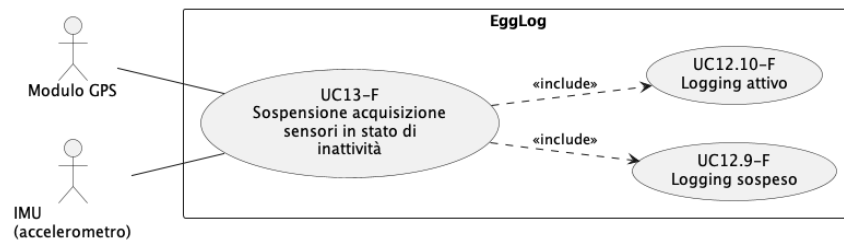


Figura 3.26: UC13-F - Sospensione dell'acquisizione sensori in stato di inattività

- **Attore Primario:** modulo GPS
- **Attore Secondario:** IMU (accelerometro).
- **Precondizioni:** è in corso una sessione di registrazione attiva.
- **Postcondizioni:** l'acquisizione dei sensori non essenziali risulta sospesa o riattivata in base allo stato di movimento rilevato.
- **Flusso Principale:**
 1. se il modulo GPS dispone di un *fix* valido, il firmware monitora la velocità rilevata dal GPS; in caso contrario monitora i dati dell'accelerometro dell'IMU;
 2. il firmware verifica se non si sono registrate variazioni significative per l'intervallo di tempo configurato;
 3. al superamento dell'intervallo, il firmware sospende l'acquisizione dei sensori non essenziali;
 4. il firmware applica il pattern LED di sospensione (cfr. [UC12.9-F](#));
 5. il firmware continua a monitorare la sorgente disponibile (GPS o IMU) per rilevare un nuovo movimento;
 6. al rilevamento di movimento, il firmware riattiva l'acquisizione e applica [UC12.10-F](#) per ripristinare il pattern di logging attivo.
- **Flusso Alternativo:**
 - se la sospensione automatica è stata disabilitata dall'utente (cfr. [UC14-A](#)), il firmware non sospende mai l'acquisizione, indipendentemente dallo stato di movimento rilevato.
- **Inclusioni:**
 - [UC12.9-F](#);
 - [UC12.10-F](#).

3.3.15 UC14-A - Configurazione del tempo di inattività

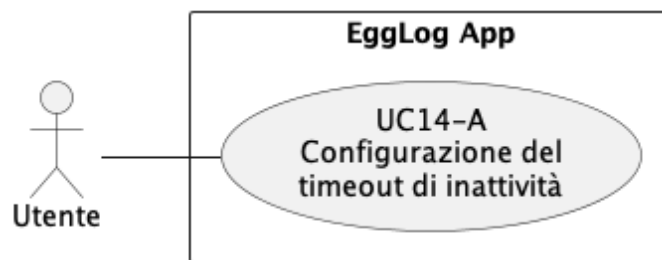


Figura 3.27: UC14-A - Configurazione del tempo di inattività

- **Attore Primario:** utente.
- **Precondizioni:** è stabilita una connessione BLE tra dispositivo e applicazione.
- **Postcondizioni:** il valore configurato per il tempo di inattività è salvato e applicato alla sospensione automatica dell'acquisizione (cfr. [UC13-F](#)).
- **Flusso Principale:**
 1. l'utente seleziona uno dei valori disponibili per il tempo di inattività;
 2. l'applicazione trasmette il valore selezionato al firmware;
 3. il firmware salva la nuova configurazione e la applica alle sessioni successive.

3.3.16 UC15-F - Avvio e interruzione sessione tramite comandi BLE

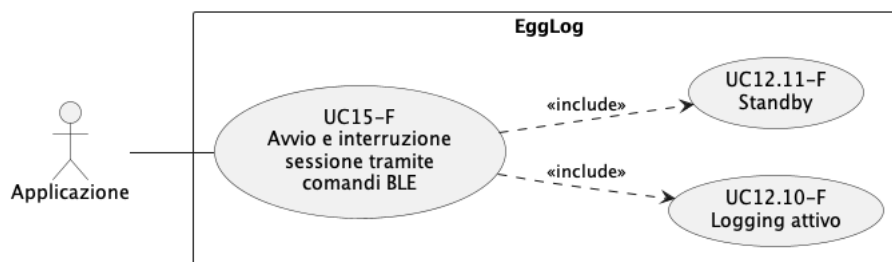


Figura 3.28: UC15-F - Avvio e interruzione sessione tramite comandi BLE

- **Attore Primario:** applicazione.
- **Precondizioni:** è stabilita una connessione BLE tra dispositivo e applicazione.
- **Postcondizioni:** la sessione di registrazione risulta avviata o interrotta in base al comando ricevuto; i sensori non essenziali restano dormienti al di fuori di una sessione attiva.
- **Flusso Principale:**
 1. il firmware riceve, tramite BLE, un comando di avvio o interruzione sessione;

2. se il comando è di avvio, il firmware attiva i sensori, crea la cartella di sessione (cfr. UC21-F), esegue le calibrazioni automatiche previste (cfr. UC20-F) e applica il pattern LED di logging attivo (cfr. UC12.10-F);
 3. se il comando è di interruzione, il firmware termina la registrazione, pone i sensori non essenziali in stato dormiente e applica il pattern LED di attesa sessione (cfr. UC12.11-F).
- **Inclusioni:**
 - UC12.10-F;
 - UC12.11-F.

3.3.17 UC16-F - Calibrazione dell'IMU

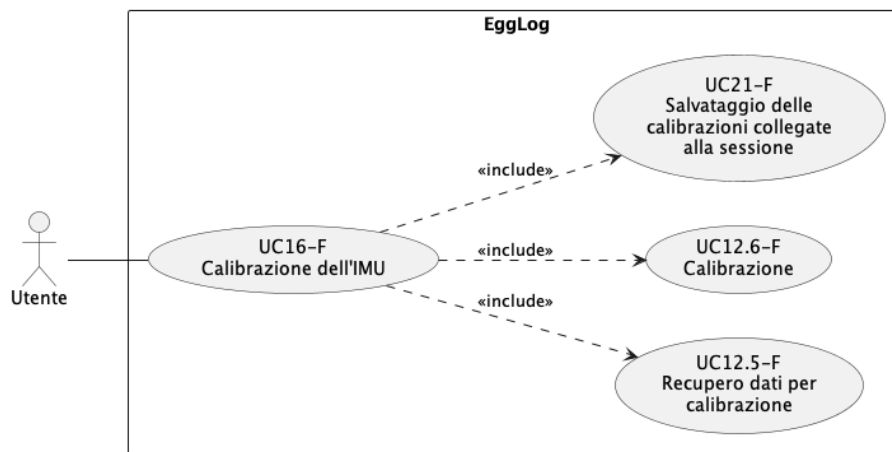


Figura 3.29: UC16-F - Calibrazione dell'IMU

- **Attore Primario:** utente.
- **Precondizioni:** è stabilita una connessione BLE tra dispositivo e applicazione.
- **Postcondizioni:** i parametri di calibrazione, calcolati a partire dai dati di giroscopio e accelerometro dell'IMU, sono salvati (cfr. UC21-F) e applicati alle letture sensore successive, indipendentemente dal posizionamento del dispositivo sul veicolo.
- **Flusso Principale:**
 1. il firmware avvia la procedura di calibrazione, guidando l'utente attraverso una sequenza di movimenti (cfr. UC17-A);
 2. per ciascun movimento, il firmware acquisisce i campioni di giroscopio e accelerometro, applicando il pattern LED corrispondente (cfr. UC12.5-F);
 3. il firmware elabora i dati raccolti applicando il pattern LED corrispondente (cfr. UC12.6-F);
 4. il firmware applica la matrice di calibrazione e i dati di calibrazione alle acquisizioni successive (cfr. UC1-F).

- **Inclusioni:**
 - UC12.5-F;
 - UC12.6-F;
 - UC21-F.

3.3.18 UC17-A - Sequenza guidata di calibrazione dell'IMU

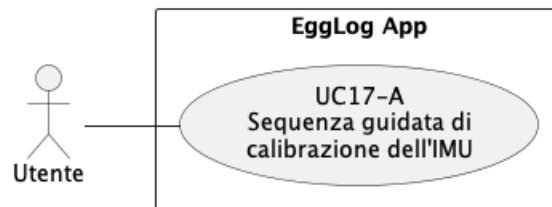


Figura 3.30: UC17-A - Sequenza guidata di calibrazione dell'IMU

- **Attore Primario:** utente.
- **Precondizioni:** è stata avviata una procedura di calibrazione dell'IMU (cfr. UC16-F).
- **Postcondizioni:** l'utente ha completato correttamente i tre movimenti richiesti dalla procedura di calibrazione.
- **Flusso Principale:**
 1. l'applicazione richiede all'utente di tenere la moto ferma e perfettamente in piedi;
 2. l'applicazione richiede all'utente di inclinare la moto verso destra;
 3. l'applicazione richiede all'utente di camminare per 2-3 metri in avanti con la moto;
 4. al termine dei tre passaggi, l'applicazione mostra l'esito della calibrazione.
- **Flusso Alternativo:**
 - se i dati raccolti durante uno dei tre passaggi risultano poco significativi o non conformi rispetto al passaggio precedente, l'applicazione mostra un messaggio che richiede la ripetizione del movimento corrente.

3.3.19 UC18-A - Visualizzazione dei dati calibrati

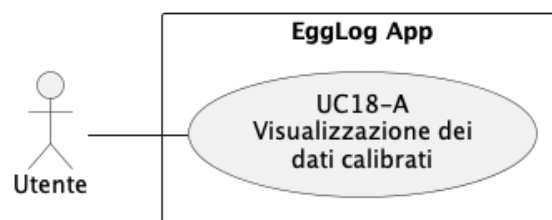


Figura 3.31: UC18-A - Visualizzazione dei dati calibrati

- **Attore Primario:** utente.
- **Precondizioni:**
 - l'utente ha avviato una sessione;
 - sono state completate le calibrazioni previste per la sessione (cfr. [UC16-F](#), [UC20-F](#)).
- **Postcondizioni:** l'utente visualizza i dati sensore corretti secondo i parametri di calibrazione applicati.
- **Flusso Principale:**
 1. il firmware trasmette all'applicazione i dati sensore già calibrati (cfr. [UC21-F](#));
 2. l'applicazione mostra i valori corretti all'utente.

3.3.20 UC19-A - Visualizzazione dei movimenti della moto in tempo reale

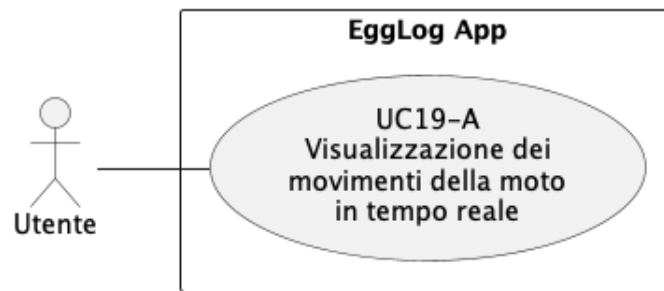


Figura 3.32: UC19-A - Visualizzazione dei movimenti della moto in tempo reale

- **Attore Primario:** utente.
- **Precondizioni:** è in corso una sessione di registrazione con calibrazione IMU completata (cfr. [UC16-F](#)).
- **Postcondizioni:** l'utente visualizza in tempo reale i movimenti della moto rilevati dall'IMU.
- **Flusso Principale:**
 1. il firmware trasmette in tempo reale all'applicazione i dati calibrati di inclinazione frontale, inclinazione laterale e accelerazione frontale;
 2. l'applicazione mostra tali valori numerici all'utente;
 3. l'applicazione applica gli stessi dati a un modello tridimensionale della moto, replicando i movimenti sui rispettivi assi X, Y e Z.

3.3.21 UC20-F - Calibrazione automatica dell'altitudine a inizio sessione

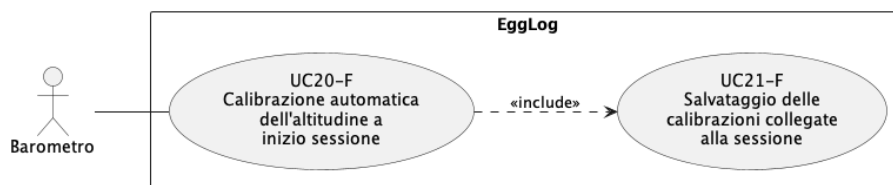


Figura 3.33: UC20-F - Calibrazione automatica dell'altitudine a inizio sessione

- **Attore Primario:** barometro.
- **Precondizioni:** è stata avviata una nuova sessione di registrazione (cfr. [UC15-F](#)).
- **Postcondizioni:** il valore di altitudine iniziale, calcolato tramite il barometro, è salvato e associato alla sessione corrente (cfr. [UC21-F](#)).
- **Flusso Principale:**
 1. all'avvio della sessione, il firmware acquisisce le letture del barometro;
 2. il firmware calcola il valore di altitudine di riferimento;
 3. il firmware applica tale valore come riferimento alle letture di altitudine successive.
- **Flusso Alternativo:** i dati raccolti dal barometro non risultano abbastanza stabili per cui la sessione si avvia senza una calibrazione dell'altitudine di partenza.
- **Inclusioni:** [UC21-F](#).

3.3.22 UC21-F - Salvataggio delle calibrazioni collegate alla sessione

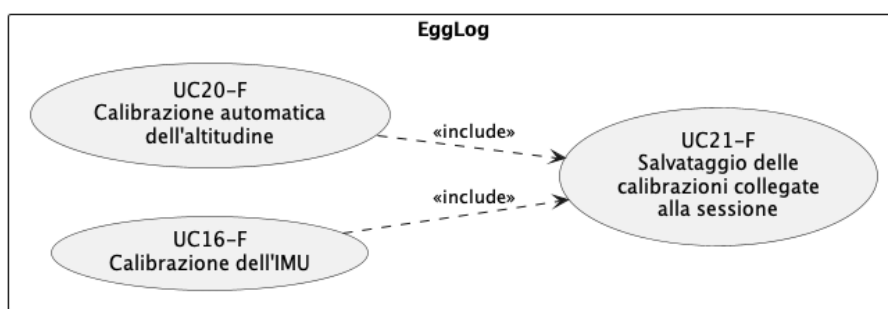


Figura 3.34: UC21-F - Salvataggio delle calibrazioni collegate alla sessione

- **Attore Primario:** nessuno; il caso d'uso è innescato internamente da [UC16-F](#) e da [UC20-F](#).
- **Precondizioni:** è stata completata una procedura di calibrazione (IMU o altitudine).

- **Postcondizioni:** i dati di calibrazione dell'altitudine sono salvati all'interno della cartella di sessione, mentre i dati di calibrazione dell'IMU sono salvati nella cartella principale contenente tutte le cartelle di sessione.
- **Flusso Principale:**
 1. il firmware crea, se non già presenti, la cartella dedicata al salvataggio delle sessioni e la cartella della sessione corrente;
 2. il firmware salva il file di calibrazione dell'altitudine all'interno della cartella di sessione, mentre salva il file di calibrazione dell'IMU nella cartella principale.

3.4 Tracciamento dei requisiti

In questa sezione si riporta la lista di requisiti rilevata in seguito all'analisi dell'utenza e degli use case; i requisiti sono identificati da un codice definito con la seguente nomenclatura:

R-(Tipo)-(Urgenza)-(Codice)

Dove:

- **Tipo:** F (Funzionale), N (Non funzionale) e V (Vincolo);
- **Urgenza:** Ob (Obbligatoria), De (Desiderabile) e Op (Opzionale);
- **Codice:** numero progressivo del requisito.

3.4.1 Requisiti funzionali

Nella Tabella 3.1 si riporta il tracciamento dei requisiti funzionali, ovvero le funzionalità e i servizi che il sistema deve offrire per essere conforme alle aspettative.

Codice	Descrizione	Fonti
R-F-Ob-1	EggLog deve acquisire e serializzare in formato binario i dati provenienti dai sensori	UC1-F
R-F-Ob-2	EggLog deve scrivere su scheda microSD i dati acquisiti in formato binario	UC2-F
R-F-Ob-3	EggLog deve calcolare e trasmettere all'applicazione, per ciascun sensore, la percentuale di spazio occupato nel relativo file di sessione	UC3-F
R-F-De-4	L'utente deve poter visualizzare in EggLog App la percentuale di spazio di archiviazione occupato da ciascun sensore, incluso lo stato di sovrascrittura in corso	UC4-A
Continua nella prossima pagina...		

Tabella 3.1 - Continuo della tabella

Codice	Descrizione	Fonti
R-F-Ob-5	EggLog deve interrompere automaticamente la sessione al raggiungimento della capacità massima dei dati GPS, mentre deve proseguire la registrazione sovrascrivendo i dati meno recenti per gli altri sensori	UC5-F
R-F-Ob-6	L'utente deve essere informato in EggLog App quando la sessione viene interrotta per esaurimento dello spazio dedicato ai dati GPS	UC6-A
R-F-Ob-7	EggLog deve poter riprendere una sessione interrotta in modo imprevisto (rimozione della scheda microSD oppure un guasto al modulo SD) senza perdita né sovrascrittura dei dati già acquisiti	UC7-F
R-F-Ob-8	EggLog deve rilevare e notificare a EggLog App l'assenza della scheda microSD	UC8-F
R-F-De-9	EggLog App deve sospendere la visualizzazione della sessione in corso quando la scheda microSD viene rimossa, offrendo la possibilità di interromperla manualmente o di riprenderla al reinserimento	UC9-A
R-F-De-10	EggLog App deve impedire l'avvio di una nuova sessione in assenza della scheda microSD, offrendo la possibilità di riprovare dopo il suo inserimento	UC10-A
R-F-Ob-11	Il nodo EggLog CAN Sniffer deve acquisire i dati esposti dalla centralina motore (ECU) tramite bus CAN e trasmetterli in broadcast BLE	UC11-S
R-F-Ob-12	EggLog deve ricevere, tramite scansione BLE passiva, i dati di telemetria motore trasmessi in broadcast dal nodo EggLog CAN Sniffer	UC11-F
R-F-Ob-13	EggLog deve marcare il dato di telemetria CAN con la marca temporale di ricezione e renderlo disponibile alla registrazione su scheda microSD	UC11-F
Continua nella prossima pagina...		

Tabella 3.1 - Continuo della tabella

Codice	Descrizione	Fonti
R-F-De-14	EggLog deve pilotare il LED integrato sulla scheda ESP32-S3 per segnalare, tramite pattern di colore e lampeggio distinti e reciprocamente esclusivi, lo stato corrente del dispositivo	UC12-F, UC12.1-F, UC12.2-F, UC12.3-F, UC12.4-F, UC12.5-F, UC12.6-F, UC12.7-F, UC12.8-F, UC12.9-F, UC12.10-F, UC12.11-F
R-F-De-15	EggLog deve sospendere automaticamente l'acquisizione dei sensori non essenziali in stato di inattività prolungata e riattivarla al rilevamento di movimento	UC13-F
R-F-Op-16	L'utente deve poter configurare da EggLog App il tempo di inattività che determina la sospensione automatica dei sensori (mai, 10, 30 o 60 minuti)	UC14-A
R-F-Ob-17	EggLog deve avviare e interrompere la sessione di registrazione in seguito a comandi BLE ricevuti da EggLog App	UC15-F
R-F-Ob-18	EggLog deve calibrare l'IMU guidando l'utente, tramite EggLog App, attraverso una sequenza di movimenti predefinita, applicando poi tali calibrazioni alle letture successive	UC16-F, UC17-A
R-F-De-19	L'utente deve poter visualizzare in EggLog App i dati sensore corretti secondo i parametri di calibrazione applicati	UC18-A
R-F-De-20	L'utente deve poter visualizzare in tempo reale, tramite un modello tridimensionale in EggLog App, i movimenti della moto rilevati dall'IMU (inclinazione frontale, inclinazione laterale e accelerazione frontale)	UC19-A
R-F-Ob-21	EggLog deve calcolare automaticamente, tramite il barometro, un valore di altitudine di riferimento a inizio sessione e applicarlo alle letture successive	UC20-F
Continua nella prossima pagina...		

Tabella 3.1 - Continuo della tabella

Codice	Descrizione	Fonti
R-F-Ob-22	EggLog deve salvare i dati di calibrazione dell'IMU nella cartella principale delle sessioni e i dati di calibrazione dell'altitudine all'interno della cartella della sessione corrente	UC21-F

Tabella 3.1: Tracciamento dei requisiti funzionali.

3.4.2 Requisiti non funzionali

Nella Tabella 3.2 si riporta il tracciamento dei requisiti non funzionali, ovvero l'insieme di criteri quantificabili di qualità che il sistema sviluppato deve rispettare per essere conforme alle aspettative e agli obiettivi preposti.

Codice	Descrizione	Fonti
R-N-Ob-1	Il consumo di RAM per il buffering dei dati sensore deve essere ridotto rispetto alla configurazione precedente del firmware	UC1-F
R-N-Ob-2	Il consumo di CPU del dispositivo deve essere ridotto rispetto alla configurazione precedente	UC1-F, UC13-F
R-N-Ob-3	Il ripristino di una sessione interrotta in modo imprevisto non deve comportare perdita né duplicazione dei dati già acquisiti	UC7-F
R-N-De-4	Il cambio di stato segnalato dal LED deve essere percepito come pressoché istantaneo dall'utente, e ogni pattern di colore/lampeggio deve risultare univocamente distinguibile dagli altri	UC12-F, UC12.1-F, UC12.2-F, UC12.3-F, UC12.4-F, UC12.5-F, UC12.6-F, UC12.7-F, UC12.8-F, UC12.9-F, UC12.10-F, UC12.11-F
R-N-De-5	L'accuratezza delle misurazioni successive alla calibrazione deve rientrare entro le tolleranze specifiche	UC16-F, UC20-F
Continua nella prossima pagina...		

Tabella 3.2 - Continuo della tabella

Codice	Descrizione	Fonti
R-N-De-6	Le notifiche relative a eventi critici (assenza scheda microSD, esaurimento spazio GPS) devono essere recapitate a EggLog App entro un tempo breve dal loro verificarsi	UC8-F, UC6-A, UC9-A, UC10-A
R-N-Op-7	L'aggiornamento dei dati di movimento mostrati sul modello tridimensionale in tempo reale deve risultare fluido e privo di percepibili ritardi	UC19-A
R-N-Ob-8	Il canale di trasmissione tra il nodo EggLog CAN Sniffer ed EggLog deve tollerare la perdita di singoli pacchetti di telemetria senza richiedere meccanismi di riconoscimento o ritrasmissione	UC11-S, UC11-F

Tabella 3.2: Tracciamento dei requisiti non funzionali.

3.4.3 Requisiti di vincolo

Nella Tabella 3.3 si riporta il tracciamento dei requisiti di vincolo, ovvero l'insieme delle limitazioni tecnologico-organizzative a cui il dispositivo deve sottostare per essere conforme alle aspettative e alle esigenze degli stakeholder.

Codice	Descrizione	Fonti
R-V-Ob-1	Il microcontrollore del dispositivo deve appartenere alla famiglia ESP32, nello specifico ESP32-S3	Tutti
R-V-Ob-2	Il firmware del dispositivo dev'essere scritto in linguaggio C/C++	Tutti
R-V-Ob-3	L'interfaccia del nodo EggLog CAN Sniffer con il bus CAN del veicolo deve essere compatibile con il protocollo CAN utilizzato dalla centralina (ECU) della moto	UC11-S
R-V-Ob-4	Il LED utilizzato per la segnalazione di stato deve essere quello integrato sulla scheda ESP32-S3	UC12-F
R-V-Ob-5	La comunicazione tra EggLog ed EggLog App per la configurazione, il controllo della sessione e lo scambio dati in tempo reale deve avvenire tramite BLE	UC14-A, UC15-F, UC16-F
R-V-Ob-6	Il nodo EggLog CAN Sniffer deve essere un dispositivo fisicamente separato da EggLog, privo di connessione elettrica diretta con esso	UC11-S
Continua nella prossima pagina...		

Tabella 3.3 - Continuo della tabella

Codice	Descrizione	Fonti
R-V-Ob-7	La comunicazione tra il nodo EggLog CAN Sniffer ed EggLog deve avvenire esclusivamente tramite BLE advertising broadcast	UC11-S, UC11-F
R-V-Ob-8	Il microcontrollore CAN del nodo EggLog CAN Sniffer deve operare in modalità di sola ricezione (<i>Listen-Only</i>), senza mai trasmettere bit di ACK o frame di errore sul bus del veicolo	UC11-S

Tabella 3.3: Tracciamento dei requisiti di vincolo.

3.4.4 Tracciamento Fonti - Requisiti

Nella Tabella 3.4 si riporta, per ciascun caso d'uso, l'elenco dei requisiti da esso derivati, a complemento del tracciamento inverso riportato nelle tabelle precedenti.

Fonte	Requisiti
UC1-F	R-F-Ob-1, R-N-Ob-1, R-N-Ob-2
UC2-F	R-F-Ob-2
UC3-F	R-F-Ob-3
UC4-A	R-F-De-4
UC5-F	R-F-Ob-5
UC6-A	R-F-Ob-6, R-N-De-6
UC7-F	R-F-Ob-7, R-N-Ob-3
UC8-F	R-F-Ob-8, R-N-De-6
UC9-A	R-F-De-9, R-N-De-6
UC10-A	R-F-De-10, R-N-De-6
UC11-S	R-F-Ob-11, R-N-Ob-8, R-V-Ob-3, R-V-Ob-6, R-V-Ob-7, R-V-Ob-8
UC11-F	R-F-Ob-12, R-F-Ob-13, R-N-Ob-8, R-V-Ob-7
UC12-F	R-F-De-14, R-N-De-4, R-V-Ob-4
UC12.1-F	R-F-De-14, R-N-De-4
UC12.2-F	R-F-De-14, R-N-De-4
UC12.3-F	R-F-De-14, R-N-De-4
UC12.4-F	R-F-De-14, R-N-De-4
UC12.5-F	R-F-De-12, R-N-De-4
UC12.6-F	R-F-De-14, R-N-De-4
UC12.7-F	R-F-De-14, R-N-De-4
UC12.8-F	R-F-De-14, R-N-De-4
UC12.9-F	R-F-De-14, R-N-De-4
Continua nella prossima pagina...	

Tabella 3.4 - Continuo della tabella

Fonte	Requisiti
UC12.10-F	R-F-De-14, R-N-De-4
UC12.11-F	R-F-De-14, R-N-De-4
UC13-F	R-F-De-15, R-N-Ob-2
UC14-A	R-F-Op-16, R-V-Ob-5
UC15-F	R-F-Ob-17, R-V-Ob-5
UC16-F	R-F-Ob-18, R-N-De-5, R-V-Ob-5
UC17-A	R-F-Ob-18
UC18-A	R-F-De-19
UC19-A	R-F-De-20, R-N-Op-7
UC20-F	R-F-Ob-21, R-N-De-5
UC21-F	R-F-Ob-22
Tutti	R-V-Ob-1, R-V-Ob-2

Tabella 3.4: Tracciamento Fonti - Requisiti.

3.4.5 Riepilogo dei requisiti

Nella Tabella 3.5 si riporta il numero di requisiti per categoria e per urgenza; in totale sono stati rilevati 38 requisiti.

	Obbligatoria	Desiderabili	Opzionali	TOTALE
RF	14	7	1	22
RN	4	3	1	8
RV	8	0	0	8
TOTALE	26	10	2	38

Tabella 3.5: Conteggio dei requisiti tracciati.

4. Tecnologie utilizzate

In questo capitolo vengono presentate le principali tecnologie impiegate durante lo sviluppo del progetto. La descrizione ha lo scopo di fornire il contesto necessario alla comprensione delle scelte implementative illustrate nei capitoli successivi.

4.1 ESP32-S3-DevKitC-1

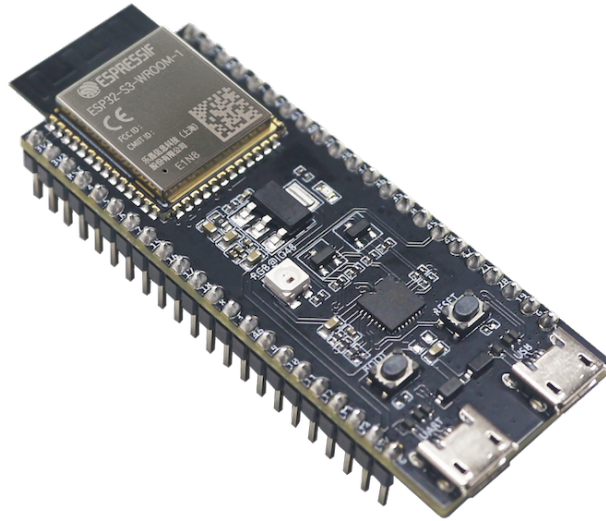


Figura 4.1: Scheda di sviluppo ESP32-S3-DevKitC-1

La *ESP32-S3-DevKitC-1*¹ è una scheda di sviluppo basata sul microcontrollore ESP32-S3, prodotto da Espressif Systems. La scheda mette a disposizione le principali periferiche del microcontrollore attraverso un'interfaccia adatta alla prototipazione e allo sviluppo di applicazioni embedded.

L'ESP32-S3 integra un processore dual-core e dispone di memoria e periferiche sufficienti per supportare applicazioni caratterizzate dalla presenza simultanea di più attività. Tra le funzionalità disponibili rientrano inoltre la connettività **Wi-Fi** e **BLE**, caratteristiche particolarmente rilevanti per il progetto in esame.

La scheda è stata utilizzata come piattaforma hardware principale per lo sviluppo del firmware, permettendo di gestire contemporaneamente l'acquisizione dei dati provenienti dai sensori, la memorizzazione e le comunicazioni con dispositivi esterni.

¹[*ESP32-S3-DevKitC-1*](#).

4.2 ESP-WROOM-32

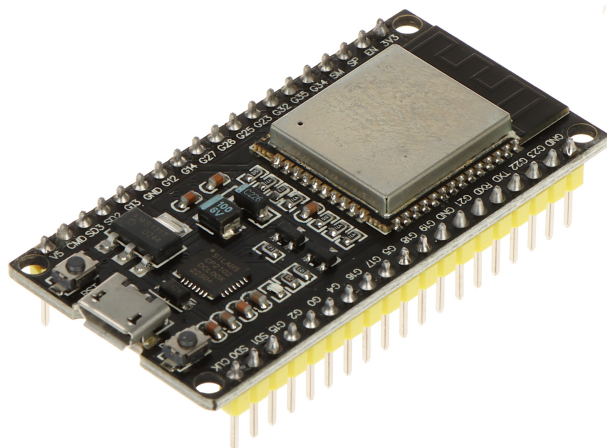


Figura 4.2: Modulo ESP-WROOM-32

L'ESP-WROOM-32² è un modulo basato sulla famiglia di microcontrollori ESP32. A differenza della ESP32-S3-DevKitC-1, che è una scheda di sviluppo, l'ESP-WROOM-32 costituisce principalmente un modulo hardware **compatto** integrabile all'interno di una scheda elettronica.

Il modulo integra il microcontrollore e i componenti necessari al suo funzionamento, oltre alle funzionalità di comunicazione wireless Wi-Fi e BLE. Le sue dimensioni contenute lo rendono particolarmente adatto all'integrazione in dispositivi embedded destinati alla produzione.

Nel progetto, l'ESP-WROOM-32 è stato utilizzato come dispositivo dedicato all'attività di *sniffing* del bus CAN, mantenendolo fisicamente separato dall'unità principale responsabile dell'acquisizione dei dati dai sensori. Questa scelta è stata adottata al fine di ridurre il rischio di danneggiamento dei componenti del sistema principale, considerando l'ambiente elettrico particolarmente critico di una motocicletta, caratterizzato da possibili disturbi e variazioni di tensione dovuti ai sistemi di alimentazione e agli attuatori presenti a bordo. In questo modo, eventuali anomalie o sovratensioni provenienti dall'interfacciamento con il bus CAN rimangono confinate, per quanto possibile, al dispositivo dedicato, preservando gli altri componenti del sistema.

²ESP32-WROOM-32 Series. URL: <https://www.espressif.com/en/products/modules/esp32>.

4.3 C++20

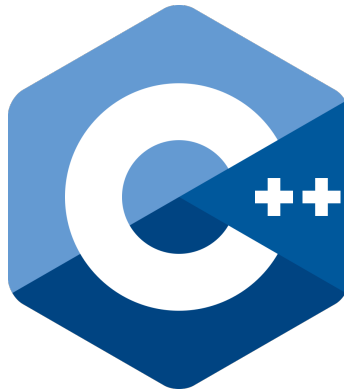


Figura 4.3: Logo del linguaggio C++

Il firmware del dispositivo è stato sviluppato utilizzando il linguaggio C++, facendo riferimento allo standard C++20³. La scelta di questo linguaggio è legata principalmente alla necessità di mantenere un controllo diretto sulle risorse hardware e computazionali del microcontrollore.

In un sistema embedded, infatti, memoria, tempo di esecuzione e capacità computazionale sono risorse limitate che devono essere gestite con attenzione. C++ permette di operare a un livello sufficientemente vicino all'hardware, offrendo allo stesso tempo strumenti di astrazione come classi, ereditarietà, incapsulamento e polimorfismo.

4.4 ESP32 Arduino Core

Lo sviluppo del firmware è stato effettuato utilizzando *ESP32 Arduino Core*⁴, un framework che permette di programmare i microcontrollori ESP32 attraverso le API e le modalità di sviluppo tipiche dell'ecosistema Arduino.

Il framework fornisce un livello di astrazione rispetto alle API native di ESP-IDF, rendendo più semplice l'utilizzo delle periferiche e delle funzionalità messe a disposizione dal microcontrollore. Questa caratteristica consente di ridurre la complessità del codice applicativo, mantenendo comunque la possibilità di accedere alle funzionalità specifiche della piattaforma quando necessario.

La scelta di questo framework ha permesso di concentrare lo sviluppo sulle funzionalità richieste dal progetto, evitando di dover gestire direttamente l'intera complessità dell'ESP-IDF.

³ C++20 – *cppreference.com*. URL: <https://en.cppreference.com/w/cpp/20>.

⁴ *arduino-esp32*. URL: <https://github.com/espressif/arduino-esp32>.

4.5 FreeRTOS



Figura 4.4: FreeRTOS

FreeRTOS⁵ è un kernel real-time progettato per sistemi embedded nei quali diverse attività devono essere eseguite in modo concorrente e rispettando specifici requisiti temporali. Il kernel fornisce principalmente un sistema di gestione delle attività, denominato task scheduling, e una serie di primitive per coordinare l'accesso alle risorse condivise.

Nel firmware sviluppato durante il progetto sono presenti numerose attività che devono operare con frequenze e tempi di risposta differenti. L'acquisizione dei dati provenienti dai sensori, la scrittura sulla memoria microSD e la gestione delle comunicazioni costituiscono infatti attività indipendenti, che devono poter essere eseguite senza bloccare reciprocamente il sistema. L'utilizzo di FreeRTOS permette di suddividere il comportamento del firmware in task indipendenti e lasciare al kernel la gestione dell'accesso al processore.

4.5.1 Sistemi real-time e concorrenza

Per comprendere l'utilità di un kernel come FreeRTOS è utile richiamare brevemente il concetto di concorrenza. In un programma eseguito in maniera sequenziale, le istruzioni vengono normalmente eseguite una dopo l'altra secondo l'ordine stabilito dal programma. Questo modello diventa poco adatto quando il dispositivo deve gestire contemporaneamente più sorgenti di dati o eventi.

In un sistema multitasking, il software viene suddiviso in più attività indipendenti. Quando il microcontrollore dispone di un singolo core, non è possibile eseguire fisicamente più istruzioni appartenenti a task differenti nello stesso istante. Il sistema operativo può tuttavia alternare rapidamente l'esecuzione dei task, dando l'impressione di una loro esecuzione contemporanea.

Il passaggio da un task a un altro viene indicato come *context switch*. Durante questo processo il sistema conserva le informazioni necessarie per riprendere successivamente l'esecuzione del task sospeso e carica il contesto del task successivo. Nel caso di un processore dotato di più core, alcuni task possono invece essere eseguiti realmente in parallelo, purché siano disponibili core liberi e la configurazione dello scheduler lo permetta.

4.5.2 Scheduler e priorità

La componente di FreeRTOS responsabile della scelta del task da eseguire è lo scheduler. A ogni task può essere assegnato un livello di priorità, che permette di stabilire quali

⁵FreeRTOS – Real-time operating system for microcontrollers. URL: <https://www.freertos.org/>.

attività debbano avere precedenza quando più task sono contemporaneamente pronte per l'esecuzione.

Un task che non deve essere eseguita in un determinato momento può inoltre rinunciare temporaneamente al processore, permettendo ad altre attività di utilizzarlo. Questo comportamento è particolarmente importante in sistemi embedded, poiché evita di consumare inutilmente risorse computazionali durante i periodi di attesa.

Il concetto di real-time non implica quindi semplicemente che il sistema sia veloce, ma che le operazioni importanti possano essere completate entro limiti temporali definiti. Nel progetto questo aspetto è rilevante soprattutto per l'acquisizione periodica dei dati dei sensori, dove il rispetto della frequenza di campionamento influenza direttamente la qualità dei dati registrati.

4.5.3 Task

Un task rappresenta un'unità indipendente di esecuzione gestita da FreeRTOS. Ogni task possiede un proprio contesto e viene definita specificando un insieme di parametri, riassunti in Tabella 4.1.

Parametro	Descrizione
Funzione	Punto di ingresso eseguito dal task, contenente il codice associato all'attività.
Priorità	Valore numerico che determina la precedenza del task rispetto ad altri nella fase di scheduling.
Dimensione dello stack	Quantità di memoria riservata al task per la gestione di variabili locali e chiamate a funzione.
Parametri	Dati opzionali passati alla funzione al momento della creazione del task.
Core di esecuzione	Nei microcontrollori multi-core, indica su quale core il task può essere <i>schedulato</i> .

Tabella 4.1: Principali parametri di configurazione di un task in FreeRTOS

Nel firmware i task sono stati utilizzati per separare funzionalità che presentano caratteristiche temporali differenti. Ad esempio, l'acquisizione dei dati dei sensori può essere eseguita indipendentemente dalle operazioni di scrittura sulla scheda microSD.

Questa suddivisione permette di evitare che un'attività caratterizzata da tempi di esecuzione variabili blocchi un'altra attività che richiede invece un'esecuzione periodica.

4.5.4 Stati dei task

Durante la propria vita un task può assumere differenti stati, in funzione della possibilità di essere eseguita in un determinato momento.

Stato	Descrizione
Running	Il task è attualmente in esecuzione su uno dei core disponibili.
Ready	Il task è pronto per essere eseguito, ma il processore è momentaneamente assegnato a un altro task.
Blocked	Il task è in attesa del verificarsi di un evento, di una notifica o della scadenza di un intervallo temporale.
Suspended	Il task è stato esplicitamente escluso dallo scheduling e rimane in questo stato fino a una successiva riattivazione.

Tabella 4.2: Principali stati di una task in FreeRTOS

La distinzione tra *Blocked* e *Suspended* è particolarmente importante. Nel primo caso il task rimane in attesa di una condizione specifica, mentre nel secondo viene sospeso esplicitamente e non esiste un evento che possa determinarne autonomamente il risveglio. Questa caratteristica può essere sfruttata, ad esempio, per disabilitare temporaneamente attività che non sono necessarie in una determinata fase del funzionamento del dispositivo.

4.5.5 Race condition e sincronizzazione

La suddivisione del firmware in più task introduce un ulteriore problema: più attività possono avere bisogno di utilizzare contemporaneamente una stessa risorsa.

Si consideri una variabile condivisa x , inizialmente impostata a zero, e due task, A e B , che devono entrambe incrementarla di una unità. Sebbene l'istruzione concettuale sia semplicemente "incrementare x ", a livello macchina l'operazione viene scomposta in tre passaggi elementari: lettura del valore corrente, modifica del valore letto e scrittura del nuovo valore in memoria. Se un task viene interrotto tra uno di questi passaggi e lo scheduler permette all'altro task di operare sulla stessa variabile, si può verificare l'interleaving mostrato in Tabella 4.3.

Istante	Task A	Task B	Valore di x in memoria
t_0	legge x (0)	—	0
t_1	—	legge x (0)	0
t_2	calcola $x + 1$	—	0
t_3	—	calcola $x + 1$	0
t_4	scrive $x = 1$	—	1
t_5	—	scrive $x = 1$	1

Tabella 4.3: Esempio di interleaving che genera una race condition

Nonostante entrambi i task abbiano eseguito un incremento, il valore finale di x è 1 anziché 2, poiché la scrittura effettuata da B sovrascrive quella di A sulla base di un valore letto prima che l'incremento di A venisse reso visibile. Il risultato è una *race condition*, ovvero un comportamento dipendente dall'ordine con cui i task accedono alla risorsa condivisa.

4.5.6 Mutex

Per proteggere le sezioni di codice che accedono a risorse condivise è possibile utilizzare un mutex, abbreviazione di *mutual exclusion*. Il mutex permette a un solo task per volta di entrare nella sezione critica associata alla risorsa protetta.

Il funzionamento è riassunto in Tabella 4.4.

Fase	Descrizione
Richiesta	Il task richiede l'acquisizione del mutex prima di accedere alla sezione critica.
Mutex libero	Se il mutex non è occupato, il task lo acquisisce ed entra nella sezione critica.
Mutex occupato	Se il mutex è già occupato da un altro task, il task richiedente viene posto in stato di attesa.
Rilascio	Al termine dell'accesso alla risorsa, il task rilascia il mutex.
Sblocco	Uno degli eventuali task in attesa viene sbloccato e può quindi procedere.

Tabella 4.4: Fasi di utilizzo di un mutex

In questo modo le operazioni che devono essere eseguite senza interferenze da parte degli altri task vengono protette, impedendo accessi concorrenti incompatibili.

Nel firmware questa tecnica risulta utile quando più attività devono accedere a risorse condivise, come strutture dati comuni o periferiche che non possono essere utilizzate contemporaneamente.

4.5.7 Comunicazione tra task

FreeRTOS mette inoltre a disposizione meccanismi specifici per consentire ai task di scambiarsi informazioni senza ricorrere necessariamente a variabili globali condivise.

Tra questi strumenti rientrano principalmente le *queue* e le *task notification*, riassunte e confrontate in Tabella 4.5.

	Queue	Task notification
Contenuto	Trasferisce dati veri e propri tra task	Trasporta un semplice valore o segnale di evento
Modalità	Produttore-consumatore, con più elementi in coda	Diretta, verso un singolo task specifico
Overhead	Maggiore, dovuto alla gestione della coda	Ridotto, essendo un meccanismo più leggero
Utilizzo tipico	Scambio di dati acquisiti tra task diversi	Notifica rapida di un evento (es. dato pronto)

Tabella 4.5: Confronto tra queue e task notification in FreeRTOS

Questi meccanismi consentono di progettare componenti maggiormente indipendenti, riducendo la necessità di condividere direttamente lo stato interno dei singoli task.

4.6 Expo & React Native



Figura 4.5: Logo di Expo & React Native

React Native⁶ è un framework open source che consente di sviluppare applicazioni mobili multiplatforma utilizzando Typescript (o Javascript) e il paradigma dichiarativo tipico di React. React Native traduce i componenti dell'interfaccia in elementi grafici nativi, permettendo di ottenere prestazioni e un aspetto più vicini a quelli di un'applicazione sviluppata nativamente per Android o iOS, pur mantenendo un'unica base di codice condivisa tra le due piattaforme.

Nel progetto React Native è stato utilizzato insieme a *Expo*⁷, un insieme di strumenti che semplifica la configurazione, la compilazione e l'esecuzione di applicazioni React Native. In particolare, lo sviluppo dell'applicazione è stato condotto tramite *development build*, generando ed eseguendo un progetto nativo sul dispositivo di test attraverso i comandi `npx expo run:ios` e `npx expo run:android`, a seconda della piattaforma di destinazione.

4.7 TypeScript

TypeScript⁸ è un linguaggio di programmazione sviluppato da Microsoft come soprainsieme (*superset*) di JavaScript, al quale aggiunge un sistema di tipizzazione statica opzionale. Il codice TypeScript viene tradotto (*transpiled*) in JavaScript standard prima dell'esecuzione, mantenendo piena compatibilità con l'ecosistema e le librerie già disponibili per quest'ultimo.

La tipizzazione statica consente di individuare in fase di sviluppo, anziché a runtime, numerosi errori legati a un uso scorretto dei dati, come il passaggio di un parametro di tipo errato a una funzione o l'accesso a una proprietà inesistente di un oggetto. Questo aspetto risulta particolarmente utile in un'applicazione di dimensioni non trascurabili come quella sviluppata nel progetto, dove il numero di componenti, funzioni e strutture dati scambiate tra le diverse parti del codice rende più difficile individuare manualmente questo tipo di errori.

Nel progetto TypeScript è stato utilizzato per l'intero sviluppo dell'applicazione React Native, contribuendo a rendere il codice più leggibile e manutenibile, oltre a facilitare

⁶ React Native. URL: <https://reactnative.dev/>.

⁷ Expo Documentation. URL: <https://expo.dev/>.

⁸ TypeScript. URL: <https://www.typescriptlang.org/>.

l'integrazione con l'editor di sviluppo grazie al supporto per l'autocompletamento e il controllo dei tipi in tempo reale.

4.8 Git



Figura 4.6: Git

Git⁹ è un sistema distribuito per il controllo della versione, utilizzato per registrare e organizzare l'evoluzione del codice sorgente nel tempo. Ogni modifica significativa può essere salvata all'interno della cronologia del progetto, permettendo di recuperare versioni precedenti e analizzare le variazioni introdotte.

Il progetto utilizza Git per la gestione del codice sorgente del firmware, facilitando inoltre lo sviluppo parallelo e la collaborazione tra più sviluppatori.

4.9 PlatformIO



Figura 4.7: PlatformIO

PlatformIO¹⁰ è un ambiente di sviluppo dedicato principalmente alla programmazione di sistemi embedded. Nel progetto è stato utilizzato all'interno di Visual Studio Code per gestire il ciclo di sviluppo del firmware.

Lo strumento automatizza diverse operazioni necessarie durante lo sviluppo, tra cui la compilazione del codice, il caricamento del firmware sulla scheda (*flash*), la gestione delle librerie e delle relative dipendenze e l'utilizzo del monitor seriale.

Un ulteriore vantaggio è rappresentato dalla gestione centralizzata della configurazione del progetto, attraverso la quale è possibile specificare la scheda utilizzata, il framework, le dipendenze e le impostazioni relative alla compilazione.

⁹ *Git*. URL: <https://git-scm.com/>.

¹⁰ *PlatformIO*. URL: <https://platformio.org/>.

4.10 L^AT_EX

L^AT_EX¹¹ è un sistema di composizione tipografica particolarmente diffuso nella produzione di documenti scientifici e accademici. A differenza dei comuni editor visuali, la struttura del documento viene definita attraverso un linguaggio di markup, separando il contenuto dalla sua formattazione.

4.11 PlantUML



Figura 4.8: Esempio di diagramma generato con PlantUML

PlantUML¹² è uno strumento che consente di generare diagrammi UML a partire da una descrizione testuale. Questa modalità permette di definire la struttura del diagramma direttamente all'interno di file di testo, rendendo semplice la modifica e il mantenimento dei diagrammi nel tempo.

Nel progetto PlantUML è stato utilizzato per la realizzazione dei diagrammi UML impiegati nella documentazione del sistema. La generazione automatica dei diagrammi consente inoltre di mantenere separati il contenuto descrittivo del diagramma e la sua rappresentazione grafica, facilitandone l'aggiornamento durante l'evoluzione del progetto.

4.12 C4 Model

Il C4 Model¹³ è un modello per la rappresentazione dell'architettura software attraverso diagrammi organizzati su diversi livelli di dettaglio. Il modello permette di descrivere il sistema progressivamente, partendo dal contesto generale fino alla struttura interna dei singoli componenti, favorendo una rappresentazione chiara e comprensibile dell'architettura.

Nel progetto il C4 Model è stato utilizzato per rappresentare l'architettura del sistema a diversi livelli di astrazione, evidenziando le principali relazioni tra il sistema, gli attori esterni e i suoi componenti interni.

¹¹ *LaTeX* – A document preparation system. URL: <https://www.latex-project.org/>.

¹² *PlantUML*. URL: <https://plantuml.com/>.

¹³ *C4 model*. URL: <https://c4model.com/>.

5. Progettazione

In questo capitolo vengono presentate le scelte progettuali definite a monte della stesura del codice: l'architettura del sistema, i protocolli di comunicazione e i formati dati adottati, nonché i pattern architetturali scelti per la loro realizzazione. La descrizione ha lo scopo di motivare tali decisioni, rimandando al capitolo successivo la loro effettiva traduzione in codice.

5.1 Situazione di partenza

Questa sezione descrive l'architettura del sistema così come si presentava prima dell'inizio del tirocinio, allo scopo di fornire un punto di riferimento rispetto a cui distinguere, nelle sezioni successive, i contributi introdotti durante il lavoro svolto.

5.1.1 Firmware

5.1.1.1 Architettura

Il firmware implementava un meccanismo di acquisizione e persistenza dei dati, basato sulla suddivisione in **producer**, uno per sensore, e un unico **consumer** dedicato alla scrittura su microSD. I due ruoli comunicavano tramite una coda di richieste di scrittura, ciascuna costituita da un riferimento al buffer dati da trascrivere, dalla sua lunghezza e dall'identificativo del sensore che lo ha prodotto.

All'accensione, ottenuta semplicemente collegando il dispositivo a una fonte di alimentazione, il firmware avviava automaticamente l'acquisizione dei dati, senza richiedere alcun comando esplicito da parte dell'utente. La persistenza avveniva affidandosi al file system **FAT_G**: ciascun blocco di dati (*chunk*) veniva salvato come file separato in formato **CSV**. Per evitare che la scrittura su microSD bloccasse l'acquisizione dati, a ogni producer erano associati due buffer alternati: mentre uno veniva scritto su microSD dal consumer, l'altro restava disponibile per contenere i dati recuperati dai sensori. Tali ruoli possono scambiarsi durante l'esecuzione normale del firmware.

Indipendentemente dal meccanismo di persistenza, il firmware trasmetteva già i dati acquisiti in tempo reale all'app EggLog tramite BLE, permettendo la visualizzazione *live* delle grandezze rilevate durante la sessione, in parallelo alla loro scrittura su microSD.

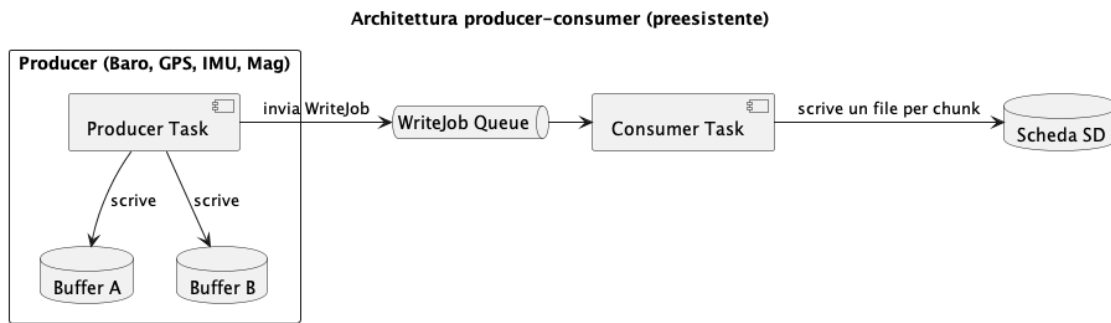


Figura 5.1: Architettura producer-consumer preesistente

5.1.1.2 Scelte progettuali

- ***Producer-Consumer_G***: separazione netta tra il ruolo di acquisizione dati e quello di persistenza, disaccoppiati tramite una coda di richieste di scrittura.
- **Double buffering (ping-pong buffer)**: permette lettura e scrittura concorrenti sullo stesso flusso di dati senza reciproco blocco.
- **Container singleton per la configurazione**: la configurazione è esposta tramite due punti di accesso globali unici, che incapsulano l'accesso concorrente allo stato condiviso. I parametri di configurazione persistenti sono memorizzati in memoria non volatile, garantendone la conservazione anche dopo il riavvio del dispositivo.
- **Trasmissione dati in tempo reale via BLE**: il flusso di dati verso l'app è disaccoppiato dal meccanismo di persistenza su microSD, permettendo la visualizzazione live delle grandezze rilevate senza dipendere dalla scrittura effettiva dei dati su file.

5.1.2 Applicazione mobile

L'applicazione EggLog, realizzata in **React Native**, era già in grado di comunicare con il dispositivo tramite BLE, secondo il modello Central-Peripheral tipico del profilo *GAP_G*: scansione dei dispositivi in stato di *advertising_G*, pairing e successiva scoperta dei *service_G* e delle *characteristic_G* esposti dal firmware.

Erano inoltre già presenti alcune schermate base (tra cui Home, Devices e Record), che mostravano la telemetria ricevuta in tempo reale via BLE, seppur in una forma non ancora rifinita.

A differenza del firmware, per cui gli interventi sono iniziati fin da subito, il lavoro sull'applicazione è iniziato a **metà tirocinio**: nella prima parte, l'applicazione è stata oggetto di un lavoro di pulizia e riorganizzazione da parte dell'altro membro del team di tirocinio, volto a rimuovere codice e funzionalità non più necessarie. Solo una volta raggiunta una base più solida si è potuto intervenire aggiungendo i componenti e i canali BLE relativi ai contributi descritti nelle sezioni seguenti.

Queste scelte erano già consolidate nella base di codice ricevuta all'inizio del tirocinio e sono state mantenute come fondamenta su cui innestare gli interventi successivi, descritti nelle sezioni seguenti.

Le sezioni che seguono descrivono in dettaglio le scelte progettuali introdotte nel corso del tirocinio, organizzate secondo le seguenti aree funzionali:

- **Firmware EggLog**

- sistema di persistenza a superfile;
- gestione dello stato globale del dispositivo;
- segnalazione visiva tramite LED;
- gestione dell'inattività dei sensori;
- protocollo di avvio e arresto della sessione;
- calibrazione dell'altitudine;
- calibrazione dell'IMU;

- **EggLog CAN Sniffer**

- valutazione tecnica dello sniffer CAN e della sua architettura.

- **Applicazione mobile EggLog**

- visualizzazione 3D del veicolo;
- segnalazione di microSD mancante durante la sessione;
- configurazione dei parametri del dispositivo.

Per inquadrare l'architettura del sistema prima di entrarne nel dettaglio, le figure seguenti adottano la notazione del modello C4, che descrive un sistema attraverso viste innestate a livello di dettaglio crescente. La Figura 5.2 presenta una vista di contesto, posizionando il dispositivo EggLog rispetto agli attori e ai sistemi esterni con cui interagisce: il pilota, EggLog App, il bus CAN del motoveicolo e EggLog CAN Sniffer.

La Figura 5.3 ne approfondisce la struttura interna a livello di container, distinguendo il firmware in esecuzione sul microcontrollore dalla persistenza su scheda microSD.

A partire da questo inquadramento generale, valido per l'intero capitolo, ciascun gruppo di sezioni successive è introdotto da un diagramma dei componenti che mette a fuoco i soli moduli rilevanti per quell'area funzionale. Il primo, riportato in Figura 5.4, riguarda la persistenza dei dati e la gestione dello stato condiviso, argomenti delle due sezioni immediatamente seguenti.

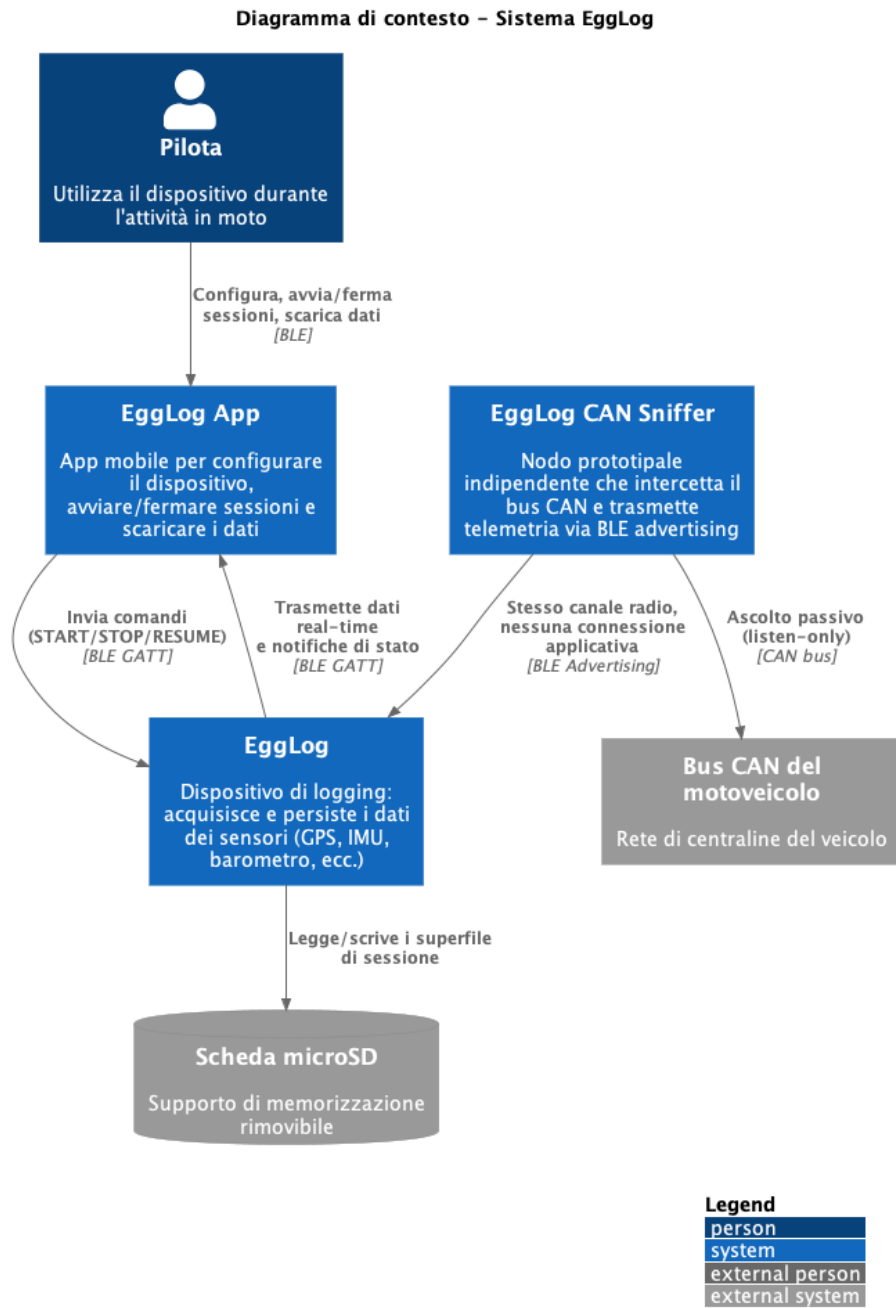


Figura 5.2: Diagramma di contesto del sistema EggLog

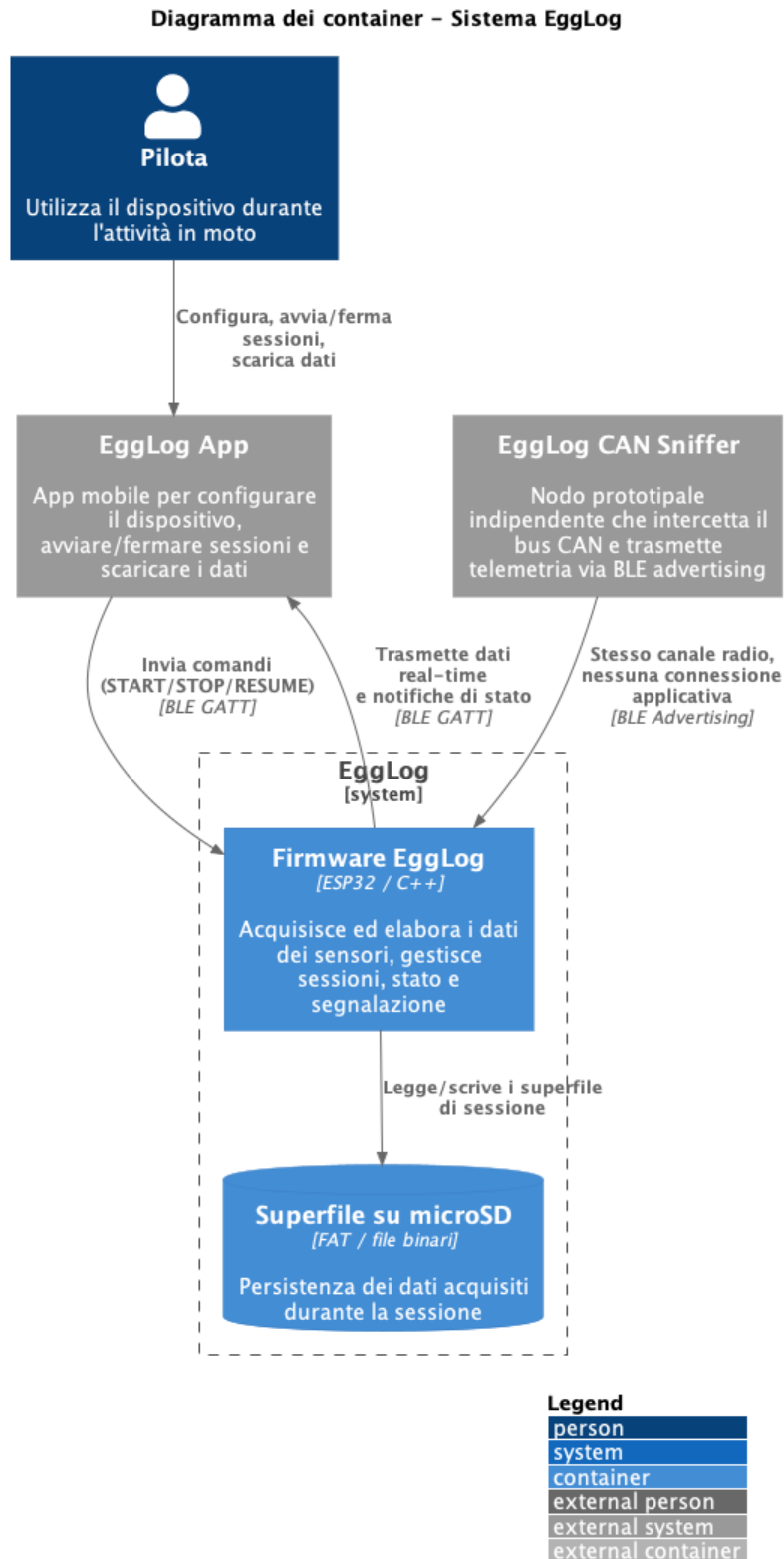


Figura 5.3: Diagramma dei container del sistema EggLog

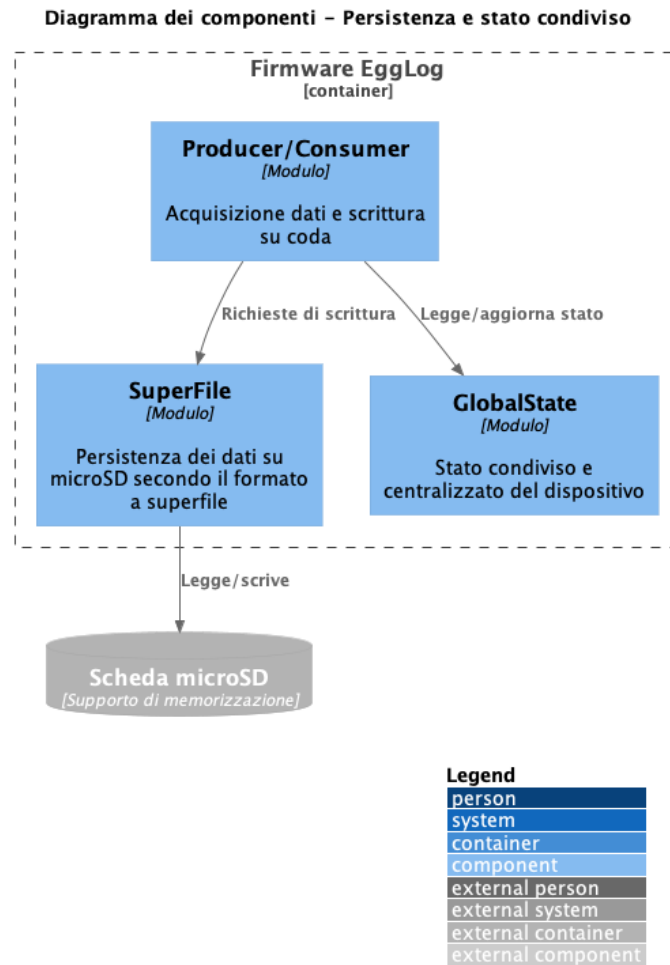


Figura 5.4: Diagramma dei componenti del firmware EggLog: persistenza e stato condiviso

5.2 Sistema di persistenza a superfile

5.2.1 Architettura

Il layer di persistenza a basso livello viene progettato come un modulo concettualmente diviso in due responsabilità nettamente separate:

- **formato su disco**, ovvero l'insieme minimo di informazioni che devono essere scritte fisicamente all'inizio del file affinché esso sia autodescrittivo e riapribile in modo corretto anche dopo un riavvio del dispositivo;
- **stato di runtime**, ovvero l'insieme di informazioni necessarie a operare sul file mentre il dispositivo è acceso. Tali informazioni possono essere: il riferimento al file aperto, i parametri derivati dal sensore associato, la dimensione del *record_G* (che varia da sensore a sensore), la capacità e soglie operative.

Le operazioni concettuali che il modulo mette a disposizione sono: creazione di un nuovo file, apertura con eventuale recupero dello stato dopo un'interruzione imprevista, scrittura di un blocco di record, sincronizzazione periodica dell'header su disco e in fine chiusura.

Tali operazioni sono pensate come funzioni indipendenti che agiscono su un'istanza esplicita del file, anziché come comportamento implicito legato a uno stato nascosto: questa scelta è motivata dalla volontà di restare aderenti a uno stile a basso *overhead*, coerente con i vincoli di un sistema embedded a risorse limitate.

Il modulo non contiene alcuna logica di concorrenza al proprio interno: l'accesso concorrente da parte di più sensori è delegato interamente al layer superiore (il consumer già presente nella base di codice preesistente), che invoca le operazioni sotto un lock esplicito sul canale di comunicazione condiviso con la scheda microSD.

Si mantiene così una separazione netta di responsabilità tra:

- cosa scrivere e come è organizzato il file;
- quando scrivere e con quali garanzie di sincronizzazione tra le diverse fonti di dati.

5.2.2 Scelte progettuali

- **Separazione tra formato persistito e stato operativo:** il layout dei dati salvati su disco è pensato come concettualmente indipendente dallo stato necessario per operare sul file durante l'esecuzione.

Vantaggio: un'evoluzione futura del formato su disco (ad esempio un nuovo campo nell'header) non richiede di intervenire sulla logica di gestione del file, e viceversa; la presenza di un identificativo di versione nell'header rende inoltre possibile, in linea di principio, gestire in futuro più versioni di formato senza rompere la compatibilità con i file già scritti.

- **Finestra di azzeramento anticipata (*pre-zeroed lookahead window*):** illustrata in dettaglio nella sezione dedicata alla recovery ([Recovery dopo interruzioni impreviste](#)).

Vantaggio: permette una recovery deterministica dopo un'interruzione imprevista, a fronte di un costo aggiuntivo di scrittura minimo.

5.2.3 Formato su disco: l'header

Ogni superfile è organizzato in due aree contigue: un'**area di intestazione** (header), di dimensione fissa e allineata al settore fisico del supporto di memorizzazione, seguita da un'**area dati**, costituita da una sequenza di record a dimensione fissa specifica per ciascun sensore.

L'header è l'unica porzione di stato che viene effettivamente persistita sul supporto e che descrive il contenuto del file: deve quindi contenere tutte le informazioni necessarie a riaprire correttamente il file dopo un riavvio del dispositivo, senza dover rileggere per intero l'area dati. Dal punto di vista concettuale, l'header è composto dai seguenti campi logici:

- **Versione del formato:** identifica la revisione del layout con cui il file è stato scritto, per consentire in futuro l'evoluzione del formato mantenendo la compatibilità con i file già esistenti;
- **Identificativo del sensore:** associa in modo univoco il file al sensore che lo ha prodotto;

- **Dimensione del record:** numero di byte occupati da ciascun record, specifico per il tipo di sensore;
- **Capacità totale:** numero massimo di record che il file può contenere, calcolato una sola volta alla creazione del file (approfondita in [Capacità del file](#));
- **Cursore di fine dati:** posizione, all'interno dell'area dati, del primo record ancora libero, cioè il punto in cui avverrà la prossima scrittura;
- **Cursore di inizio dati:** posizione del più vecchio record ancora considerato valido; rilevante unicamente per i sensori che operano in modalità a sovrascrittura;
- **Soglia di sincronizzazione:** intervallo, espresso in numero di record scritti, dopo il quale l'header viene ripersistito su disco (approfondito in [Sincronizzazione header](#));
- **Dimensione della finestra di azzeramento:** numero di record, immediatamente successivi al cursore di fine dati, mantenuti costantemente pre-azzerati (approfondito in [Finestra di azzeramento](#));
- **Indicatori di stato:** informazioni booleane quali l'abilitazione della modalità a sovrascrittura o il raggiungimento della capacità massima del file.

La figura seguente riassume questi campi logici e la loro collocazione all'interno del file.

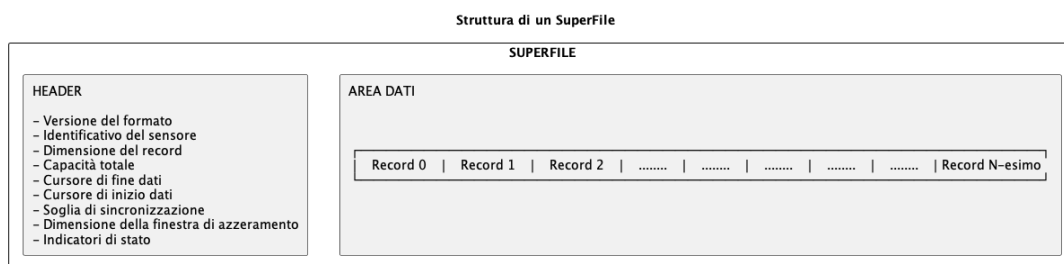


Figura 5.5: Campi logici dell'header del superfile

5.2.4 Capacità del file

La capacità totale del file, in numero di record, viene calcolata una sola volta al momento della creazione, a partire dall'*ODR_G* del sensore, dalla dimensione del suo record e dalla finestra temporale di conservazione dei dati desiderata (*retention*). Il valore ottenuto viene arrotondato per eccesso in modo da allinearsi alle unità di allocazione del supporto di memorizzazione, così da garantire che il file, una volta preallocato per intero, occupi esclusivamente spazio contiguo, evitando la frammentazione che affliggeva la soluzione a file multipli descritta nella [Situazione di partenza](#).

5.2.5 Scrittura e *wraparound*

L'area dati del superfile è concettualmente organizzata come un **buffer circolare**: la scrittura di un nuovo blocco di record calcola la posizione di partenza a partire dal cursore di fine dati corrente e verifica se il blocco attraversa il limite superiore dell'area dati; in

tal caso l'operazione viene concettualmente spezzata in due scritture separate (*pre-wrap* e *post-wrap*), la seconda delle quali riparte dall'inizio dell'area dati.

Per i sensori in modalità a sovrascrittura, il cursore di inizio dati avanza automaticamente quando il buffer si riempie, sacrificando i record più vecchi. Per il sensore GPS, per il quale la sovrascrittura è disabilitata in quanto ritenuto una sorgente di dati più importante, al raggiungimento della capacità massima le scritture successive vengono ignorate e il file viene marcato come pieno, propedeuticamente alla terminazione automatica della sessione.

5.2.6 Sincronizzazione dell'header

Per limitare l'usura del supporto di memorizzazione e massimizzare il *throughput* di scrittura, l'header non viene sincronizzato su disco a ogni singolo record scritto, ma periodicamente, al raggiungimento della soglia di sincronizzazione descritta in precedenza. Tale soglia non è un valore fisso comune a tutti i sensori, ma viene ricavata a partire da un budget di byte target da sincronizzare, rapportato alla dimensione del record del sensore. In questo modo, la frequenza di sincronizzazione risulta proporzionata al volume di dati effettivamente prodotto, indipendentemente dalla dimensione del singolo record.

Questa scelta introduce tuttavia un problema di consistenza in caso di arresto imprevisto del dispositivo. Tra due sincronizzazioni dell'header, infatti, possono essere già presenti sul supporto record scritti successivamente all'ultimo aggiornamento dell'header, dei quali quest'ultimo non è ancora a conoscenza. È quindi necessario disporre di un meccanismo che permetta di individuare e recuperare correttamente tali record durante la successiva apertura del file. Tale meccanismo è descritto nella sezione immediatamente successiva ([Finestra di azzeramento](#)).

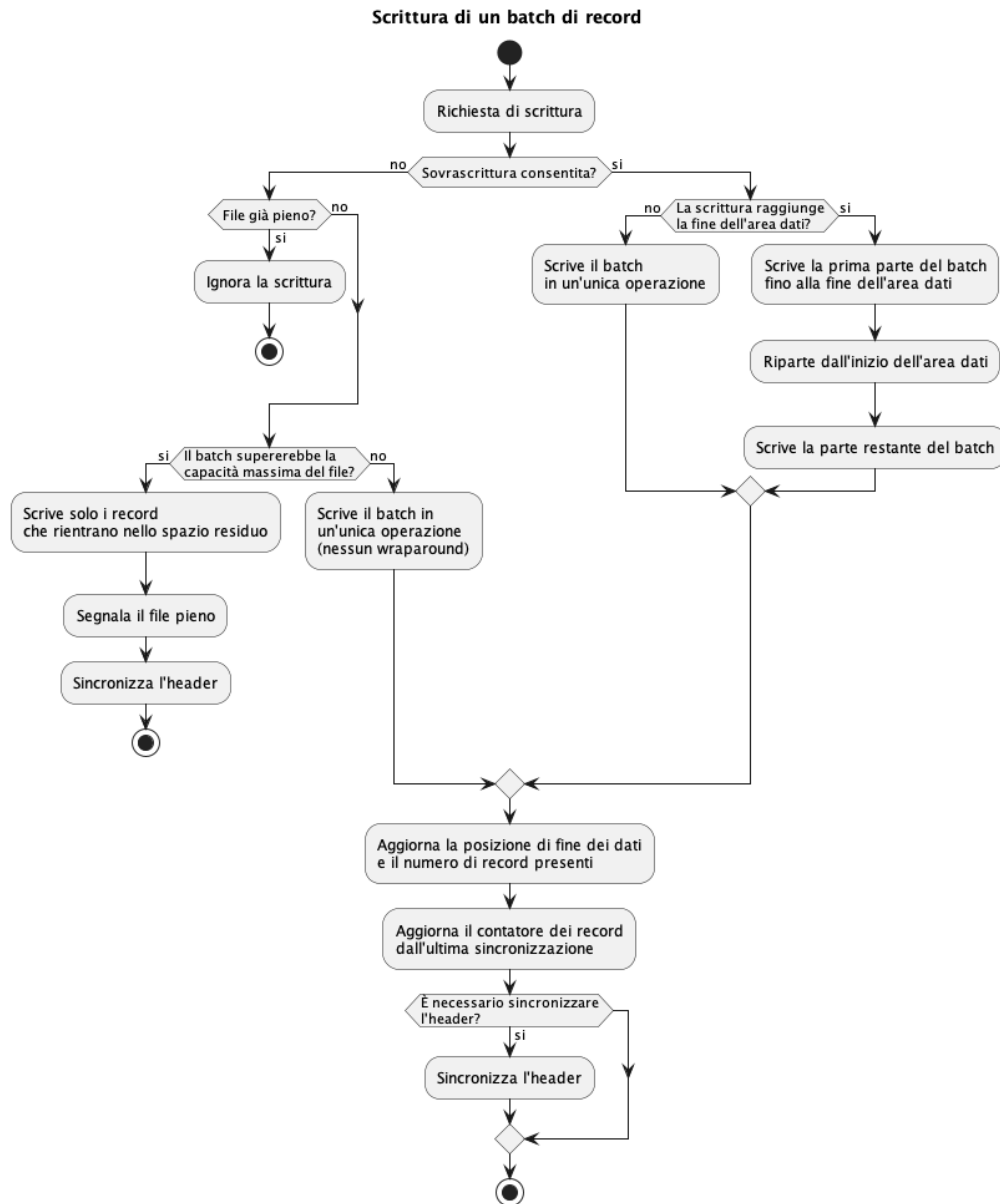


Figura 5.6: Diagramma di flusso della scrittura di un insieme di record

5.2.7 Finestra di azzeramento

A supporto della procedura di recovery viene mantenuta una **finestra di record pre-azzerati** immediatamente davanti al cursore di scrittura. La finestra ha una dimensione pari a un multiplo della soglia di sincronizzazione ed è costituita da record inizializzati a zero. Poiché ogni record valido inizia con un marcatore temporale non nullo, un record contenente esclusivamente valori azzerati può essere identificato come non ancora scritto. La finestra viene inizializzata alla creazione del file e ricostituita a ogni sincronizzazione dell'header, subito dopo l'avanzamento del cursore di scrittura. In caso di arresto imprevisto, la presenza dei record pre-azzerati permette quindi di delimitare l'area potenzialmente contenente dati non ancora riportati nell'header e di effettuare una recovery coerente dello stato del file.

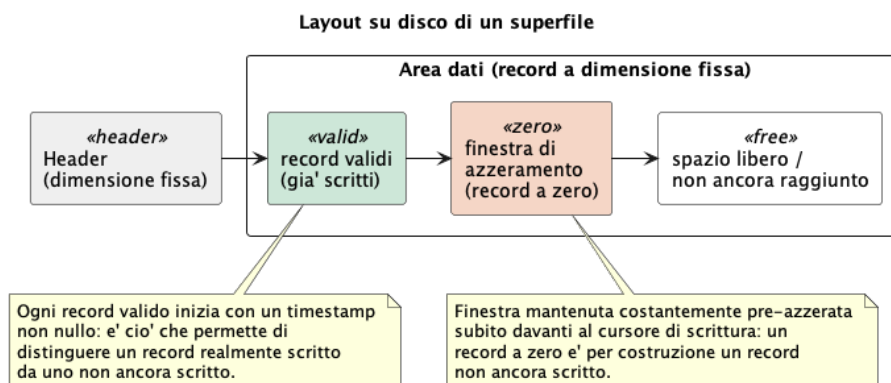


Figura 5.7: Layout su disco del superfile: area dati valida, finestra di azzeramento e spazio libero

5.2.8 Recovery dopo interruzioni impreviste

Il meccanismo di recovery si appoggia, a livello di singolo superfile, proprio sulla finestra di azzeramento appena descritta. All'apertura di un superfile in modalità recupero, l'header letto da disco può quindi non riflettere l'esatto stato dei dati realmente presenti: la procedura di riconciliazione scandisce in avanti, record per record, a partire dalla posizione indicata dall'header, leggendo unicamente il marcatore temporale iniziale di ciascun record. Finché il marcatore letto è diverso da zero, il record viene considerato realmente scritto e il cursore viene avanzato, con la stessa logica usata durante la scrittura normale; la scansione si interrompe al primo marcatore nullo, che identifica in modo affidabile il vero confine tra dati validi e finestra pre-azzerata. Al termine della riconciliazione, l'header aggiornato viene risincronizzato su disco e la finestra di azzeramento ricostituita a partire dalla nuova posizione del cursore, ripristinando le condizioni necessarie per riprendere normalmente l'acquisizione. Tale algoritmo è illustrato in Figura 5.8.

A livello di dispositivo, l'intera procedura di recovery rimonta il file system, ricarica la configurazione persistita e, solo se lo stato globale indica una sessione di logging in corso, recupera il riferimento alla sessione attiva e richiama la riconciliazione per tutti i superfile della sessione; un fallimento in una qualsiasi di queste fasi imposta lo stato di errore globale, mentre il successo lo azzer esplicitamente, permettendo all'acquisizione di riprendere da dove si era interrotta, senza perdita né duplicazione dei dati già scritti.

5.2.9 Comunicazione dello stato di occupazione

Oltre alla propria area dati, ogni superfile espone in lettura, tramite il registro dei superfile attivi, lo stato necessario a determinare quanto spazio disponibile stia effettivamente occupando e se, per i sensori configurati in modalità a sovrascrittura, sia già in corso l'eliminazione di dati meno recenti. Questa informazione consente all'app di segnalare tempestivamente all'utente l'approssimarsi dell'esaurimento dello spazio disponibile, in particolare per il sensore GPS, per il quale il raggiungimento della capacità massima comporta l'arresto automatico della sessione.

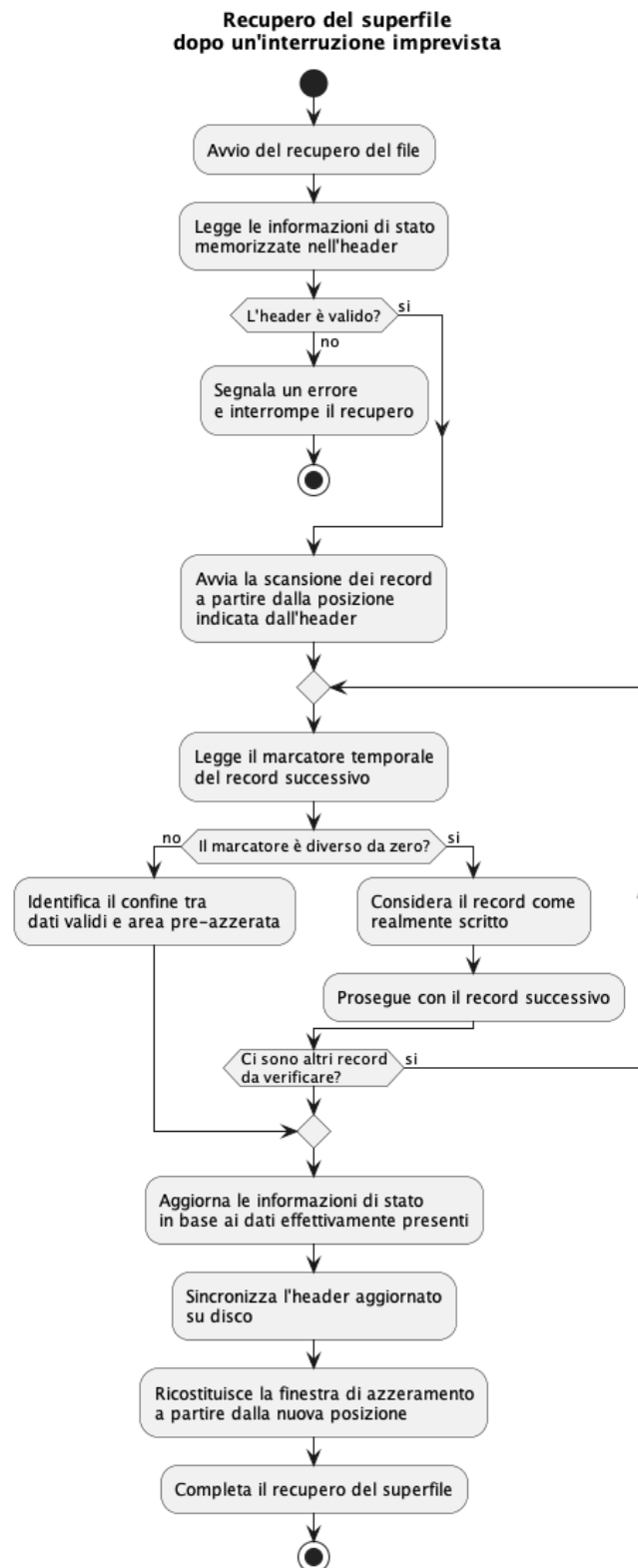


Figura 5.8: Flusso di recovery di un superfile dopo un'interruzione imprevista

5.3 Stato globale del dispositivo

5.3.1 Architettura

Lo stato globale del dispositivo rappresenta il punto di accesso centralizzato allo stato operativo, condiviso in lettura e scrittura da un numero elevato di componenti indipendenti: la gestione della sessione, la segnalazione mediante LED, la gestione dell'inattività, lo stato della microSD, la calibrazione dell'IMU, il modulo di comunicazione BLE e tutti i task producer/consumer.

Concettualmente espone un insieme di indicatori booleani indipendenti: supporto di memorizzazione mancante, condizione di errore, calibrazione in corso, sospensione per inattività, *pairing* in corso e download/cancellazione in corso, insieme a uno stato principale di sessione, con un numero finito di valori: `STAND_BY`, `LOGGING`, `SUSPENDED`, `DOWNLOADING` e `DELETING`. Ciascun indicatore è accessibile esclusivamente tramite un'interfaccia dedicata di lettura e scrittura.

Il modulo era già presente, in forma preliminare, nella base di codice ricevuta all'inizio del tirocinio. Tuttavia, l'implementazione originale gestiva solo un sottoinsieme degli stati e non veniva utilizzata in tutte le transizioni previste dal codice. È stato pertanto completamente riprogettato e reimplementato nell'ambito del lavoro svolto durante il tirocinio.

5.3.2 Scelte progettuali

- **Facade su stato condiviso atomico:** ogni variabile di stato è rappresentata come un'unità atomica ed è accessibile esclusivamente attraverso un'interfaccia uniforme di lettura e scrittura.

Vantaggio: consente di gestire in modo sicuro gli accessi concorrenti ai singoli indicatori senza ricorrere a meccanismi di mutua esclusione, poiché le operazioni di lettura e scrittura sono atomiche e indipendenti tra loro. Ciò riduce l'overhead associato all'utilizzo di mutex e limita il rischio di contesa tra i componenti che accedono allo stato condiviso.

- **Single Source of Truth:** nessun componente mantiene una propria copia locale dello stato del dispositivo; ogni decisione (ad esempio se il LED deve lampeggiare rosso, se un nuovo comando è accettabile, se sospendere i sensori) viene presa interrogando direttamente lo stato globale.

Vantaggio: evita disallineamenti tra componenti diversi che, in una versione precedente basata su variabili locali duplicate, potevano restare temporaneamente in stati incoerenti tra loro.

Analogamente, la Figura 5.9 restringe il campo ai soli moduli coinvolti nella segnalazione di stato e nel monitoraggio dei sensori, argomenti delle due sezioni immediatamente seguenti: la segnalazione tramite LED e la gestione dell'inattività dei sensori.

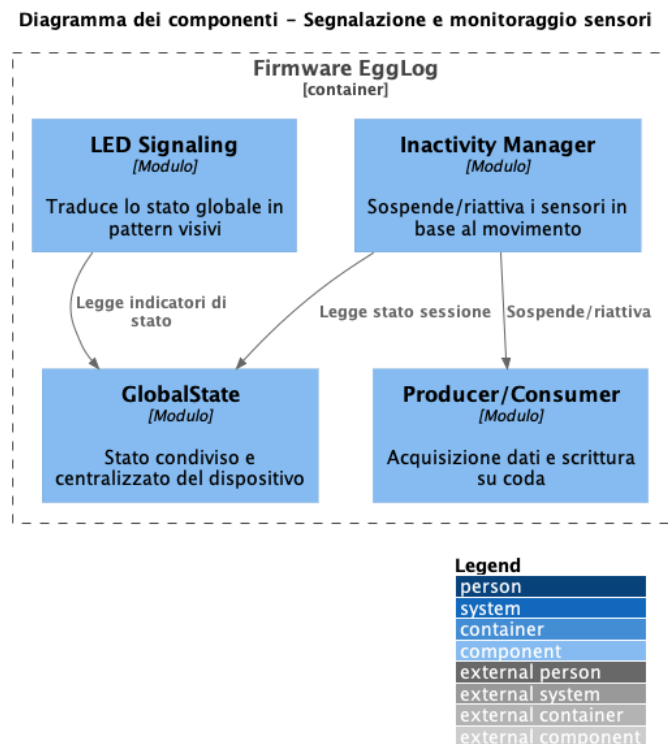


Figura 5.9: Diagramma dei componenti del firmware EggLog: segnalazione e monitoraggio sensori

5.4 Segnalazione di stato tramite LED

5.4.1 Architettura

Il sottosistema di segnalazione LED è organizzato concettualmente in tre responsabilità nettamente separate

- **Livello hardware:** incapsula esclusivamente il pilotaggio fisico del LED, isolando il resto del sistema dai dettagli dell'hardware.
- **Livello decisionale:** interroga lo stato globale del dispositivo e determina il pattern visivo da mostrare, senza accedere direttamente all'hardware.
- **Livello di rendering:** esegue un ciclo periodico, richiama il livello decisionale e traduce il pattern restituito in un'animazione effettiva, gestendo la temporizzazione, il lampeggio e la dissolvenza.

5.4.2 Scelte progettuali

- **Risoluzione a priorità esplicita:** il livello di decisione valuta lo stato del dispositivo secondo un ordine di priorità fisso ed esplicito (Tabella 5.1):

Priorità	Stato	Colore	Animazione
1	microSD mancante	Rosso	Lampeggio rapido
2	Errore	Rosso	Lampeggio lento
3	Pairing BLE	Blu	Lampeggio
4	Pairing Wi-Fi	Blu	Lampeggio veloce
5	Recupero dati durante la calibrazione	Giallo	Luce fissa
6	Calibrazione attiva	Giallo	Dissolvenza
7	Download	Viola	Fisso
8	Cancellazione	Viola	Lampeggio
9	Sospensione	Verde	Dissolvenza lenta
10	Logging	Verde	Luce fissa
11	Standby	Bianco	Luce fissa

Tabella 5.1: Priorità e segnalazioni visive del sottosistema LED

Vantaggio: garantisce che, in presenza di più condizioni vere contemporaneamente (ad esempio, microSD mancante durante il logging), l'utente veda sempre l'informazione più critica, invece di un comportamento indefinito o di un LED che cambia in modo instabile tra più stati concorrenti.

- **Value Object per il pattern visivo:** il pattern da visualizzare è rappresentato come un dato immutabile che descrive le caratteristiche dell'animazione, quali colore, modalità di animazione e frequenza. Questa rappresentazione è mantenuta separata sia dalla logica che determina il pattern da mostrare sia dal componente responsabile della sua effettiva visualizzazione. *Vantaggio:* rende il livello di decisione verificabile in isolamento e disaccoppia completamente la logica di business dal dettaglio di rendering.

5.5 Gestione dell'inattività dei sensori

5.5.1 Architettura

Il meccanismo di gestione dell'inattività implementa la sospensione automatica dei sensori non essenziali dopo un periodo di inattività configurabile, e la loro riattivazione al rilevamento di movimento. La sorgente di riferimento primaria per il rilevamento del movimento è il GPS, mentre l'accelerometro dell'IMU viene utilizzato come sorgente di **fallback**, attiva solo quando il GPS non è considerato affidabile; tipicamente in assenza di un *fix* GPS_G valido o recente, come potrebbe accadrebbe in ambienti con scarsa visibilità satellitare (gallerie o zone urbane dense).

5.5.2 Scelte progettuali

- **Strategy con sorgente autorevole commutabile:** una funzione di decisione determina, in base alla freschezza dell'ultimo fix GPS ricevuto, quale delle due sorgenti (GPS o IMU) debba governare la decisione di sospensione. Il percorso di rilevamento basato su IMU rimane sempre attivo, ma il suo risultato viene ignorato quando il GPS è considerato affidabile.

Vantaggio: i due percorsi di rilevamento rimangono indipendenti, mentre la selezione della sorgente autorevole è centralizzata in un unico punto di decisione, rendendo la logica più semplice da verificare e modificare.

- **$debounce_G$ basato su campioni consecutivi:** il rilevamento del movimento tramite GPS viene considerato valido solo dopo aver ricevuto un numero minimo di campioni consecutivi al di sopra della soglia prevista. Il conteggio viene azzerato quando viene rilevato un campione al di sotto della soglia.

Vantaggio: evita che singole misure anomale, dovute al rumore del segnale o agli effetti di *multipath* $_G$, in particolare in ambiente urbano, provochino riattivazioni spurie dei sensori sospesi.

- **Sospensione esclude il GPS:** la sospensione e la riattivazione, propagate a tutti i task producer, escludono esplicitamente il GPS.

Vantaggio: il GPS rimane attivo durante la sospensione degli altri sensori, permettendo sia di rilevare autonomamente l'eventuale ripresa del movimento, sia di mantenere il fix già acquisito. Alla riattivazione degli altri sensori non è quindi necessario attendere una nuova acquisizione del fix, riducendo il buco di dati tra la sospensione e la ripresa della registrazione completa.

5.5.3 Configurazione e comportamento ai bordi

Il timeout di inattività è configurabile dall'utente tramite l'app tra i valori "mai", 10, 30 o 60 minuti; il valore "mai" disabilita completamente la logica di sospensione, mantenendo i sensori sempre attivi indipendentemente dal movimento rilevato.

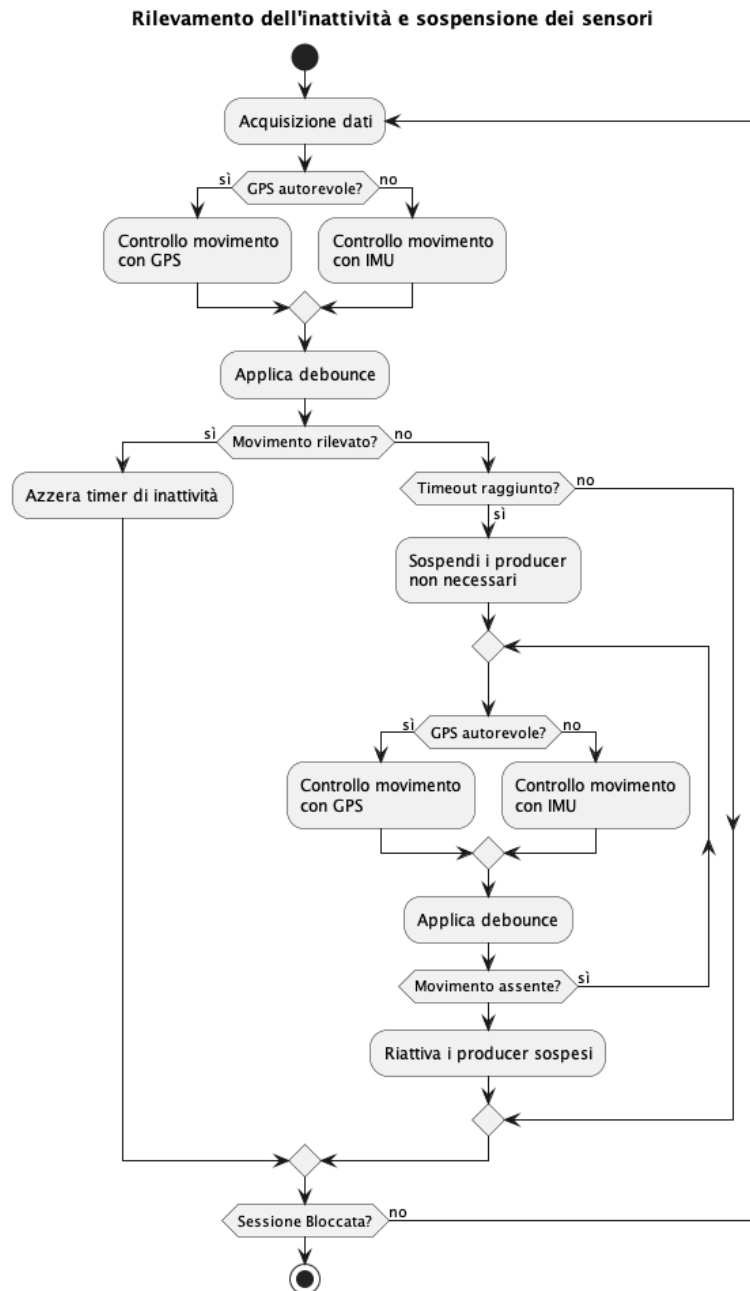


Figura 5.10: Transizioni alla sospensione guidate da GPS con fallback IMU

In modo analogo, la Figura 5.11 mette a fuoco i moduli coinvolti nella gestione della sessione e nelle calibrazioni, argomenti delle tre sezioni seguenti: il protocollo di avvio e arresto della sessione, la calibrazione dell'altitudine e la calibrazione dell'IMU.

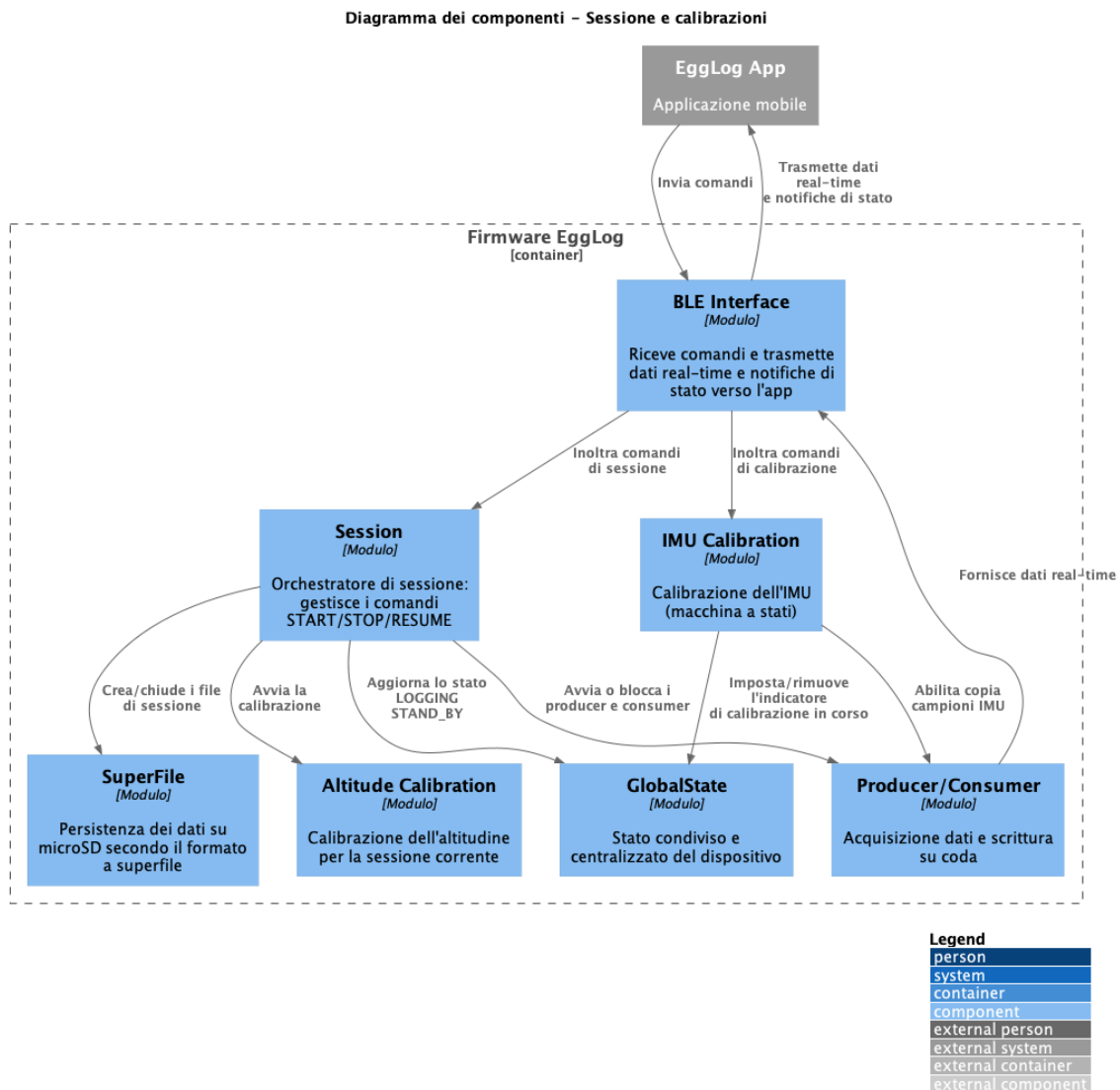


Figura 5.11: Diagramma dei componenti del firmware EggLog: sessione e calibrazioni

5.6 Protocollo di avvio e arresto della sessione

5.6.1 Architettura

Il modulo di gestione della sessione si colloca come livello di orchestrazione sopra le componenti preesistenti: riceve i comandi dall'app tramite BLE, coordina la creazione e la chiusura dei superfile (approfondito in [superfile](#)), l'esecuzione della calibrazione dell'altitudine (approfondita in [calibrazione altitudine](#)) e gli aggiornamenti dello stato globale del dispositivo, senza intervenire direttamente sulla logica interna di acquisizione dei singoli sensori.

5.6.2 Scelte progettuali

- **Command Pattern:** i comandi provenienti dall'app (START, STOP, RESUME) sono rappresentati come valori distinti di un insieme chiuso e instradati da un unico punto di gestione, disaccoppiando l'esecuzione del comando dalla sua sorgente.

Vantaggio: la logica di gestione della sessione risulta verificabile e riutilizzabile anche con trigger diversi dal BLE, senza modifiche. In particolare, RESUME non determina una transizione di stato, ma consente all'applicazione di richiedere una risincronizzazione immediata con lo stato corrente del dispositivo, risultando utile dopo una riconnessione BLE avvenuta mentre una sessione era già in corso.

- **State Pattern:** le transizioni di stato del dispositivo (STAND_BY, LOGGING, SUSPENDED, DOWNLOADING, DELETING) sono gestite centralmente dallo stato globale, e ogni decisione dipende esclusivamente dallo stato corrente.

Vantaggio: elimina condizionali diffusi nel codice che replicherebbero la stessa logica di stato in più punti, centralizzando le regole di transizione in un solo posto.

5.6.3 Avvio sessione

Alla ricezione del comando di avvio, se il dispositivo non è occupato in un'operazione di download o cancellazione e non è già in una sessione attiva, il firmware esegue in sequenza:

1. un tentativo di recovery della scheda microSD; in caso di fallimento, la sessione non viene avviata e viene notificato l'errore all'app;
2. la generazione di un identificativo univoco di sessione, ottenuto combinando: l'identificativo del dispositivo, un riferimento temporale di inizio sessione e il marcatore `_ACTIVE` che segnala la sessione come ancora in corso;
3. la creazione del contenitore di sessione e dei relativi superfile per tutti i sensori;
4. l'esecuzione sincrona della calibrazione dell'altitudine, il cui esito viene applicato immediatamente come riferimento di pressione per la sessione;
5. la scrittura di un insieme di metadati di sessione, contenente l'esito della calibrazione altitudine, il valore di pressione di riferimento ed eventuali condizioni di timeout;
6. l'aggiornamento dello stato globale a LOGGING, la riattivazione forzata dei produttori di dati e l'avvio di una notifica periodica dello stato di sessione verso l'app.

Ogni fase è protetta da un accesso esclusivo al canale di comunicazione condiviso, con condizioni di timeout configurabili: in caso di mancata acquisizione dell'accesso esclusivo o di fallimento di una delle operazioni, il firmware imposta lo stato di errore e interrompe l'avvio, notificando comunque lo stato aggiornato all'app.

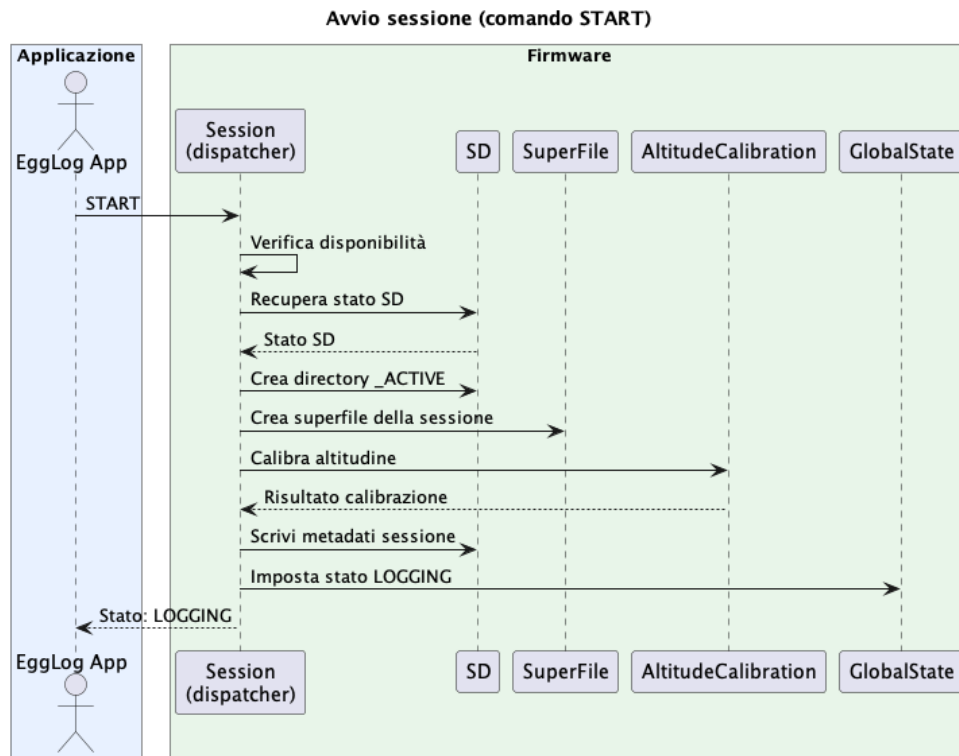


Figura 5.12: Diagramma di sequenza del comando START

5.6.4 Arresto sessione

Il comando di arresto (così come l'arresto automatico per esaurimento spazio GPS) innesca una sequenza di chiusura pensata per non perdere dati residui nei buffer:

1. se il dispositivo era in stato `SUSPENDED`, i produttori di dati vengono prima riattivati forzatamente;
2. viene richiesto lo svuotamento sincrono di tutti i buffer dei sensori verso il meccanismo di scrittura, con timeout;
3. il firmware attende che tutte le richieste di scrittura pendenti vengano completate;
4. tutti i superfile della sessione vengono chiusi, sempre sotto accesso esclusivo al canale condiviso;
5. il contenitore di sessione viene rinominato, sostituendo il marcatore `_ACTIVE` con un riferimento temporale di fine sessione;
6. lo stato torna a `STAND_BY` e la notifica periodica viene interrotta.

Un caso particolare è l'arresto automatico per esaurimento dello spazio dedicato al GPS: il firmware imposta un indicatore dedicato prima di eseguire la stessa procedura di arresto, distinguendo per l'app un arresto voluto dall'utente da uno automatico.

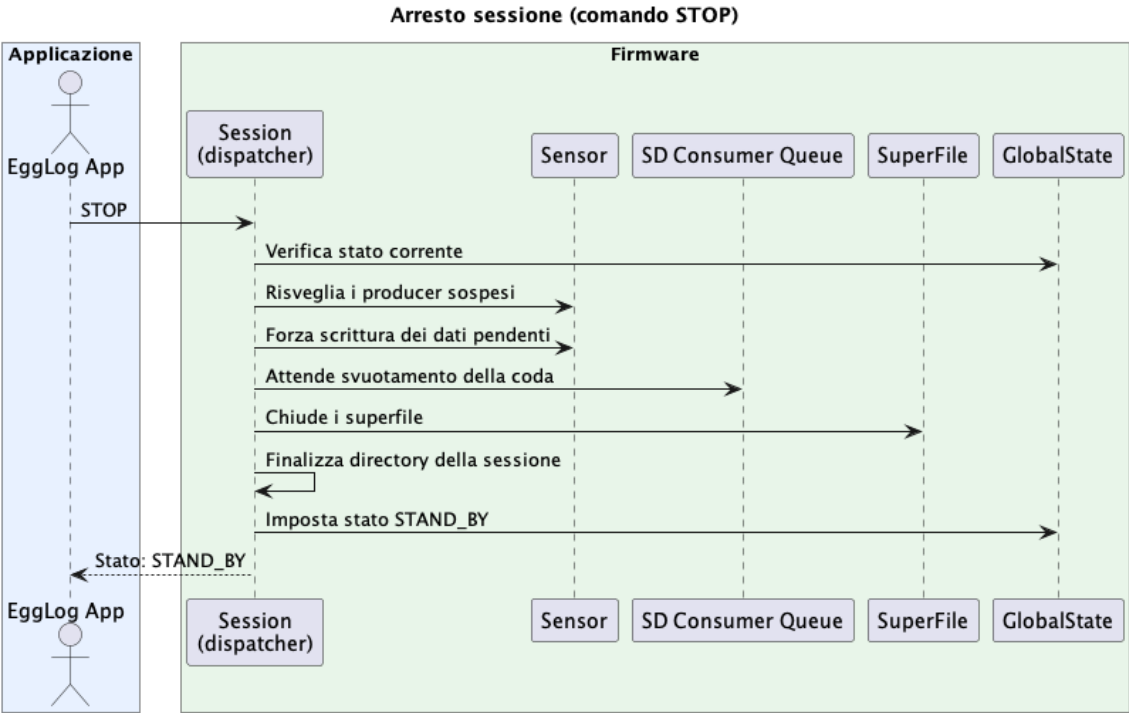


Figura 5.13: Diagramma di sequenza del comando STOP

5.7 Calibrazione dell'altitudine

Per il calcolo dell'altitudine è stato scelto il barometro come sorgente di riferimento, anziché il GPS, già presente sul dispositivo per la posizione e la velocità. Nelle prove effettuate, l'altitudine riportata dal GPS ha mostrato un errore fino a circa 25 metri, un margine incompatibile con la necessità di rilevare variazioni di quota anche modeste durante il tragitto; il barometro, per contro, ha mostrato un errore contenuto entro circa 2 metri nelle stesse condizioni di prova. Il GPS mantiene quindi il solo ruolo di sorgente di posizione e velocità, mentre il barometro diventa l'unica sorgente utilizzata per l'altitudine.

5.7.1 Architettura

La calibrazione dell'altitudine è invocata in modo sincrono all'interno del flusso di avvio sessione, prima della transizione a LOGGING, e si basa sull'ultima lettura disponibile del barometro, senza richiedere un canale dedicato di comunicazione con il relativo sensore.

5.7.2 Algoritmo di calibrazione

La calibrazione dell'altitudine garantisce che, se disponibile, il riferimento di pressione sia valido fin dal primo campione registrato nella sessione. L'algoritmo raccoglie ripetutamente campioni di pressione, filtrando i duplicati tramite confronto del riferimento temporale. Raggiunto un numero sufficiente di campioni, viene calcolata la **mediana** del gruppo (scelta più robusta della media rispetto a campioni anomali) e la relativa **deviazione standard**: se questa è sotto una soglia prestabilita, il valore mediano viene accettato come riferimento; altrimenti il gruppo viene scartato e la raccolta ricomincia. L'intera procedura è racchiusa in un timeout massimo, oltre il quale la calibrazione è marcata come fallita e la sessione prosegue comunque, senza riferimento di altitudine valido.

A differenza della calibrazione IMU, valida per tutte le sessioni successive fino a nuova ricalibrazione, la calibrazione dell'altitudine è **specifica di ogni singola sessione**: viene ripetuta a ogni avvio e persistita solo nei metadati della sessione corrente.

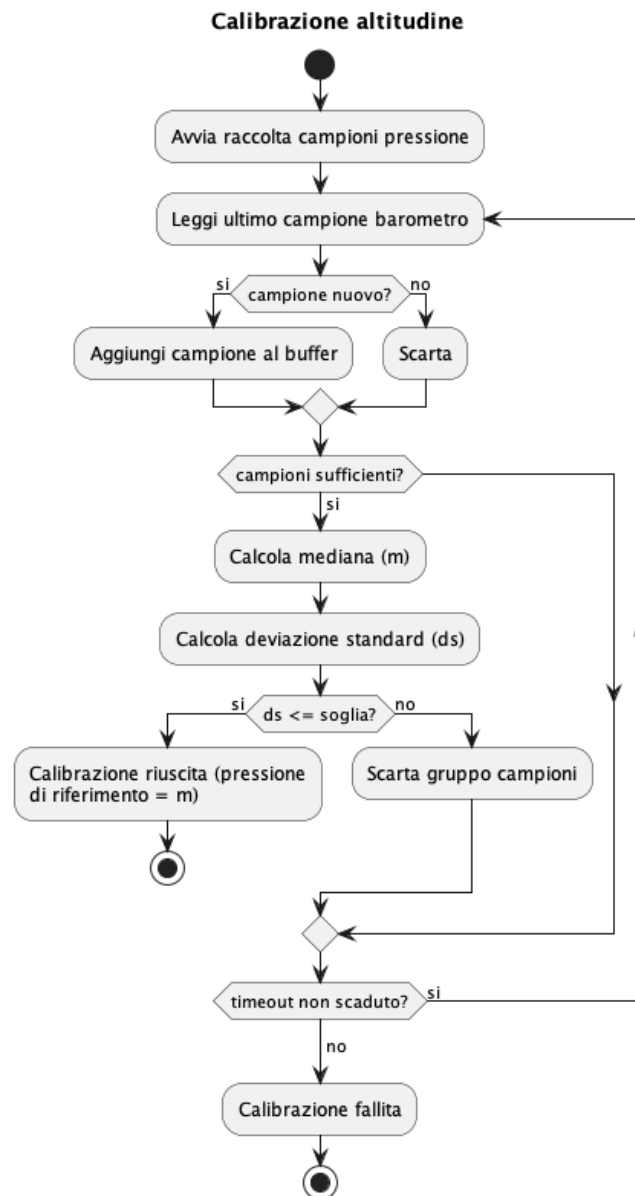


Figura 5.14: Diagramma di flusso della calibrazione altitudine

5.8 Calibrazione dell'IMU

Data l'importanza di questo modulo ai fini dell'accuratezza dei dati registrati, la progettazione della calibrazione dell'IMU è stata condotta con un livello di dettaglio maggiore rispetto agli altri componenti del sistema.

5.8.1 Architettura

La calibrazione dell'IMU è un'attività indipendente eseguibile esclusivamente nello stato `STAND_BY`, quindi mai durante una sessione attiva o in presenza di un errore. All'avvio, viene risvegliato il task producer dell'IMU, che acquisisce normalmente i campioni di accelerometro e giroscopio. Quando la calibrazione è attiva, una variabile booleana ne abilita la copia in un **buffer dedicato**, separato dai due buffer utilizzati per il normale *ping-pong buffering*. Il task di calibrazione consuma quindi questi campioni ed esegue le elaborazioni previste a ogni step, svuotando il buffer all'inizio di ciascuna fase per considerare esclusivamente i dati pertinenti.

La procedura è gestita tramite due distinti canali BLE: uno per i comandi inviati dall'app e uno per le notifiche con cui il firmware comunica in tempo reale lo stato della calibrazione.

5.8.2 Scelte progettuali

- **State Pattern:** l'intera procedura è modellata come una macchina a stati esplicita (inattiva, pronta per lo step n , step n in corso, step n da ripetere, dati assenti, successo, errore), con un unico punto di gestione che interpreta il comando ricevuto in funzione dello stato corrente.

Vantaggio: ogni stato ammette solo un sottoinsieme ben definito di transizioni valide, evitando che comandi ricevuti fuori sequenza (ad esempio una conferma prima di un avvio) producano comportamenti indefiniti.

- **Command Pattern:** i comandi provenienti dall'app (avvio, conferma, interruzione) sono valori distinti di un insieme chiuso, instradati tramite una coda verso l'unico punto di gestione.

Vantaggio: la ricezione del comando è completamente disaccoppiata dalla sua esecuzione, evitando di eseguire logica di calibrazione potenzialmente bloccante direttamente nel contesto di ricezione BLE.

- **Fan-out non invasivo sul flusso dati:** l'acquisizione IMU continua a funzionare esattamente come se la calibrazione non esistesse, limitandosi a duplicare ogni campione anche verso il canale dedicato quando la calibrazione è attiva.

Vantaggio: la funzionalità di calibrazione è pensata come estensione additiva dell'acquisizione esistente, senza modificarne la logica interna né rischiare di introdurre egressioni sul percorso dati principale.

- **Retry isolato per singolo step:** ogni fallimento per dati insufficienti riporta la macchina a uno stato di ripetizione specifico per lo step corrente, anziché allo stato d'errore.

Vantaggio: l'utente può ripetere solo lo step fallito, mantenendo il lavoro già validato negli step precedenti.

5.8.2.1 Algoritmo di calibrazione

Ogni step di misura si basa su un meccanismo comune di raccolta campioni, che integra tre elementi distinti:

- un intervallo di *assestamento* durante il quale i campioni ricevuti vengono scartati, per dare all'utente il tempo di portare il dispositivo nella posizione richiesta prima che la misura effettiva inizi;
- una raccolta a tempo, con calcolo incrementale di media e, dove richiesto, varianza del modulo dell'accelerazione;
- un meccanismo di **uscita anticipata**: superata una durata minima e un numero minimo di campioni, viene valutata la stabilità delle ultime letture; se il segnale risulta sufficientemente stabile, lo step termina prima dello scadere del timeout completo, riducendo i tempi di attesa percepiti dall'utente quando il dispositivo è già fermo nella posizione corretta.

5.8.2.1.1 Step 1 - Posizione dritta

Con il dispositivo fermo vengono raccolti i campioni dell'IMU per verificarne la stabilità e la coerenza con l'accelerazione di gravità attesa. In caso di esito positivo, lo step determina l'asse verticale del sistema di riferimento e i bias di accelerometro e giroscopio da applicare nelle acquisizioni successive; in caso contrario, lo step viene rifiutato.

5.8.2.1.2 Step 2 - Inclinazione a destra

Il dispositivo viene inclinato lateralmente rispetto al mezzo su cui è montato. Dallo scostamento rispetto alla posizione registrata nello step precedente viene derivato l'asse laterale del sistema di riferimento; un'inclinazione giudicata insufficiente comporta il rifiuto dello step.

5.8.2.1.3 Step 3 - Asse longitudinale e salvataggio

Il terzo asse del sistema di riferimento viene derivato dai due precedenti, completando la matrice di rotazione che descrive l'orientamento del dispositivo rispetto al mezzo. Una geometria giudicata degenera comporta la ripetizione della calibrazione. Al termine di tutti gli step, bias e matrice di rotazione vengono salvati nella configurazione e persistiti su microSD; un eventuale errore di scrittura porta la procedura nello stato di errore.

5.8.2.1.4 Gestione degli esiti negativi

Ogni step può concludersi in tre modi oltre al successo: assenza di dati, dati insufficienti e interruzione esplicita da comando utente, quest'ultima gestita riportando immediatamente lo stato a inattivo.

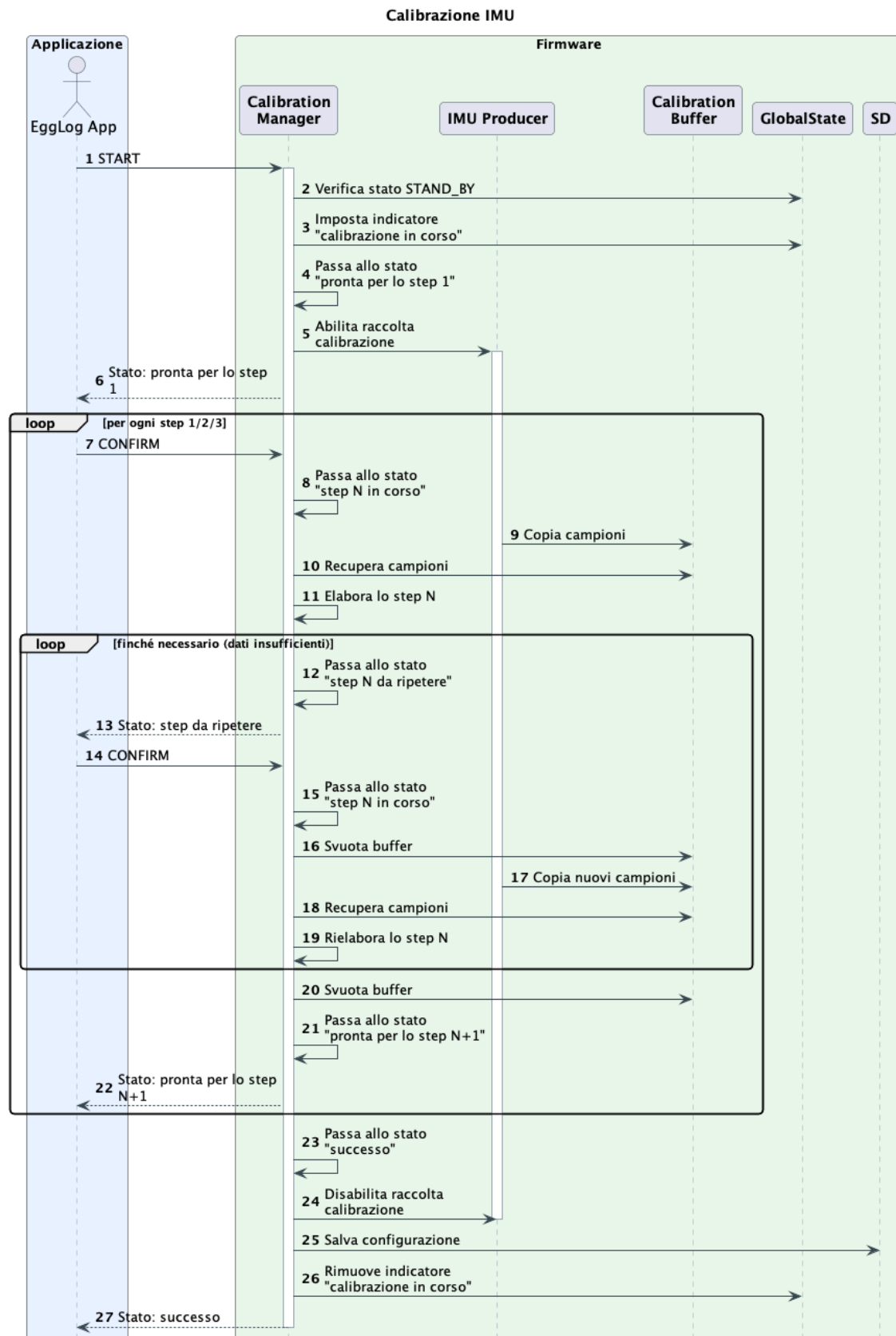


Figura 5.15: Diagramma della calibrazione dell'IMU

5.9 Valutazione tecnica sniffer CAN

Durante il tirocinio è stata condotta una **valutazione tecnica** dell'interfacciamento di un nuovo microcontrollore con il bus CAN del motoveicolo, propedeutica alla realizzazione di un nodo *sniffer* separato dal dispositivo EggLog principale. L'analisi ha riguardato sia l'aspetto **hardware**, relativo all'interfacciamento elettrico con il bus, sia l'aspetto **software**, relativo alla configurazione e alla gestione del controller CAN.

5.9.1 Hardware

Sono state valutate tre soluzioni hardware per l'interfacciamento elettrico con il bus, poste a confronto secondo un *trade-off* tra **costo** e **rischio elettrico**, sia per il motoveicolo che per il microcontrollore.

Opzione	Descrizione	Costo	Rischio
1	Controller CAN nativo integrato nel microcontrollore, con <i>transceiver_G</i> esterno e diodi (<i>TVS_G</i>) in serie a protezione	Basso	Alto
2	Modulo CAN esterno dedicato, con transceiver a 5V, collegato tramite un adattatore di livello logico bidirezionale e dei diodi (TVS) in serie a protezione	Medio-Alto	Medio-Basso
3	Transceiver CAN a <i>isolamento galvanico_G</i> completo	Alto	Basso

Tabella 5.2: Confronto tra le soluzioni considerate per l'interfacciamento CAN

5.9.1.1 Opzione 1 - protezione passiva

Il controller CAN nativo del microcontrollore viene utilizzato con un transceiver esterno e dei diodi TVS dedicato alla soppressione dei *transienti_G*, posto a protezione delle linee CAN in prossimità del connettore d'ingresso.

5.9.1.2 Opzione 2 - modulo esterno con adattatore logico

Impiego di un modulo CAN esterno con adattamento di livello logico bidirezionale. Questa opzione era stata inizialmente valutata nell'ottica di integrare la logica di traduzione CAN direttamente nel nodo centrale EggLog; tale scelta architettuale è stata tuttavia rivista nel corso della stessa valutazione tecnica, optando per un nodo sniffer separato e modulare, anche a causa dei limitati spazi disponibili sotto il codino della moto.

5.9.1.3 Opzione 3 - isolamento galvanico completo

Impiego di un transceiver CAN a isolamento galvanico integrato, sia di segnale sia di alimentazione, disponibile in commercio come componente singolo che integra isolatore digitale e convertitore di alimentazione isolato in un unico pacchetto. Elimina completamente il riferimento di massa comune tra il nodo sniffer e il veicolo, a fronte di costo e complessità di progettazione elettronica nettamente superiori.

5.9.2 Scelta dell'Opzione 1 e schema elettrico del prototipo

A completamento della valutazione tecnica delle tre soluzioni di interfacciamento (Tabella 5.2), questa sezione approfondisce lo schema elettrico dell'**Opzione 1**, scelta per la realizzazione del prototipo. Tale soluzione è stata preferita in considerazione della natura ancora **esplorativa del progetto**, che ha reso prioritario disporre di un'architettura semplice e modulare, adeguata alla fase iniziale di sperimentazione. Lo schema viene quindi analizzato nel dettaglio, con particolare attenzione ai rischi elettrici individuati e alle relative mitigazioni adottate per renderli accettabili nel contesto del prototipo.

5.9.2.1 Componenti dello schema

EggLog CAN Sniffer è realizzato con i seguenti componenti:

- **Microcontrollore:** ESP-WROOM-32, con controller CAN (*TWAI_G*) nativo integrato;
- **Transceiver CAN:** modulo *Waveshare SN65HVD230 CAN Board*¹, alimentato a 3.3V. Espone i pin: VCC 3.3v per l'alimentazione, GND, CAN_TX e CAN_RX (come ingresso e uscita dati verso il microcontrollore), CAN_H e CAN_L;
- **Protezione:** diodo TVS *onsemi NUP2105LT1G*², *dual-channel*, posizionato in derivazione lungo le linee CAN_H e CAN_L, vicino al connettore diagnostico;
- **Massa:** collegamento a Y che parte dal GND dello shield CAN e arriva al GND della presa diagnostica della moto e il GND dell'ESP-WROOM-32.

Le linee dati verso il microcontrollore CAN_TX/CAN_RX restano a livello 3.3V, direttamente compatibili con i GPIO dell'ESP-WROOM-32: non è quindi necessario alcun adattatore di livello logico, a differenza di quanto previsto per l'Opzione 2 con modulo a 5V.

5.9.2.2 Schema elettrico

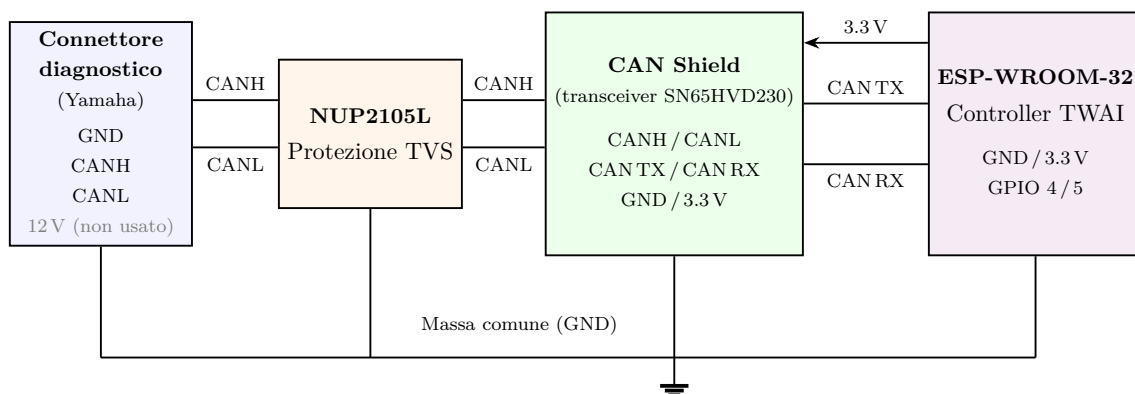


Figura 5.16: Schema elettrico di EggLog CAN Sniffer.

¹Datasheet di SN65HVD230. URL: <https://files.waveshare.com/upload/3/3a/SN65HVD230-CAN-Board-Datasheets.pdf>.

²Datasheet di NUP2105LT1G. URL: <https://www.onsemi.cn/pdf/datasheet/nup2105l-d.pdf>.

Lo schema evidenzia i tre punti chiave della soluzione adottata:

1. il TVS **NUP2105L** è inserito *in derivazione* su entrambe le linee **CAN_H/CAN_L**, il più vicino possibile all'ingresso, cosicché qualunque transiente proveniente dal bus moto venga limitato prima di raggiungere il transceiver dello shield;
2. le linee digitali **TX/RX** tra shield ed ESP-WROOM-32 non richiedono protezione aggiuntiva né adattamento di livello, essendo lo shield già a 3.3V;
3. la massa è condivisa tra moto, shield ed ESP-WROOM-32, poiché l'assenza di isolamento galvanico richiede un riferimento elettrico comune e consente di mantenere le tensioni di modo comune delle linee CAN entro l'intervallo operativo del transceiver.

Il prototipo è pensato per una **Yamaha MT-07**, la cui porta diagnostica è del tipo *Sumitomo* a 4 pin: GND, 12 V, **CAN_H** e **CAN_L**. È stato quindi necessario prevedere l'aggiunta di un adattatore dedicato a tale connettore.

Per estendere la compatibilità ad altri motoveicoli, uno sviluppo futuro prevede l'implementazione degli adattatori per i restanti formati più diffusi (3 pin, 6 pin e il connettore OBD standard).

5.9.2.3 Analisi dei rischi

Rispetto al confronto generale di Tabella 5.2, il rischio elettrico più elevato dell'Opzione 1 deriva dall'assenza di isolamento galvanico tra il nodo sniffer e il bus del veicolo: shield ed ESP-WROOM-32 condividono lo stesso riferimento di massa delle centraline originali. I rischi specifici individuati e le relative mitigazioni sono riassunti di seguito.

Rischio	Descrizione	Mitigazione
Transienti sulle linee CAN	Scariche elettrostatiche o disturbi impulsivi del veicolo (accensione, bobine, motorino di avviamento) possono superare la tensione di breakdown dei pin del transceiver	TVS NUP2105L in serie: $V_{wm} = 24V$, $V_{br} = 26.2V$, $P_{ppm} = 350W/linea$, $I_{ppm} = 8A$
Interferenza con il bus del veicolo	Un nodo software difettoso potrebbe inviare ACK_G o frame di errore, disturbando la comunicazione tra le centraline	Controller TWAI configurato in modalità listen-only : nessun ACK né frame di errore attivo, indipendentemente da bug software
Riferimento di massa flottante	Se la massa dello shield non coincide con quella della moto, il transceiver può superare il proprio range di modo comune, causando errori di ricezione o stress sul componente	Collegamento di massa a Y tra moto, shield ed ESP-WROOM-32, sullo stesso nodo

Tabella 5.3: Rischi elettrici e relative mitigazioni

La combinazione delle tre mitigazioni rende il rischio residuo coerente con l'obiettivo del prototipo, cioè verificare la fattibilità della soluzione minimale prima di eventualmente investire nelle Opzioni 2 o 3, che restano disponibili come evoluzione futura senza modifiche architetturali al resto del sistema.

5.10 Architettura dello sniffer CAN

5.10.1 Architettura

L'architettura software di EggLog CAN Sniffer è progettata secondo un'architettura **esagonale**, scelta motivata dal requisito più delicato del sistema: gli identificativi dei frame CAN e le formule di decodifica variano da marca a marca, e spesso anche tra modelli diversi dello stesso costruttore. Un'architettura a livelli tradizionale tenderebbe a concentrare questa conoscenza nel livello di logica applicativa sotto forma di condizionali crescenti nel tempo.

Il dominio espone tre porte principali, ciascuna corrispondente a una responsabilità concettualmente indipendente:

- una porta che astrae l'accesso al bus CAN fisico, implementata da un adattatore basato sul controller CAN in modalità ascolto passivo;
- una porta che astrae la decodifica specifica del veicolo, implementata da un adattatore dedicato per ciascun modello di veicolo supportato;
- una porta che astrae il canale di trasporto verso il nodo centrale, implementata da un adattatore basato su advertising BLE.

L'orchestrazione è affidata a un unico caso d'uso, che non conosce alcun dettaglio implementativo degli adattatori concreti, come illustrato in Figura [5.17](#).

Diagramma dei componenti – Nodo Sniffer CAN (architettura esagonale)

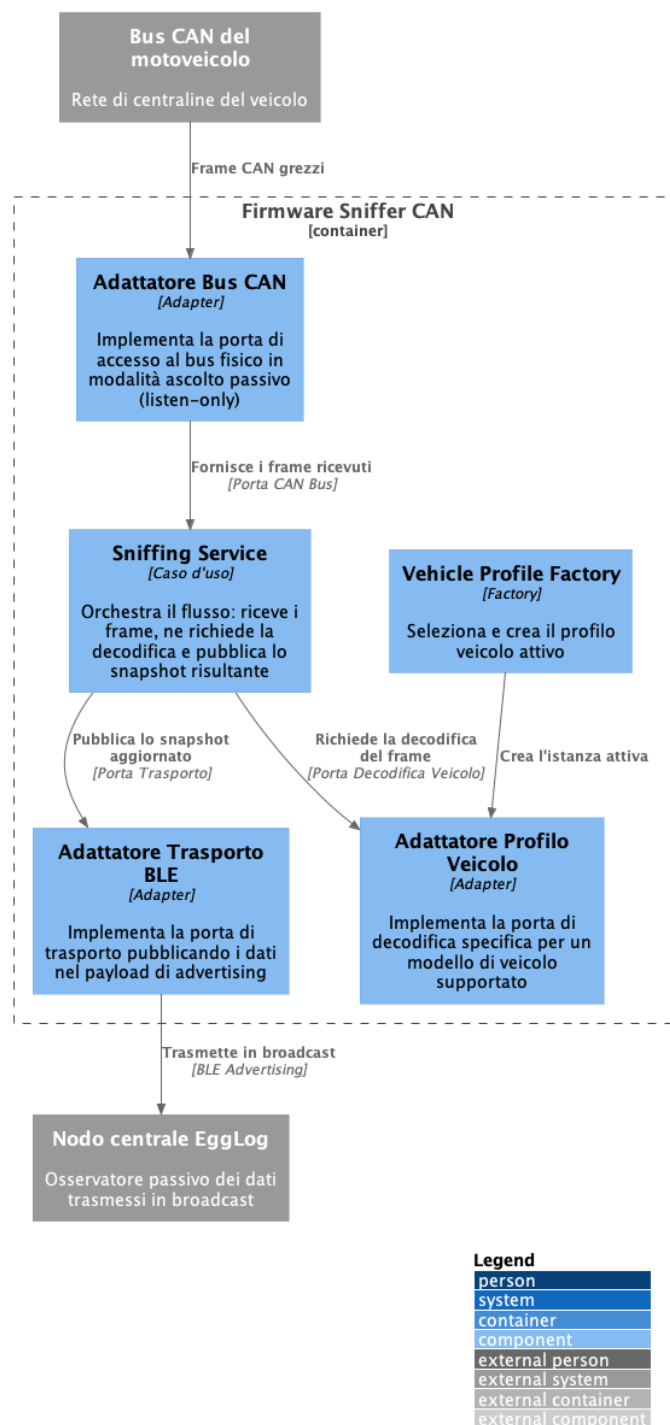


Figura 5.17: Diagramma dei componenti di EggLog CAN Sniffer: architettura esagonale

5.10.2 Scelte progettuali

- **Strategy Pattern:** la conoscenza del veicolo è isolata dietro la porta di decodifica, di cui ogni modello supportato fornisce un adattatore concreto.
Vantaggio: la strategia di decodifica è selezionabile e sostituibile senza toccare dominio o logica applicativa, rendendo l'aggiunta di un nuovo veicolo un'estensione isolata anziché una modifica pervasiva.

- **Factory Pattern:** la selezione del profilo veicolo attivo è centralizzata in un unico punto di creazione.

Vantaggio: aggiungere il supporto a un nuovo modello richiede solo un nuovo adattatore e la relativa voce nel punto di creazione, senza modificare altre parti del firmware.

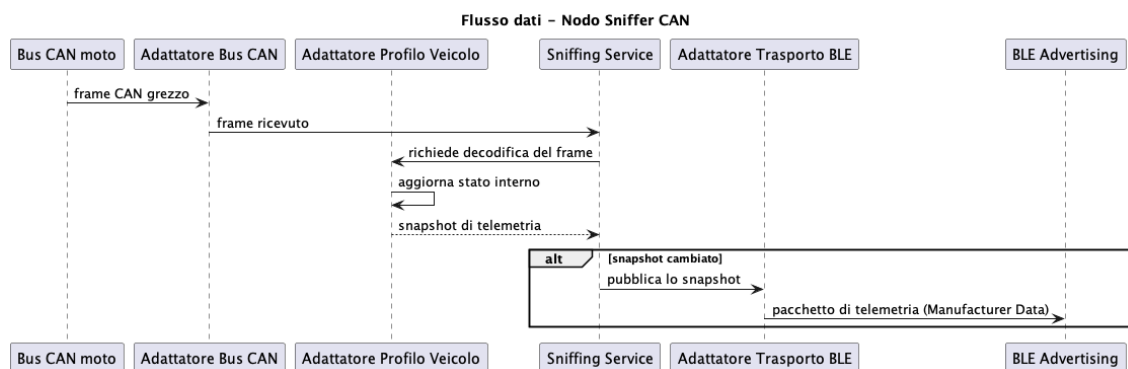


Figura 5.18: Flusso dati dal bus CAN al payload BLE

5.10.3 Trasporto dei dati: BLE advertising broadcast

Il requisito di trasmissione continua e indipendente dalla presenza di un ricevitore attivo ha escluso l'uso di un canale BLE classico basato su connessione con notifiche: senza un ricevitore connesso e sottoscritto, le notifiche non avrebbero alcun canale su cui viaggiare. La soluzione adottata sfrutta invece il payload di **advertising** BLE: EggLog CAN Sniffer non espone alcun servizio dedicato e non accetta connessioni, trasmettendo i dati all'interno del pacchetto di advertising stesso. Il nodo centrale EggLog agisce da semplice osservatore passivo, leggendo il payload durante le proprie scansioni senza mai stabilire una connessione: un modello *publish-only*, tollerante alle perdite, coerente con un flusso di telemetria senza garanzie di consegna per il singolo campione.

Il pacchetto trasmesso include la versione del protocollo, il profilo veicolo attivo, marcia, apertura farfalla, RPM, velocità, temperature motore e aria, e un contatore progressivo per rilevare pacchetti persi o duplicati.

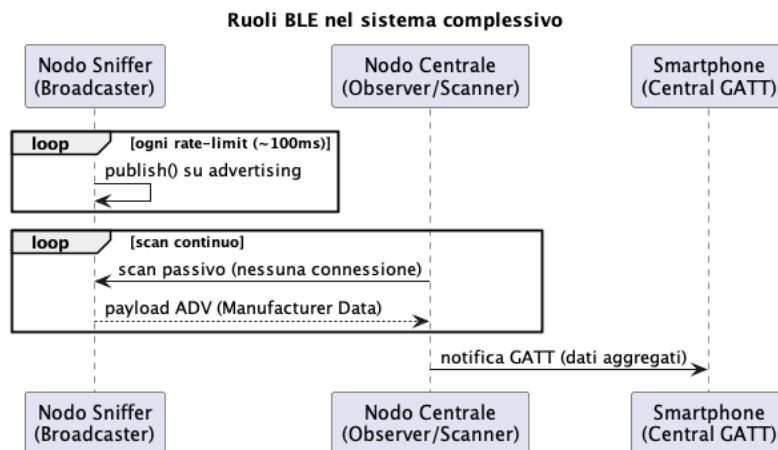


Figura 5.19: I tre ruoli BLE nel sistema complessivo

In modo analogo a quanto fatto per il firmware, la Figura 5.20 mette a fuoco i moduli dell'applicazione coinvolti nelle sezioni seguenti: la visualizzazione 3D del veicolo, la segnalazione di microSD mancante e la configurazione dei parametri del dispositivo.

Diagramma dei componenti – Applicazione: schermata Record e configurazione

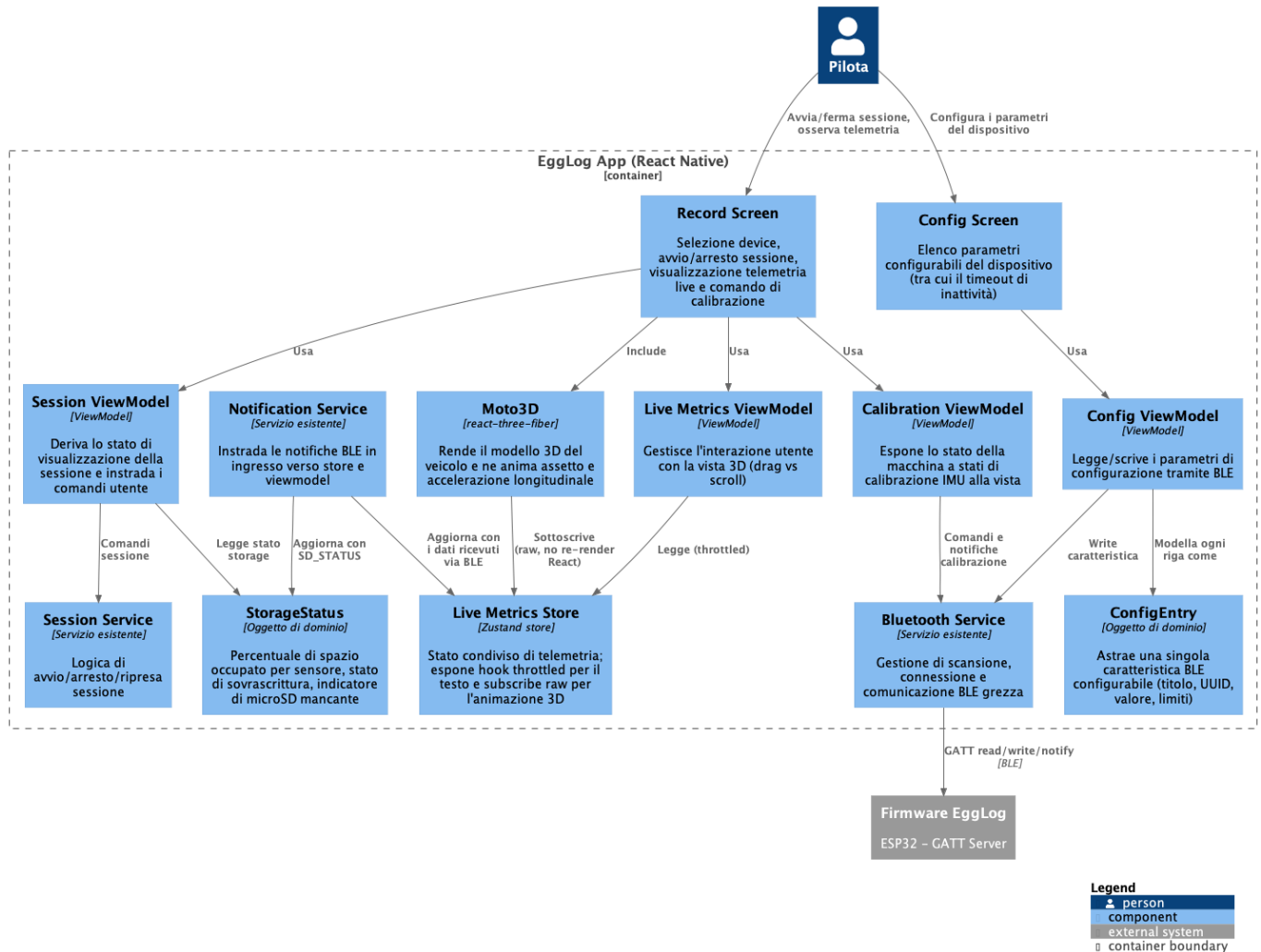


Figura 5.20: Diagramma dei componenti dell'applicazione EggLog: schermata Record e configurazione

5.11 Visualizzazione 3D del veicolo

Il modello 3D del veicolo è un componente introdotto durante il tirocinio, assente nella versione precedente dell'applicazione. Una prima versione lo inseriva semplicemente nell'albero di componenti React della schermata di registrazione, allo stesso livello degli elementi testuali di telemetria (velocità, quota, angoli di inclinazione): ogni notifica BLE in arrivo aggiornava lo stato React condiviso e innescava quindi un nuovo render dell'intera schermata, incluso il componente 3D. Poiché il render 3D è computazionalmente più oneroso di un semplice aggiornamento di testo, il risultato era un evidente rallentamento nella fluidità del movimento del modello, tanto più percepibile quanto più frequenti erano le notifiche in arrivo. La scelta progettuale descritta in questa sezione è la soluzione adottata fin dalla revisione successiva del componente: separare il rendering 3D dal ciclo di render React della schermata, facendogli leggere i dati di telemetria attraverso un canale proprio, indipendente dall'aggiornamento degli altri elementi della pagina.

5.11.1 Architettura

La visualizzazione 3D è incapsulata in un componente autonomo, separato dalla logica della schermata *Record*. Il componente riceve esclusivamente i callback necessari per le interazioni e accede direttamente allo store condiviso delle metriche live, evitando di propagare la telemetria attraverso la gerarchia dei componenti React.

5.11.2 Scelte progettuali

- **Game Loop Pattern (Disaccoppiamento acquisizione/rendering):** il componente 3D accede direttamente allo store e mantiene i valori ricevuti in un riferimento mutabile, letto a ogni frame del ciclo di rendering 3D. In questo modo, la frequenza di aggiornamento della visualizzazione non dipende dalla cadenza delle notifiche BLE. I valori vengono inoltre interpolati frame per frame anziché applicati istantaneamente.

Vantaggio: la fluidità dell'animazione non dipende dall'irregolarità con cui arrivano le notifiche BLE. Gli elementi testuali della stessa schermata, per i quali non è necessaria una frequenza di aggiornamento elevata, continuano invece a leggere lo stesso store tramite *hook* `useEffect` con frequenza limitata, evitando render superflui.

5.12 Segnalazione di microSD mancante durante la sessione

5.12.1 Architettura

Lo stato di occupazione della microSD, incluso l'indicatore booleano di assenza della scheda, è ricavato dalle notifiche della *Characteristic* dedicata del Service **STORAGE** e reso disponibile alla schermata *Record* come oggetto di dominio. Lo stesso oggetto porta con sé, per ciascun sensore, la percentuale di spazio occupato e l'indicatore di sovrascrittura in corso: un'informazione mostrata all'utente in ogni sessione attiva, indipendentemente dal fatto che la scheda sia presente o meno, per permettergli di accorgersi con anticipo dell'approssimarsi dell'esaurimento dello spazio disponibile. Sono stati individuati due punti distinti in cui la sola condizione di assenza della scheda deve essere comunicata con un'interruzione esplicita all'utente, riflettendo le due fasi già discusse lato firmware ([Comunicazione dello stato di occupazione](#)): prima dell'avvio di una sessione e durante una sessione già in corso.

5.12.2 Scelte progettuali

- **Presentational Component Pattern:** la segnalazione è realizzata come un unico componente parametrico (colore dell'azione, etichetta, stato di caricamento), riutilizzato nei due punti distinti del flusso anziché duplicato.

Vantaggio: garantisce all'utente un linguaggio visivo coerente per la stessa condizione di errore, indipendentemente dal momento in cui si presenta, riducendo al contempo la duplicazione di codice tra i due casi d'uso.

- **Strategy Pattern:** nel primo caso, un tentativo di avvio sessione rifiutato dal firmware per assenza della microSD offre un'azione di ripetizione (*retry*); nel secondo caso, la comparsa dell'indicatore durante una sessione già in corso offre esclusivamente un'azione di arresto forzato, poiché non ha senso continuare una registrazione priva di un supporto su cui persistere i dati.

Vantaggio: l'azione proposta all'utente riflette correttamente ciò che è effettivamente recuperabile nel contesto specifico, evitando di suggerire un tentativo di ripetizione quando ormai non porterebbe ad alcun beneficio.

- **Blocking Modal Pattern:** in nessuno dei due casi l'overlay può essere chiuso senza intraprendere l'azione proposta.

Vantaggio: impedisce che l'utente continui a interagire con una sessione che, di fatto, non può più salvare correttamente i dati acquisiti, richiedendo una presa d'atto esplicita del problema.

- **Ripristino automatico alla scomparsa della condizione:** nessuno dei due overlay mantiene uno stato proprio; entrambi sono condizionati direttamente dall'indicatore booleano di assenza della microSD, aggiornato in tempo reale dalle notifiche BLE.

Vantaggio: se la microSD viene reinserita durante una sessione, l'utente non deve compiere alcuna azione per far scomparire l'overlay: non appena il firmware segnala

il ripristino, l'app smette semplicemente di mostrarlo e la sessione, già ripresa lato firmware, torna visibile senza interruzioni percepite nell'interfaccia.

- **Ripartizione per sensore, non solo indicatore aggregato:** la percentuale di occupazione è mostrata separatamente per ciascun sensore, anziché come singolo valore complessivo del dispositivo.

Vantaggio: sensori con capacità e frequenza di scrittura molto diverse tra loro (ad esempio l'IMU rispetto al barometro) riempiono il proprio spazio a velocità differenti; un valore aggregato nasconderebbe quale specifico sensore si sta avvicinando al limite, mentre la ripartizione per sensore permette all'utente di correlare l'informazione con l'eventuale imminente interruzione automatica della sessione per esaurimento spazio GPS, già descritta lato firmware.

5.13 Configurazione dei parametri del dispositivo

5.13.1 Architettura

La schermata di configurazione presenta un elenco di parametri modificabili dall'utente, ciascuno rappresentato da un oggetto di dominio che astrae una singola Characteristic BLE configurabile (titolo descrittivo, Service e Characteristic di riferimento, valore corrente, eventuali limiti). Il tipo di controllo grafico con cui ciascun parametro viene mostrato non è fisso, ma **selezionato in base al tipo di Characteristic** che l'oggetto rappresenta.

5.13.2 Scelte progettuali

- **Adapter/Facade Pattern:** la vista non conosce mai direttamente il significato di uno UUID, ma opera esclusivamente sull'oggetto di dominio che lo rappresenta.
Vantaggio: aggiungere un nuovo parametro configurabile in futuro richiede di intervenire solo nel livello di dominio e, se necessario, nella selezione del controllo grafico, senza toccare la logica generale di rendering della lista.
- **Codifica esplicita del valore scritto:** il valore selezionato dall'utente viene serializzato secondo il formato atteso dal firmware (intero a 16 bit, little-endian) immediatamente prima della scrittura sulla Characteristic corrispondente.
Vantaggio: rende esplicito, nel punto di scrittura, il contratto di formato condiviso con il firmware, evitando ambiguità sul livello di codifica dei dati scambiati.

Questa schermata è, in particolare, il punto in cui l'utente imposta il timeout di inattività dei sensori descritto in [Gestione dell'inattività dei sensori](#) lato firmware, chiudendo il ciclo tra la scelta configurata dall'utente sull'applicazione e il comportamento realmente adottato dal dispositivo.

6. Implementazione

In questo capitolo viene descritto il lavoro di implementazione svolto, mostrando come le scelte progettuali illustrate nel capitolo precedente si sono tradotte in codice effettivo.

6.1 Sistema di persistenza a superfile

Questa sezione traduce in codice l'architettura del superfile descritta in [Sistema di persistenza a superfile](#): la separazione tra formato su disco e stato di runtime, le operazioni di creazione, apertura con recovery, scrittura, sincronizzazione e chiusura, la finestra di azzeramento anticipata a supporto della recovery e la comunicazione dello stato di occupazione.

Il formato su disco descritto in progettazione elencava nove campi logici dell'header. Nella struttura effettivamente persistita, tuttavia, ne compaiono solo alcuni:

```
1 struct Header {
2     uint32_t magic;           // costante di validita' ("EGL1")
3     uint8_t  version;
4     uint32_t dataStartOffset;
5     uint32_t dataEndOffset;
6     uint32_t recordCount;
7     uint8_t  wrapped;
8 };
```

Codice 6.1: Header effettivamente persistito su disco

La scelta è giustificata: solo i campi che *variano nel tempo* e non sono altrimenti ricostruibili vengono scritti su microSD; tutto ciò che è invece derivabile dal firmware in esecuzione viene ricalcolato a ogni apertura, evitando di duplicare un'informazione che potrebbe disallinearsi dalla sua sorgente (le costanti del firmware). La Tabella [6.1](#) riassume la corrispondenza.

Campo logico	Persistito	Se non persistito, ricavato da
Versione del formato	Sì	-
Identificativo del sensore	No	nome del file (<code>buildPath</code>)
Dimensione del record	No	<code>getSensorParams(sensorId)</code>
Capacità totale	No	<code>computeTotalRecords(...)</code>
Cursore di fine dati	Sì	-
Cursore di inizio dati	Sì	-
Soglia di sincronizzazione	No	<code>HEADER_SYNC_BYTES_TARGET/recordSize</code>
Dim. finestra di azzeramento	No	<code>syncEveryRecords * 2</code>
Indicatori di stato	Parziale	<code>wrapped</code> persistito; <code>isFull</code> ricalcolato

Tabella 6.1: Corrispondenza tra i campi logici dell'header e la loro effettiva persistenza

Il motivo comune alla maggior parte dei campi non persistiti è il costo delle operazioni di scrittura: l'header non viene risincronizzato a ogni record, ma periodicamente ([Sincronizzazione dell'header](#)), quindi ogni campo incluso nell'area persistita viene riscritto a ogni sincronizzazione per l'intera durata della sessione. Un valore che non cambia mai, o che cambia solo alla creazione del file, comporterebbe un costo ricorrente per un'informazione di fatto statica. Nel dettaglio:

- **Identificativo del sensore:** già codificato nel nome del file; ripeterlo nell'header sarebbe ridondante.
- **Dimensione del record:** dipende esclusivamente dal tipo di sensore ed è fissata a livello di firmware, come anche l'*ODR_C* dei sensori, la finestra di retention e i pin logici usati dai vari moduli.
- **Capacità totale:** dipende da parametri di configurazione firmware, invarianti per l'intera vita del file; ricalcolarla costa poche operazioni aritmetiche, contro un campo che andrebbe scritto una sola volta e mai più aggiornato.
- **Soglia di sincronizzazione:** deriva dalla dimensione del record e da una costante di firmware; persisterla vincolerebbe il file al valore scelto alla creazione anche a fronte di un futuro aggiornamento firmware.
- **Dimensione della finestra di azzeramento:** deriva interamente dalla soglia di sincronizzazione, stessa motivazione del punto precedente.
- **Indicatori di stato:** rispetto ai due esempi previsti in progettazione per questo campo logico, ovvero l'abilitazione dell'overwrite e il raggiungimento della capacità massima, l'abilitazione dell'overwrite è in realtà una costante per sensore (Tabella [6.2](#)) anziché un campo dell'header. `wrapped` costituisce invece un ulteriore indicatore, non anticipato in progettazione ma della stessa natura concettuale, ed è persistito perché rappresenta un evento realmente accaduto sul supporto (un giro completo del buffer) che nessun ricalcolo potrebbe altrimenti ricostruire. `isFull` è invece sempre deducibile confrontando `recordCount` con la capacità totale, quindi persisterlo aggiungerebbe un'informazione ridondante da mantenere sincronizzata a ogni scrittura.

Questa equivalenza tra ricalcolo e riletture regge perché la recovery avviene sempre nello stesso firmware con cui il file era stato creato: un riavvio non cambia le costanti di configurazione.

Due campi compaiono nella struttura persistita senza comparire tra i nove campi logici elencati in progettazione.

Il primo è la **costante magica** (`magic`), scritta all’inizio dell’header e verificata a ogni apertura insieme alla versione: a differenza degli altri campi non descrive lo stato del file, ma ne attesta la validità prima di fidarsi del resto del contenuto, distinguendo un header corretto da uno corrotto o mai inizializzato.

Il secondo è `recordCount`: ridondante rispetto ai due soli cursori, ma necessario per distinguere senza ambiguità un buffer vuoto da uno pieno, casi in cui `dataStartOffset` e `dataEndOffset` coinciderebbero altrimenti.

Lo stato di runtime (`Instance`) mantiene invece il riferimento al file aperto, i parametri ricalcolati sopra elencati e i contatori di servizio (`recordsSinceSync`). Le operazioni concettuali di progettazione sono realizzate come funzioni libere che operano su un’istanza esplicita, senza stato nascosto:

```
1 bool create(SdFat& sdFat, Instance& inst, const char* path,
2             uint32_t recordSize, uint32_t totalRecords,
3             uint32_t headerAreaSize, bool allowOverwrite);
4
5 bool openAndRecover(SdFat& sdFat, Instance& inst, const char* path, /* ... */);
6
7 bool writeRecords(Instance& inst, const uint8_t* src, uint32_t count);
8 bool syncHeader(Instance& inst);
9 void close(Instance& inst);
```

Codice 6.2: Interfaccia del modulo superfile (`openAndRecover` condivide gli stessi parametri di `create`, omessi per brevità)

6.1.1 Creazione e calcolo della capacità

La capacità totale, descritta in [Capacità del file](#), è calcolata arrotondando per eccesso alla dimensione del *cluster* del volume. In questo modo, la dimensione del superfile viene allineata, per quanto possibile, alla granularità di allocazione della scheda microSD, evitando di lasciare porzioni di spazio inutilizzate all’interno dell’ultimo cluster allocato e riducendo al minimo la frammentazione del file system. L’obiettivo è quindi fare in modo che le operazioni di allocazione e gestione dello spazio siano quanto più possibile allineate ai cluster fisici della scheda:

```

1 uint64_t rawBytes = uint64_t(hz) * 3600ULL * retentionHours * recordSize;
2 uint64_t alignedBytes = ((rawBytes + clusterSizeBytes - 1) / clusterSizeBytes)
3 * clusterSizeBytes;
4 return uint32_t(alignedBytes / recordSize);

```

Codice 6.3: Calcolo della capacità del superfile

I parametri da cui questi valori derivano (`hz`, `retentionHours`, dimensione del record e pin logici) sono centralizzati in un unico file di configurazione firmware, `constants.h`, anziché essere sparsi tra i singoli moduli sensore: una modifica alla frequenza di campionamento o alla finestra di retention di un sensore richiede quindi di intervenire in un solo punto del codice. La Tabella 6.2 riporta i valori concreti attualmente configurati per ciascun sensore.

Sensore	ODR	RETENTION HOURS	ALLOW OVERWRITE	FLUSH INTERVAL_S
Barometro	1 Hz	10 h	sì	120 s
Magnetometro	1 Hz	10 h	sì	5 s
IMU	417 Hz	10 h	sì	8 s
GPS	10 Hz	20 h	no	48 s

Tabella 6.2: Parametri concreti per sensore definiti in `constants.h`

6.1.2 Scrittura e wraparound

Quando il blocco da scrivere attraversa il limite superiore dell'area dati, la scrittura viene spezzata in due parti, come descritto in progettazione:

```

1 // wraparound: split in scrittura pre-wrap e post-wrap
2 uint32_t firstPartRecords = recordsToWrapBoundary;
3 uint32_t secondPartRecords = writable - firstPartRecords;
4
5 file.write(src, firstPartRecords * recordSize); // fino a fine area dati
6 file.write(src + firstPartRecords * recordSize,
7           secondPartRecords * recordSize); // dall'inizio dell'area dati

```

Codice 6.4: Split della scrittura in caso di wraparound (gestione errori omessa per brevità)

Per il sensore GPS, in modalità non-overwrite, al raggiungimento della capacità massima le scritture successive vengono silenziosamente ignorate e l'header viene marcato pieno, propedeuticamente alla terminazione automatica della sessione già descritta in progettazione.

6.1.3 Sincronizzazione e finestra di azzeramento

Ogni sincronizzazione dell'header ricostituisce la finestra di azzeramento a partire dal nuovo cursore. La dimensione della finestra è pari al doppio della soglia di sincronizzazione:

poiché la finestra viene consumata progressivamente tra una sincronizzazione e la successiva, raddoppiarla garantisce che, anche immediatamente prima di un *sync*, resti sempre almeno un intero intervallo di sincronizzazione di record pre-azzerati davanti al cursore.

6.1.4 Recovery su più livelli

La recovery descritta in [Recovery dopo interruzioni impreviste](#) è realizzata su tre livelli distinti: a livello di dispositivo (`SD::recover()`) rimonta il file system, ricarica la configurazione e, se una sessione risulta attiva, delega a un secondo livello (`recoverAllForSession`) che itera tutti i sensori della sessione sotto un unico lock; per ciascuno di essi invoca infine `openAndRecover`, che a livello di singolo file esegue la riconciliazione:

```
1 for (uint32_t i = 0; i < inst.zeroWindowRecords; i++) {
2     uint64_t ts = 0;
3     if (!readRecordTimestamp(inst, inst.header.dataEndOffset, ts)) break;
4     if (ts == 0) break;           // primo record non ancora scritto: confine
    ↪ trovato
5     advanceAfterAppend(inst);
6 }
```

Codice 6.5: Riconciliazione del cursore in fase di recovery

6.1.5 Comunicazione dello stato di occupazione

Come descritto in [Comunicazione dello stato di occupazione](#), il calcolo è isolato in un componente separato dal modulo di persistenza, invocato dal consumer subito dopo ogni scrittura riuscita e a ogni controllo di presenza della scheda. Il valore condiviso con l'app raggruppa, per ciascun sensore, la percentuale di occupazione e un indicatore di sovrascrittura in corso, oltre a un indicatore complessivo di assenza della scheda; viene reso disponibile al sottosistema BLE come singolo valore sempre sovrascritto, coerentemente con la scelta di trattarlo come *snapshot* più recente piuttosto che come sequenza di eventi da accodare.

6.1.6 Problematiche riscontrate e soluzioni

Nella fase iniziale del tirocinio erano disponibili due prototipi: uno con i componenti saldati e uno su breadboard. Su quest'ultimo, il contatto poco affidabile del pin DET dello slot microSD produceva un errore particolarmente insidioso: la scheda veniva segnalata come inserita anche quando fisicamente assente.

```
1 // Prima: stato meccanico del pin DET
2 bool isCardInserted() {
3     return digitalRead(SD::PIN_DET);
4 }
```

Codice 6.6: Rilevamento della scheda basato sul pin DET

```
1 // Dopo: prova elettrica reale della scheda via SPI
2 bool isCardInserted() {
3     auto* card = sdFat.card();
4     if (!card) return false;
5     cid_t cid;
6     return card->readCID(&cid);
7 }
```

Codice 6.7: Rilevamento della scheda tramite lettura del CID

Questo tipo di falso positivo è più pericoloso del suo opposto: un falso negativo si traduce semplicemente in un'attesa e in un nuovo tentativo, mentre un falso positivo lascia proseguire il firmware oltre il controllo pensato per impedire l'accesso in assenza di scheda, facendo fallire in modo meno prevedibile le operazioni di apertura o scrittura successive. La lettura del *CID_G*, richiedendo una transazione *SPI_G* che riesce solo se una scheda è fisicamente presente e risponde correttamente, sposta il controllo dallo stato meccanico di un contatto alla prova elettrica reale della sua operatività, risultando affidabile su entrambi i prototipi indipendentemente dalla qualità del contatto fisico. Il costo aggiuntivo, ossia l'esecuzione sotto lock SPI anziché una lettura diretta di pin, è coerente con l'intervallo di polling già in uso nel consumer.

6.2 Stato globale del dispositivo

Questa sezione traduce in codice quanto descritto in [Stato globale del dispositivo](#): un punto di accesso unico allo stato del dispositivo, in cui nessun componente mantiene una propria copia locale delle informazioni condivise (*Single Source of Truth*), e in cui ogni indicatore è protetto da accessi concorrenti senza bisogno di un lock esplicito.

Lo stato principale della sessione è rappresentato da un unico valore enumerato, che può assumere uno dei cinque stati mutuamente esclusivi definiti in fase di progettazione:

```
1 enum class State : uint8_t {  
2   STAND_BY, LOGGING, SUSPENDED, DOWNLOADING, DELETING  
3 };  
4  
5 State getCurrentState();  
6 void setCurrentState(State s);
```

Codice 6.8: Stato principale di sessione

Gli altri indicatori (SD mancante, errore, calibrazione in corso, pairing, download e cancellazione) sono invece rappresentati da booleani indipendenti, ciascuno con una propria coppia di funzioni per la lettura e la modifica:

```
1 bool isSdMissing();  
2 void setSdMissing(bool value);
```

Codice 6.9: Interfaccia di un indicatore booleano indipendente

Ogni altro indicatore segue lo stesso schema, omissso per brevità.

La scelta di un unico valore enumerato, anziché di un booleano indipendente per ciascuno dei cinque stati, deriva dalla loro natura mutuamente esclusiva: il dispositivo non può trovarsi contemporaneamente, ad esempio, in `LOGGING` e `DOWNLOADING`. Un valore enumerato garantisce tale vincolo direttamente a livello di rappresentazione, poiché l'impostazione di un nuovo stato sostituisce quello precedente.

Gli indicatori booleani sono invece appropriati per condizioni ortogonali allo stato principale: una condizione di errore o l'assenza della scheda microSD, ad esempio, possono verificarsi indipendentemente dallo stato corrente.

6.2.1 Accesso senza lock

Ogni variabile di stato è dichiarata come `std::atomic`, cioè come un tipo che la libreria standard c++ garantisce come leggibile e scrivibile da più task contemporaneamente:

```
1 std::atomic<bool> sdMissing{false};
2
3 bool isSdMissing() { return sdMissing.load(); }
4 void setSdMissing(bool v) { sdMissing.store(v); }
```

Codice 6.10: Un indicatore come variabile atomica

Questo evita di dover proteggere ogni lettura o scrittura con un mutex, come sarebbe invece necessario se lo stato fosse raggruppato in un'unica struttura non atomica: ogni indicatore è una variabile a sé, indipendente dalle altre, quindi due componenti possono leggere o scrivere indicatori diversi nello stesso istante senza mai doversi attendere a vicenda.

6.2.2 Un caso concreto: due task sullo stesso indicatore

L'indicatore `sdMissing` viene scritto dal task consumer ogni volta che rileva un cambiamento nella presenza della scheda, e viene letto dal task che gestisce il LED, per decidere se mostrare la segnalazione di massima priorità prevista dalla Tabella 5.1. Si tratta di due task completamente indipendenti, che non comunicano tra loro in alcun altro modo:

```
1 // Task che gestisce la scrittura su SD
2 if (!cardInserted) {
3     GlobalState::setSdMissing(true);
4 }
```

Codice 6.11: Scrittura dell'indicatore da parte del task consumer

```
1 // Task che gestisce il LED, eseguito periodicamente
2 if (GlobalState::isSdMissing()) {
3     return {255, 0, 0, PatternType::BLINK, 4.0f};
4 }
```

Codice 6.12: Lettura dello stesso indicatore da parte del task LED

Il task LED non riceve alcuna notifica dal task consumer, né mantiene una propria copia di "SD mancante": si limita a interrogare, ogni volta che deve decidere cosa mostrare, la stessa variabile che il task consumer aggiorna.

6.3 Segnalazione di stato tramite LED

Questa sezione traduce in codice l'architettura a tre livelli descritta in [Segnalazione di stato tramite LED](#): livello hardware, livello decisionale e livello di rendering.

Il pattern visivo è rappresentato come dato immutabile, indipendente sia dalla logica di decisione sia dal rendering:

```

1 enum class PatternType : uint8_t { SOLID, BLINK, BREATHING };
2
3 struct Pattern {
4     uint8_t r, g, b;
5     PatternType type;
6     float frequencyHz; // usato solo per BLINK/BREATHING
7 };

```

Codice 6.13: Value Object del pattern visivo

I tre livelli sono realizzati come tre punti di ingresso distinti e non interdipendenti: `LED::showColor(r, g, b)` sul livello hardware, `LED::resolvePattern()` sul livello decisionale, e un task periodico che richiama entrambi sul livello di rendering.

6.3.1 Livello decisionale

La funzione `resolvePattern` implementa l'ordine di priorità della Tabella 5.1 come una sequenza di controlli a cascata, in cui il primo che risulta vero determina il pattern restituito:

```

1 Pattern resolvePattern() {
2     if (GlobalState::isSdMissing())
3         return {255, 0, 0, PatternType::BLINK, 4.0f}; // 1: SD mancante
4     if (GlobalState::isError())
5         return {255, 0, 0, PatternType::BLINK, 1.0f}; // 2: errore
6     if (GlobalState::isPairingBle())
7         return {0, 0, 255, PatternType::BLINK, 1.0f}; // 3: pairing BLE
8     if (GlobalState::isPairingWifi())
9         return {0, 0, 255, PatternType::BLINK, 4.0f}; // 4: pairing Wi-Fi
10    if (GlobalState::isRetrievingData())
11        return {255, 255, 0, PatternType::SOLID, 1.0f}; // 5: recupero dati
12    if (GlobalState::isCalibrating())
13        return {255, 255, 0, PatternType::BREATHING, 1.0f}; // 6: calibrazione
14    // ... download, cancellazione, sospensione, logging, standby
15 }

```

Codice 6.14: Risoluzione del pattern per priorità esplicita (troncato per brevità)

La struttura a cascata rende la Tabella 5.1 direttamente leggibile nel codice: l'ordine dei controlli è l'ordine di priorità, senza bisogno di una struttura dati o di un meccanismo di confronto separato.

6.3.2 Livello di rendering

Il task di rendering richiama `resolvePattern()` periodicamente e traduce il risultato in un'animazione effettiva. Un cambio di pattern fa ripartire da zero la fase dell'animazione, cosicché ogni transizione di stato, ad esempio l'inizio di un lampeggio, parta sempre dal proprio fronte iniziale invece che da un punto arbitrario del ciclo precedente:

```
1 if (changed) {  
2     lastPattern = p;  
3     phaseStartMs = millis();  
4 }  
5 uint32_t elapsed = millis() - phaseStartMs;
```

Codice 6.15: Aggancio della fase dell'animazione al cambio di pattern

Per BLINK la fase determina semplicemente se il tempo trascorso ricade nella prima o nella seconda metà del periodo; per BREATHING la stessa fase viene invece usata come argomento di una funzione sinusoidale, che modula la luminosità del colore in modo continuo anziché a scatti.

6.3.3 Livello hardware

Il livello hardware si riduce all'interfacciamento con la libreria del LED sulla scheda, con due sole funzioni esposte al resto del firmware:

```
1 bool setupModule(); // inizializza il LED  
2 void showColor(uint8_t r, uint8_t g, uint8_t b); // imposta un colore RGB
```

Codice 6.16: Interfaccia del livello hardware

Nessun dettaglio della libreria sottostante trapela oltre questa interfaccia: il livello decisionale e quello di rendering, descritti di seguito, non sanno né devono sapere quale libreria o quale LED fisico sia effettivamente in uso.

6.4 Gestione dell'inattività dei sensori

Questa sezione traduce in codice quanto descritto in [Gestione dell'inattività dei sensori](#): la sorgente autorevole commutabile tra GPS e IMU, il debounce basato su campioni consecutivi, e l'esclusione del GPS dalla sospensione.

Il modulo espone poche funzioni, ciascuna invocata da un punto preciso del firmware: il producer GPS riporta ogni campione ricevuto, il producer IMU riporta l'esito del proprio controllo di fallback, mentre il resto del firmware può solo interrogare quale sorgente sia autorevole in un dato istante:

```
1 void reportGpsSample(bool hasFix, float speedMps);
2 void reportImuFallbackSample(bool motionDetected);
3 bool isGpsAuthoritative();
4 uint32_t getTimeoutMs();
```

Codice 6.17: Interfaccia del modulo di gestione dell'inattività

6.4.1 Sorgente autorevole

La funzione di decisione si riduce al confronto tra l'istante dell'ultimo fix GPS e una soglia di freschezza: se il fix è recente, il GPS governa la decisione di sospensione; altrimenti il compito passa all'IMU.

```
1 bool isGpsAuthoritative() {
2     uint32_t lastFix = lastGpsFixMillis.load();
3     if (lastFix == 0) return false;
4     return (millis() - lastFix) <= GPS_FIX_FRESHNESS_MS;
5 }
```

Codice 6.18: Determinazione della sorgente autorevole

Questa singola funzione è il punto centrale a cui ogni altra parte del sistema si appoggia per sapere quale sorgente considerare valida in un dato momento, coerentemente con la scelta progettuale di centralizzare la selezione pur mantenendo i due percorsi di rilevamento indipendenti.

6.4.2 Debounce sul segnale GPS

Il rilevamento del movimento da GPS richiede un numero minimo di campioni consecutivi sopra soglia prima di essere accettato, azzerando il conteggio a ogni campione sotto soglia:

```
1 if (speedMps >= GPS_SPEED_THRESHOLD_MPS) {  
2     if (++consecutiveAboveThreshold >= GPS_MOTION_DEBOUNCE_SAMPLES) {  
3         resume();  
4     }  
5 } else {  
6     consecutiveAboveThreshold = 0;  
7 }
```

Codice 6.19: Debounce del segnale di movimento GPS (semplificato)

6.4.3 Soglie e parametri di temporizzazione

Il meccanismo si appoggia a quattro parametri, tutti tarati empiricamente:

- `GPS_SPEED_THRESHOLD_MPS` (velocità GPS, 0.8 m/s, circa 2.9 km/h): sopra questo valore un campione GPS è considerato indicativo di movimento;
- `IMU_ACCEL_VARIANCE_THRESHOLD` (varianza dell'accelerazione, circa 1.7g): soglia equivalente usata dal fallback IMU quando il GPS non è autorevole;
- `GPS_FIX_FRESHNESS_MS` (3000 ms): intervallo massimo dall'ultimo fix entro cui il GPS è considerato autorevole;
- `GPS_MOTION_DEBOUNCE_SAMPLES` (3): numero di campioni consecutivi sopra soglia richiesti per confermare il movimento.

È stata fatta una prima taratura da banco, seguita da un affinamento sul campo, in condizioni di utilizzo reali sul motoveicolo.

6.4.4 Attivazione mirata del fallback IMU

Il percorso di rilevamento basato su IMU non viene campionato in continuazione, ma solo mentre il dispositivo è già sospeso e il GPS non risulta autorevole: è il producer IMU stesso, nel proprio ciclo di attesa, a decidere quando interrogare l'accelerometro:

```
1 bool isImuFallbackPoll = (state == SUSPENDED &&  
    ↪ !Inactivity::isGpsAuthoritative());  
2 if (isImuFallbackPoll) {  
3     IMU::sampleFallbackMotion(); // calcola la varianza e chiama  
    ↪ reportImuFallbackSample  
4 }
```

Codice 6.20: Campionamento mirato del fallback IMU nel producer

Questo evita di consumare cicli sull'accelerometro nei momenti in cui la sorgente autorevole è comunque il GPS o il dispositivo è già attivo, senza per questo modificare la logica di selezione della sorgente, che resta interamente delegata a `isGpsAuthoritative()`.

6.4.5 Sospensione esclude il GPS

Sia `suspend()` sia `resume()` notificano solo i producer diversi dal GPS:

```
1 void notifyNonGpsProducers(uint32_t bit) {  
2     for (auto& ctx : Sensor::getProducerContexts()) {  
3         if (ctx.spec->sensorId == Sensor::SensorId::GPS) continue;  
4         xTaskNotify(handles[i], bit, eSetBits);  
5     }  
6 }
```

Codice 6.21: Propagazione della sospensione a tutti i producer tranne il GPS

Il producer GPS, dal canto suo, esce immediatamente dalla propria funzione di attesa senza mai fermarsi, restando l'unico sensore sempre acquisito indipendentemente dallo stato di sospensione.

6.4.6 Sospensione disabilitata

Quando il timeout è configurato a zero, ogni campione GPS aggiorna immediatamente l'istante di ultimo movimento e forza il dispositivo a LOGGING, disabilitando di fatto la sospensione:

```
1 if (timeoutMs == 0) {  
2     lastMotionMillis = now;  
3     resume();  
4     return;  
5 }
```

Codice 6.22: Disattivazione della sospensione tramite timeout a zero

6.5 Protocollo di avvio e arresto della sessione

Questa sezione traduce in codice quanto descritto in [Protocollo di avvio e arresto della sessione](#): i comandi START, STOP e RESUME come valori di un insieme chiuso, e le sequenze di avvio e arresto orchestrate sopra i moduli di superfile, stato globale e calibrazione dell'altitudine.

I comandi provenienti dall'app sono un enum a tre valori, instradati da un unico punto di gestione:

```
1 enum class Command : uint8_t { START, STOP, RESUME };  
2  
3 void handleCommand(Command cmd);
```

Codice 6.23: Interfaccia del modulo sessione

6.5.1 Un prerequisito: producer in grado di dormire

Come descritto in [Situazione di partenza](#), il firmware ricevuto all'inizio del tirocinio avviava l'acquisizione dati automaticamente all'accensione, senza attendere alcun comando. Introdurre un comando esplicito di avvio e arresto ha richiesto quindi, a monte del protocollo vero e proprio, un intervento sui task producer stessi: ciascuno di essi doveva poter restare inattivo finché nessuna sessione è in corso, e riattivarsi solo alla ricezione del comando di avvio, anziché acquisire dati in modo incondizionato dal boot del dispositivo. Ogni producer, a ogni ciclo, verifica quindi lo stato di sessione prima di procedere:

```
1 while (GlobalState::getCurrentState() != GlobalState::State::LOGGING) {  
2     xTaskNotifyWait(0x0, ULONG_MAX, &notif, portMAX_DELAY); // dorme finche' non  
    ↪ arriva un segnale  
3 }
```

Codice 6.24: Attesa del producer finché non è in corso una sessione (semplificato)

Senza questa modifica preliminare, il concetto stesso di "avvio" e "arresto" di una sessione sarebbe stato privo di effetto: i producer avrebbero continuato a produrre dati indipendentemente da qualunque comando ricevuto. Il protocollo descritto nel resto di questa sezione si appoggia interamente su questa capacità dei producer di restare in attesa.

6.5.2 Comandi come valori instradati centralmente

`handleCommand` riceve il comando già decodificato dal canale BLE e lo instrada con un semplice switch sul valore ricevuto, senza che la logica di sessione debba conoscere nulla del trasporto con cui il comando è arrivato: la stessa funzione potrebbe essere invocata da un trigger diverso da BLE senza alcuna modifica.

6.5.3 Avvio sessione

Alla ricezione di START, se il dispositivo non è occupato in download o cancellazione e non è già in sessione, vengono eseguiti in sequenza: un tentativo di recovery della scheda; la generazione del nome della cartella di sessione, combinando identificativo del dispositivo, timestamp di inizio e il suffisso `_ACTIVE`; la creazione di tutti i superfile tramite il registro descritto in [Sistema di persistenza a superfile](#); l'esecuzione sincrona della calibrazione dell'altitudine, il cui esito viene comunque scritto nei metadati di sessione anche in caso di fallimento; infine la transizione a LOGGING e la riattivazione forzata dei producer:

```
1 snprintf(folderName, sizeof(folderName), "%s_%s%s",
2         chipIdStr, sessionStartPrefix, "_ACTIVE"); //1
3 SD::SuperFile::createAllForSession(SD::sdFat, folderName); //2
4 auto cal =
5     ↳ Session::AltitudeCalibration::run(SessionBle::ALTITUDE_CAL_TIMEOUT_MS); //3
6 GlobalState::setCurrentState(GlobalState::State::LOGGING); //4
7 Inactivity::forceWakeProducers(); //5
```

Codice 6.25: Sequenza di avvio sessione (semplificata)

Ogni fase è protetta da un accesso esclusivo al canale SPI condiviso con timeout configurabile: il fallimento di una qualsiasi fase imposta lo stato di errore e interrompe l'avvio, senza proseguire alle fasi successive.

6.5.4 Arresto sessione

Alla ricezione di STOP (o della richiesta di arresto automatico per esaurimento spazio GPS), la sequenza di chiusura segue il seguente ordine: se il dispositivo era sospeso, i producer vengono prima riattivati forzatamente; viene richiesto lo svuotamento sincrono dei buffer verso il meccanismo di scrittura; il firmware attende che la coda di scrittura sia completamente smaltita prima di chiudere i superfile, per non perdere record ancora in transito; infine la cartella di sessione viene rinominata sostituendo il suffisso `_ACTIVE` con un riferimento temporale di fine sessione, e lo stato torna a `STAND_BY`.

```
1 Sensor::requestForceFlushAndWait(FORCE_FLUSH_TIMEOUT_TICKS); //1
2 waitForQueueDrain(); //2
3 SD::SuperFile::closeAll(); //3
4 SD::sdFat.rename(oldFolder, newFolder); //4
5 GlobalState::setCurrentState(GlobalState::State::STAND_BY); //5
```

Codice 6.26: Sequenza di arresto sessione (semplificata)

6.5.5 Notifica periodica dello stato di sessione

Un timer software FreeRTOS (`statusTimer`), creato una sola volta all'inizializzazione del modulo, viene avviato alla transizione a LOGGING e fermato al ritorno a `STAND_BY`. A ogni scadenza richiama `notifyStatus()`, che compone una struttura compatta (stato

della sessione, secondi trascorsi dall'inizio sessione e numero di campioni) e la pubblica come notifica BLE dedicata.

6.5.6 Comando RESUME: risincronizzazione dopo la riconnessione BLE

Una sessione prosegue sul dispositivo indipendentemente dallo stato della connessione BLE: se l'app si disconnette e si riconnette mentre una sessione è già in corso, non riceve alcuna informazione di stato fino al successivo scadere del timer periodico appena descritto. Alla ricezione di RESUME il firmware richiama direttamente `notifyStatus()`, ottenendo una risincronizzazione immediata senza attendere il ciclo del timer e senza alcuna transizione dell'enum di sessione, che resta sotto il controllo esclusivo di START e STOP.

6.5.7 Un caso particolare: l'arresto per spazio esaurito

Come descritto in progettazione, l'arresto automatico per esaurimento dello spazio GPS esegue la stessa sequenza di arresto appena descritta, con un'unica differenza a monte: prima di avviarla, il firmware imposta un indicatore dedicato, che permette all'app di distinguere, leggendo lo stato di sessione, un arresto voluto dall'utente da uno automatico.

6.6 Calibrazione dell'altitudine

Questa sezione traduce in codice l'algoritmo di calibrazione descritto in [Calibrazione dell'altitudine](#): raccolta campioni con filtro dei duplicati, calcolo di mediana e deviazione standard, accettazione o scarto del gruppo, e timeout massimo complessivo.

Il risultato della calibrazione è un dato immutabile che distingue esplicitamente un esito riuscito da un fallimento per timeout:

```
1 struct Result {  
2     bool    calibrated;  
3     float   referencePressureHpa; // valido solo se calibrated == true  
4     bool    timedOut;  
5 };  
6  
7 Result run(uint32_t timeoutMs);
```

Codice 6.27: Interfaccia del modulo di calibrazione dell'altitudine

Il valore di riferimento accettato viene poi reso disponibile globalmente in lettura tramite `setReferencePressure/getReferencePressure`, protetto da una sezione critica anziché da un mutex, essendo consultato anche dal producer del barometro a ogni campione.

6.6.1 Raccolta campioni e filtro duplicati

L'algoritmo interroga ripetutamente l'ultimo campione disponibile dal producer del barometro, scartando quelli il cui timestamp coincide con l'ultimo già considerato:

```
1 Barometer::Sample s = Barometer::getLastSample();  
2 bool goodSample = s.timestamp != 0 && s.timestamp != lastTimestamp;
```

Codice 6.28: Filtro dei campioni duplicati tramite confronto del timestamp

Questo evita che una lettura più lenta del ciclo di polling della calibrazione porti a contare più volte lo stesso campione fisico, falsando sia la mediana sia la deviazione standard.

6.6.2 Mediana, deviazione standard e accettazione del gruppo

Raggiunto il numero di campioni richiesto, il gruppo viene ordinato per estrarne la mediana, e la deviazione standard viene calcolata rispetto a quest'ultima anziché rispetto alla media, coerentemente con la scelta della mediana come stima robusta descritta in progettazione:


```
1 std::sort(sorted, sorted + N);
2 float median = sorted[N / 2];
3 for (auto v : samples) variance += (v - median) * (v - median);
4 float stddev = sqrtf(variance / N);
5
6 if (stddev <= MAX_STDDEV_HPA) {
7     return Result{ true, median, false };
8 }
```

Codice 6.29: Calcolo di mediana e deviazione standard, e criterio di accettazione

Se la deviazione standard supera la soglia prevista, l'intero insieme di campioni viene scartato e la raccolta viene riavviata da zero, in accordo con quanto definito in progettazione. Qualora, invece, il timeout complessivo scada senza che sia stato possibile ottenere un insieme di campioni valido, la funzione restituisce un risultato con `timedOut = true` e la sessione prosegue comunque, seppur priva di un riferimento di altitudine valido. Tale comportamento è coerente con i requisiti della calibrazione: essa è necessaria per ottenere un'altitudine relativa priva dell'errore sistematico dovuto alla quota iniziale, ma non costituisce una condizione vincolante per l'avvio o la prosecuzione della sessione.

6.6.3 Formula di conversione e scelta del dato da conservare

L'altitudine relativa a un dato istante è calcolata dalla formula barometrica standard:

$$h = 44330 \left(1 - \left(\frac{p}{p_0} \right)^{\frac{1}{5.255}} \right) \quad (6.1)$$

dove h è l'altitudine relativa in metri, p la pressione istantanea misurata e p_0 la pressione di riferimento acquisita in fase di calibrazione. La costante 44330 rappresenta l'altezza approssimativa, in metri, dell'atmosfera terrestre secondo il modello standard, mentre l'esponente $1/5.255$ deriva dal tasso di variazione della pressione con la quota previsto dallo stesso modello.

Il valore effettivamente conservato al termine della calibrazione, tuttavia, non è un'altitudine già calcolata, ma la sola pressione di riferimento in *hPa_G*. La scelta è motivata dalla formula stessa: per restituire un'altitudine priva di errore sistematico, il calcolo richiede sempre *due* pressioni, quella istantanea e quella di riferimento, non un singolo valore di quota assoluta.

6.7 Calibrazione dell'IMU

Questa sezione traduce in codice quanto descritto in [Calibrazione dell'IMU](#): la macchina a stati esplicita, il Command Pattern per i comandi dall'app, il fan-out non invasivo sul flusso dati IMU, il retry isolato per singolo step, e l'algoritmo dei tre step di misura.

Gli stati e i comandi sono rappresentati come due enum distinti, coerenti con il Command Pattern e lo State Pattern descritti in progettazione:

```
1 enum class CalibCommand : uint8_t { START, CONFIRM, ABORT };
2
3 enum class CalibState : uint8_t {
4     IDLE,
5     STEP1_READY, STEP1_RUNNING, STEP1_RETRY,
6     STEP2_READY, STEP2_RUNNING, STEP2_RETRY,
7     STEP3_READY, STEP3_RUNNING, STEP3_RETRY,
8     SUCCESS, NO_DATA, ERR
9 };
```

Codice 6.30: Comandi e stati della calibrazione IMU

Il campione grezzo duplicato dal producer verso il canale di calibrazione è anch'esso una struttura minima, priva di qualunque elaborazione:

```
1 struct CalibSample {
2     int32_t accel[3];
3     int32_t gyro[3];
4 };
```

Codice 6.31: Campione grezzo instradato al task di calibrazione

6.7.1 Fan-out non invasivo sul flusso dati

Il producer dell'IMU continua a funzionare come se la calibrazione non esistesse: l'unica aggiunta è, quando la calibrazione è attiva, la duplicazione di ogni campione verso una coda dedicata, senza alcuna modifica al percorso di acquisizione ed elaborazione già in uso durante il logging:

```
1 if (IMU::isCalibrationActive()) {
2     IMU::CalibSample calibSample;
3     calibSample.accel[0] = currentSample.rawAccelX;
4     // ... e le altre componenti
5     xQueueSend(IMU::calibQueue(), &calibSample, 0);
6 }
```

Codice 6.32: Duplicazione del campione verso il canale di calibrazione

6.7.2 Macchina a stati e instradamento dei comandi

Un unico task consuma i comandi dalla coda e li interpreta in funzione dello stato corrente, secondo un semplice switch a due livelli (stato, poi comando), realizzando così sia lo State Pattern sia il Command Pattern descritti in progettazione: l'esecuzione effettiva di uno step è sempre delegata a una funzione comune, che riceve come parametri lo stato di esecuzione, lo stato di retry, lo stato successivo e la funzione da eseguire, disaccoppiando la sequenza delle transizioni dal contenuto di ciascuno step:

```

1 void runMeasuredStep(CalibState runningState, CalibState retryState,
2                     CalibState nextReadyState,
3                     CalibStepResult (*stepFn)(), bool isLastStep);

```

Codice 6.33: Firma della funzione comune di esecuzione di uno step

Un solo indice (`activeStepIndex`) tiene traccia dello step attualmente in corso: è ciò che permette, in caso di comando CONFIRM ricevuto mentre lo stato è NO_DATA, di sapere quale dei tre step ripetere, poiché quello stato è raggiungibile da ciascuno dei tre step del percorso di calibrazione.

6.7.3 Soglie di accettazione degli step

Ciascuno dei controlli descritti nei paragrafi seguenti confronta una grandezza misurata con una soglia fissa, riportata in Tabella 6.3. I valori sono stati tarati empiricamente, con una combinazione di prove da banco e prove sul campo, montando il dispositivo sul motoveicolo nelle condizioni reali di utilizzo.

Soglia	Valore	Utilizzo
STEP1_GRAVITY_MIN/MAX_MG	700-1300 mG	Intervallo accettabile per il modulo di gravità misurato nello Step 1
STEP1_STILL_VARIANCE _THRESHOLD_MG2	17000 mG ²	Varianza massima ammessa per considerare il dispositivo fermo nello Step 1
STEP2_TILT_MIN_MG	120 mG	Componente laterale minima per considerare valida l'inclinazione dello Step 2
STEP3_FWD_MIN_NORM	0.1	Modulo minimo dell'asse longitudinale calcolato, sotto cui la geometria è giudicata degenera
EARLY_EXIT_STD_DEV_MG	15 mG	Soglia di stabilità del segnale per l'uscita anticipata da uno step

Tabella 6.3: Soglie di accettazione della calibrazione IMU

6.7.4 Uscita anticipata basata sulla stabilità del segnale

Ogni step condivide la stessa routine di raccolta campioni, che integra un periodo di assestamento iniziale (i campioni ricevuti in questa fase vengono scartati) e un meccanismo di uscita anticipata: superato un numero minimo di campioni, la deviazione standard del

modulo dell'accelerazione viene calcolata su una finestra scorrevole delle ultime letture, e se risulta sotto soglia lo step termina subito, senza attendere il timeout completo:

```
1 if (count >= EARLY_EXIT_MIN_SAMPLES && windowCount == EARLY_EXIT_CHECK_WINDOW) {  
2     // media e deviazione standard sulla finestra di ultimi campioni  
3     if (windowStdDev < EARLY_EXIT_STD_DEV_MG) break; // segnale stabile: esce  
4     ↪ prima  
5 }
```

Codice 6.34: Uscita anticipata quando il segnale è già stabile (semplificato)

6.7.5 Step 1 - Posizione dritta

Con il dispositivo fermo, la media dei campioni raccolti approssima il vettore di gravità così come misurato dal sensore. Il modulo di questo vettore deve cadere in un intervallo plausibile attorno a 1 g, e la sua varianza deve restare sotto una soglia di stabilità: al di fuori di questi limiti lo step viene rifiutato, perché il dispositivo non era sufficientemente fermo o orientato in modo anomalo. Se accettato, il vettore normalizzato diventa l'asse verticale del sistema di riferimento, mentre lo scostamento del vettore misurato rispetto a quello atteso (1 g lungo la verticale) diventa il bias dell'accelerometro da sottrarre nelle acquisizioni successive; la media dei campioni del giroscopio, che a dispositivo fermo dovrebbe essere nulla, ne costituisce analogamente il bias.

6.7.6 Step 2 - Inclinazione a destra

Il dispositivo viene inclinato lateralmente. Lo scostamento tra il vettore medio misurato in questo step e quello registrato nello Step 1 viene proiettato sul piano perpendicolare all'asse verticale già determinato, isolando così la sola componente dello scostamento dovuta all'inclinazione laterale, indipendentemente da eventuali piccole variazioni residue lungo la verticale. Se il modulo di questa componente risulta troppo piccolo, l'inclinazione è giudicata insufficiente e lo step viene rifiutato; altrimenti, il vettore normalizzato diventa l'asse laterale.

6.7.7 Step 3 - Asse longitudinale e salvataggio

Il terzo asse non richiede una nuova misura: è calcolato geometricamente come prodotto vettoriale tra l'asse laterale e quello verticale già determinati, il che garantisce per costruzione un sistema di riferimento ortogonale. Un breve intervallo di raccolta campioni resta comunque necessario per verificare che il flusso dati dal sensore sia ancora presente. Se il modulo del vettore risultante è troppo piccolo, significa che i due assi precedenti erano quasi paralleli tra loro, cioè che l'inclinazione dello Step 2 non era sufficientemente distinta dalla posizione dello Step 1: una geometria degenera, che comporta il rifiuto dello step. In caso di successo, i tre assi compongono la matrice di rotazione salvata in configurazione insieme ai bias calcolati nello Step 1.

6.7.8 Gestione degli esiti negativi

I tre step condividono lo stesso schema di esito: nessun campione ricevuto entro il timeout porta allo stato `NO_DATA` (probabile problema hardware, non di postura), un numero di campioni insufficiente o una geometria/misura non valida porta allo stato di retry specifico dello step corrente, mentre un comando di interruzione esplicita riporta immediatamente lo stato a `IDLE` indipendentemente dallo step in corso.

6.7.9 Problematiche riscontrate e soluzioni

La soglia di varianza dello Step 1 (`STEP1_STILL_VARIANCE_THRESHOLD_MG2`) era stata inizialmente tarata mediante prove da banco. Il primo test sul campo ha evidenziato che tale taratura risultava ancora troppo restrittiva, poiché non teneva conto delle vibrazioni trasmesse dal motoveicolo durante il funzionamento del motore. La soglia è stata quindi ritarata sulla base delle condizioni reali di utilizzo, considerando anche i tremolii rilevati con il mezzo fermo e il motore acceso.

6.8 Architettura dello sniffer CAN

Questa sezione traduce in codice l'architettura esagonale descritta in [Architettura dello sniffer CAN](#): le tre porte del dominio, il caso d'uso che le orchestra senza conoscerne gli adattatori concreti, e le implementazioni realizzate per il prototipo (bus CAN in ascolto passivo, profilo Yamaha MT-07, trasporto via advertising BLE).

Le tre porte sono interfacce astratte pure, ciascuna con una responsabilità unica:

```
1 class ICanBus {
2 public:
3     virtual bool init() = 0;
4     virtual bool receive(CanFrame& out, uint32_t timeoutMs) = 0;
5 };
6
7 class IVehicleProfile {
8 public:
9     virtual uint8_t profileId() const = 0;
10    virtual bool handleFrame(const CanFrame& frame) = 0;
11    virtual TelemetrySnapshot currentSnapshot() const = 0;
12 };
13
14 class ITelemetryBroadcaster {
15 public:
16     virtual bool init() = 0;
17     virtual void publish(const TelemetrySnapshot& snapshot) = 0;
18 };
```

Codice 6.35: Le tre porte del dominio

Il dato scambiato tra bus e profilo (`CanFrame`) e quello scambiato tra profilo e broadcaster (`TelemetrySnapshot`) sono entrambi strutture dati semplici, prive di logica: il primo rispecchia un frame CAN grezzo, il secondo lo stato di telemetria già decodificato e leggibile.

6.8.1 Il caso d'uso: orchestrazione senza conoscenza degli adattatori

`SniffingService` riceve le tre porte nel costruttore come riferimenti alle interfacce astratte, non alle classi concrete che le implementano:

```
1 class SniffingService {
2 public:
3     SniffingService(ICanBus& bus, IVehicleProfile& profile,
4                     ITelemetryBroadcaster& broadcaster)
5         : bus_(bus), profile_(profile), broadcaster_(broadcaster) {}
6 private:
7     ICanBus& bus_;
8     IVehicleProfile& profile_;
9     ITelemetryBroadcaster& broadcaster_;
10 };
```

Codice 6.36: Costruzione del caso d'uso tramite le sole interfacce di dominio

In pratica, questa classe non sa mai quale bus CAN, quale profilo veicolo o quale sistema di trasmissione stia realmente usando: conosce solo che ciascuno di essi sa fare le operazioni previste dalla propria interfaccia (ricevere un frame, interpretarlo, pubblicare un risultato). Decidere quali classi concrete usare avviene altrove, nel punto in cui il firmware crea gli oggetti: `SniffingService` non deve essere toccato nemmeno se si cambia, ad esempio, il modo in cui i dati vengono trasmessi.

Il ciclo che sfrutta questa astrazione è invocato una volta per ogni iterazione del `loop()` del firmware:

```
1 void loop() {
2     sniffingService->tick(/*receiveTimeoutMs=*/10);
3 }
```

Codice 6.37: Punto di chiamata del ciclo di orchestrazione

Il corpo di `tick()` è breve:

```
1 void tick(uint32_t receiveTimeoutMs = 0) {
2     CanFrame frame;
3     if (!bus_.receive(frame, receiveTimeoutMs)) return;
4
5     if (profile_.handleFrame(frame)) {
6         broadcaster_.publish(profile_.currentSnapshot());
7     }
8 }
```

Codice 6.38: Ciclo di orchestrazione del caso d'uso

Un frame CAN viene ricevuto, passato al profilo veicolo perché lo interpreti, e solo se il profilo riconosce quell'identificativo (`handleFrame()` restituisce vero) il risultato aggiornato viene pubblicato. I frame che non appartengono a nessun segnale noto per quel veicolo vengono quindi ignorati, senza generare alcuna pubblicazione.

6.8.2 Adattatore del bus CAN: ascolto passivo

L'adattatore concreto della porta `ICanBus` configura il controller TWAI nativo dell'ESP-WROOM-32 in modalità di solo ascolto, realizzando direttamente la mitigazione discussa in Tabella 5.3:

```
1 twai_general_config_t g_config =  
2   TWAI_GENERAL_CONFIG_DEFAULT(txGpio_, rxGpio_, TWAI_MODE_LISTEN_ONLY);
```

Codice 6.39: Configurazione del controller TWAI in modalità listen-only

In questa modalità il controller non trasmette mai alcun ACK né frame di errore sul bus, indipendentemente da qualunque comportamento abbia software a monte.

6.8.3 Selezione del profilo veicolo

La Factory Pattern descritta in progettazione è realizzata come una funzione che mappa un identificativo di modello alla relativa implementazione concreta:

```
1 std::unique_ptr<IVehicleProfile> VehicleProfileFactory::create(VehicleModel  
  ↳ model) {  
2     switch (model) {  
3         case VehicleModel::YamahaMT07:  
4             return std::make_unique<YamahaMT07Profile>();  
5         default:  
6             return nullptr;  
7     }  
8 }
```

Codice 6.40: Selezione del profilo veicolo tramite Factory Pattern

Aggiungere il supporto a un nuovo modello richiederebbe solo un nuovo adattatore concreto e la relativa voce in questo switch, senza toccare `SniffingService` né le altre porte.

6.8.4 Decodifica specifica del veicolo

L'unico adattatore concreto della porta `IVehicleProfile` attualmente realizzato instrada ciascun frame in funzione del relativo identificativo. La mappatura degli identificativi è stata ricostruita mediante uno sniffing preliminare del bus e successivamente confrontata con le informazioni reperibili su forum e documentazione specifici del modello, poiché il costruttore non rende pubblica una mappatura completa dei frame:


```
1 switch (frame.id) {  
2     case GEAR_POSITION_ID: snapshot_.gear = decodeGear(frame.data[0]); break;  
3     case TPS_ID: snapshot_.tpsPercent = decodeTps(frame.data[0]); break;  
4     case TEMP_ID: /* motore e aria, un byte ciascuno */ break;  
5     case RPM_SPEED_ID: /* RPM e velocità */ break;  
6     default: return false; // ID non appartenente a questo profilo  
7 }
```

Codice 6.41: Instradamento dei frame per identificativo nel profilo Yamaha MT-07 (semplificato)

Ogni funzione di decodifica applica alla porzione di payload rilevante la formula di conversione specifica del segnale: dettagli intrinsecamente legati a questo singolo modello di veicolo, che restano isolati dentro l'adattatore.

6.8.5 Trasporto dati: advertising BLE broadcast-only

L'adattatore della porta `ITelemetryBroadcaster` configura l'advertising BLE come non connettabile, coerentemente con il modello *publish-only* descritto in progettazione:

```
1 advertising->setConnectableMode(BLE_GAP_CONN_MODE_NON);  
2 advertising->enableScanResponse(false);
```

Codice 6.42: Configurazione dell'advertising come non connettabile

Non ogni aggiornamento di telemetria genera un pacchetto: le pubblicazioni sono limitate a un intervallo minimo configurabile, evitando di rigenerare il payload di advertising più spesso di quanto un eventuale ricevitore possa comunque scansionarlo. Il pacchetto trasmesso è una struttura compatta a dimensione fissa, verificata a tempo di compilazione contro il budget disponibile nel payload di advertising:

```
1 static_assert(sizeof(CanTelemetryWirePacket) == 10,  
2     "The packet must remain within the BLE ADV payload budget.");
```

Codice 6.43: Verifica statica della dimensione del pacchetto trasmesso

6.9 Visualizzazione 3D del veicolo

Questa sezione traduce in codice il Game Loop Pattern descritto in [Visualizzazione 3D del veicolo](#): l'accesso diretto allo store di telemetria, disaccoppiato dalla cadenza delle notifiche BLE, e l'interpolazione dei valori frame per frame.

I tre valori di assetto letti a ogni frame sono raccolti in un'unica struttura, mantenuta in un riferimento mutabile anziché in stato React:

```
1 type TargetState = { roll: number; pitch: number; longAccel: number }
2
3 const targetRef = useRef<TargetState>({ roll: 0, pitch: 0, longAccel: 0 })
```

Codice 6.44: Stato di assetto mantenuto fuori dal ciclo di rendering React

6.9.1 Aggiornamento del riferimento, non dello stato React

Il componente si sottoscrive direttamente allo store condiviso, scrivendo i valori ricevuti nel riferimento mutabile invece che tramite `setState`:

```
1 useEffect(() => {
2   return useLiveMetricsStore.subscribe(state => {
3     targetRef.current.roll = state.motionState.roll
4     targetRef.current.pitch = state.motionState.pitch
5     targetRef.current.longAccel = state.motionState.longAccel
6   })
7 }, [])
```

Codice 6.45: Sottoscrizione diretta allo store, senza passare da `setState` React

Questo è il punto centrale del disaccoppiamento: un aggiornamento dello store non causa mai un nuovo render del componente React che ospita la scena 3D, ma si limita ad aggiornare un valore che il ciclo di rendering 3D leggerà al prossimo frame.

6.9.2 Lettura e interpolazione nel ciclo di rendering 3D

Il valore letto dallo store non viene applicato direttamente al modello 3D, ma avvicinato gradualmente ad esso, un piccolo passo per volta, a ogni fotogramma disegnato dal motore 3D:

```
1 useFrame(() => {
2   current.current.roll = THREE.MathUtils.lerp(
3     current.current.roll, targetRoll, SMOOTHING_FACTOR)
4   groupRef.current.rotation.z = current.current.roll
5 })
```

Codice 6.46: Interpolazione del valore letto a ogni frame (semplificato)

Questo serve a colmare una differenza di frequenza: il motore 3D disegna molti fotogrammi al secondo, a intervalli regolari, mentre le notifiche BLE con i nuovi dati di telemetria arrivano più di rado e a intervalli irregolari. Se il modello 3D venisse ruotato di scatto ogni volta che arriva una nuova notifica, il movimento risulterebbe a scatti, con un salto visibile ogni volta che il valore cambia. Avvicinando invece il valore mostrato a quello target un poco alla volta, ad ogni fotogramma, anche tra due notifiche il modello continua a muoversi in modo fluido verso l'ultimo valore ricevuto, invece di restare fermo in attesa del prossimo salto.

6.9.3 Elementi testuali a frequenza ridotta

Le card numeriche utilizzano hook dedicati con aggiornamento limitato (`useThrottledSpeed`, `useThrottledHeight`, `useThrottledMotion`), riducendo i render React rispetto alla sottoscrizione raw usata per le animazioni. La stessa fonte dati viene quindi consumata con frequenze diverse in base alle esigenze del componente, evitando render superflui per valori la cui variazione non è percepibile dall'utente.

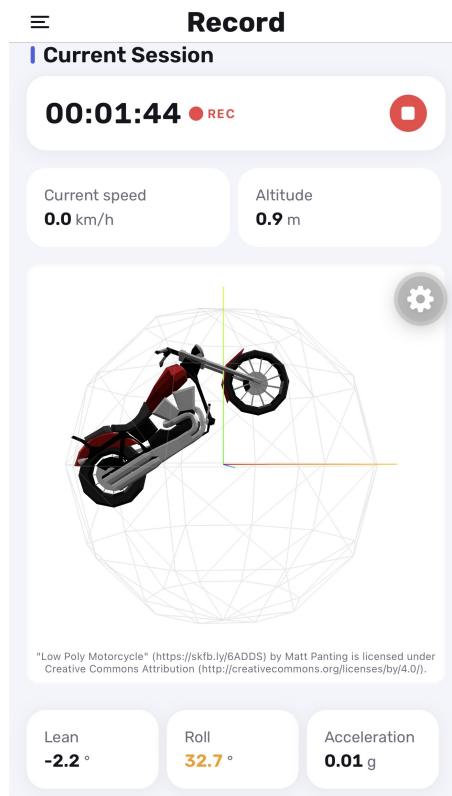


Figura 6.1: Visualizzazione 3D della moto, con assetto aggiornato in tempo reale

6.10 Segnalazione di microSD mancante durante la sessione

Questa sezione traduce in codice quanto descritto in [Segnalazione di microSD mancante durante la sessione](#): il componente presentazionale riutilizzato nei due contesti, l'azione differenziata per contesto e il comportamento non richiudibile.

Il componente è parametrico rispetto a etichetta, colore e stato di caricamento dell'azione proposta, senza contenere alcuna logica su quando essere mostrato:

```
1 function SdMissingOverlay({
2   onAction, actionLabel, actionLoading, actionColorStyle, loadingText
3 }): { ... } { ... }
```

Codice 6.47: Firma del componente presentazionale di segnalazione SD mancante

6.10.1 Ripartizione dell'occupazione per sensore

A differenza dei due overlay, che dipendono dal solo indicatore booleano di assenza scheda, questo componente legge dallo stesso oggetto di dominio la percentuale di occupazione e l'indicatore di sovrascrittura di ciascun sensore, mostrandoli in una griglia:

```
1 {order.map(idx => {
2   const percent = storageStatus.percentUsed[idx]
3   const rewriting = storageStatus.rewriting[idx]
4   return (
5     <View key={idx}>
6       <Text>{SENSOR_LABELS[idx]}</Text>
7       <Text>{percent.toFixed(1)}%</Text>
8       {rewriting && <Text>Rewriting</Text>}
9     </View>
10  )
11 }}
```

Codice 6.48: Ripartizione per sensore della percentuale di occupazione

L'ordine dei sensori nella griglia è fissato esplicitamente (`order`), invece di seguire l'ordine in cui i valori arrivano dall'oggetto `StorageStatus`: una scelta che rende la disposizione stabile e prevedibile a schermo indipendentemente da come il firmware serializza i dati sulla `Characteristic`. L'etichetta `Rewriting`, mostrata solo per i sensori configurati in modalità a sovrascrittura che hanno già compiuto un giro completo del proprio buffer circolare, avvisa l'utente che, per quel sensore, i dati più vecchi della sessione corrente sono già stati sostituiti da dati più recenti.

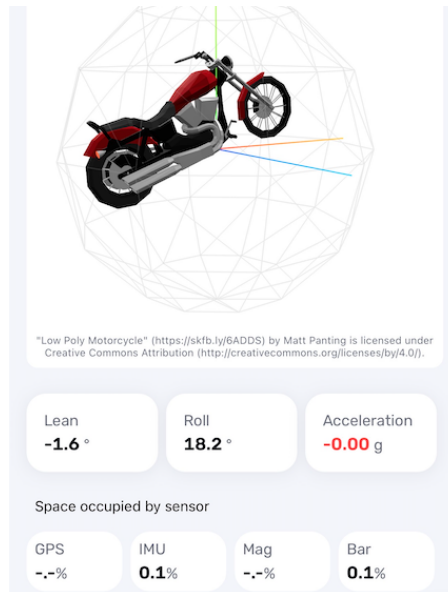


Figura 6.2: Ripartizione dell'occupazione per sensore, con indicatore di sovrascrittura

6.10.2 Stesso componente, azione differente per contesto

Il componente compare in due punti della schermata Record, ciascuno con i propri parametri: prima dell'avvio, offre un tentativo di ripetizione; durante una sessione già in corso, offre solo l'arresto forzato:

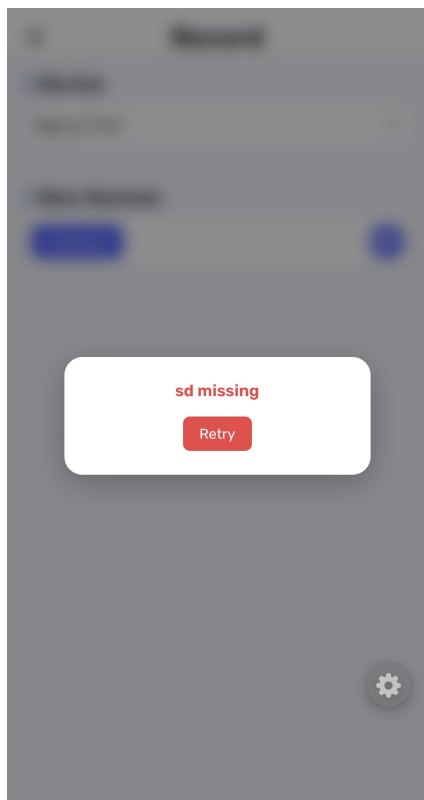
```

1 // Prima dell'avvio: retry
2 {(startSdMissing || retrying) && (
3     <SdMissingOverlay onAction={retryStartRecording} actionLabel="Retry"
4     actionLoading={retrying} loadingText="Retrying..." ... />
5 )}
6
7 // Durante la sessione: arresto forzato
8 {storageStatus?.sdMissing && (
9     <SdMissingOverlay onAction={() => stopRecording(activityName)}
10    actionLabel="Block session" actionLoading={true} ... />
11 )}

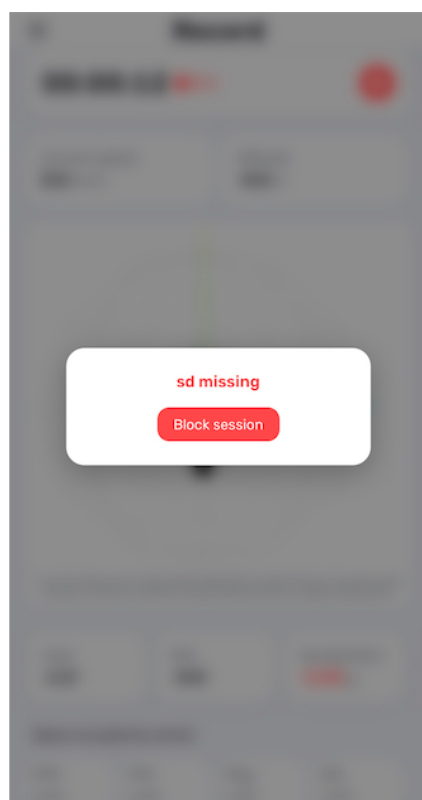
```

Codice 6.49: Le due invocazioni del componente, con azioni differenti

Nessun markup è duplicato tra i due casi: cambia solo quale funzione viene passata come `onAction` e quale etichetta la accompagna, realizzando lo Strategy Pattern descritto in progettazione senza introdurre due componenti distinti.



(a) Retry prima dell'avvio



(b) Arresto durante l'acquisizione

Figura 6.3: I due utilizzi del componente `SdMissingOverlay`

6.10.3 Impossibilità di chiusura senza azione

Il componente è realizzato con un `Modal` a schermo intero, privo di qualunque controllo di chiusura:

```

1 <Modal visible transparent animationType="fade">
2   <View style={styles.sdOverlay}>
3     <BlurView intensity={40} tint="dark" style={...} />
4     <Text style={styles.sdOverlayTitle}>sd missing</Text>
5     <Button title={actionLabel} onPress={onAction}>
6       ↪ showLoading={actionLoading} />
7   </View>
</Modal>

```

Codice 6.50: Overlay bloccante senza via di chiusura alternativa

L'unica interazione possibile con la schermata sottostante, finché la condizione persiste, è quella offerta dal singolo pulsante del componente, coerentemente con il `Blocking Modal Pattern` descritto in progettazione.

6.10.4 Scomparsa automatica al reinserimento della microSD

Né il componente `SdMissingOverlay` né la schermata che lo ospita mantengono uno stato proprio su "SD mancante": la sua visibilità è interamente derivata, a ogni render, dal

valore corrente di `storageStatus.sdMissing`. Quando la scheda viene reinserita, è il firmware stesso a rilevarlo ed eseguire il [ripristino della sessione](#) già descritto lato firmware, ripubblicando poi lo stato di storage aggiornato. Nell'app questo si traduce in un semplice cambio di valore della variabile booleana da cui dipende la condizione di rendering:

```
1 {storageStatus?.sdMissing && (  
2   <SdMissingOverlay onAction={() => stopRecording(activityName)}  
3     actionLabel="Block session" actionLoading={true} ... />  
4 )}
```

Codice 6.51: L'overlay scompare da solo non appena la condizione diventa falsa

6.11 Configurazione dei parametri del dispositivo

Questa sezione traduce in codice quanto descritto in [Configurazione dei parametri del dispositivo](#): l'Adapter/Facade Pattern che nasconde alla vista il significato di ogni UUID e la codifica esplicita del valore scritto, applicati al selettore del timeout di inattività dei sensori.

La vista non riceve mai un UUID grezzo da confrontare: riceve un oggetto di dominio già pronto, e lo confronta solo con una costante nominata per determinare se il parametro corrente è il timeout di inattività:

```
1 const isInactivityTimeout =  
2   item.getCharacteristicUUID() === BLE_CHARACTERISTICS.INACTIVITY_TIMEOUT
```

Codice 6.52: Riconoscimento del parametro di timeout tramite l'oggetto di dominio

6.11.1 Un controllo dedicato per un insieme discreto di valori

Il timeout di inattività non è un valore continuo su cui abbia senso uno slider, ma un insieme chiuso di opzioni con etichetta (mai, 10, 30 o 60 minuti), coerentemente con quanto già descritto [lato firmware](#): la vista mostra quindi un menu a tendina:

```
1 const INACTIVITY_TIMEOUT_OPTIONS = [  
2   { label: 'Never', value: 0 },  
3   { label: '10 minutes', value: 10 },  
4   { label: '30 minutes', value: 30 },  
5   { label: '60 minutes', value: 60 },  
6 ]  
7  
8 {isInactivityTimeout && (  
9   <DropDownPicker items={INACTIVITY_TIMEOUT_OPTIONS} value={value}  
10     setValue={...} />  
11 )}
```

Codice 6.53: Menu a tendina per il timeout di inattività

L'etichetta mostrata accanto al valore riflette lo stesso caso speciale già visibile nelle opzioni: il valore 0 non è mostrato come "0 minuti", ma come **Never**, rendendo esplicito all'utente che quel valore disabilita del tutto la sospensione dei sensori, non che ne azzerà il tempo di attesa.

6.11.2 Codifica esplicita immediatamente prima della scrittura

Il valore numerico scelto dall'utente nel menu a tendina viene serializzato secondo il formato concordato con il firmware solo nel punto in cui la scrittura avviene effettivamente, non prima:


```
1 const toUint16Bytes = (v: number): number[] => {  
2   const buf = new ArrayBuffer(2)  
3   new DataView(buf).setUint16(0, v, true) // little-endian  
4   return Array.from(new Uint8Array(buf))  
5 }
```

Codice 6.54: Codifica del valore in un intero a 16 bit little-endian

Fino a questo punto il valore resta un semplice numero JavaScript, scelto dall'utente tramite il menu a tendina senza alcuna conoscenza del formato di trasmissione; è solo immediatamente prima della chiamata di scrittura sulla Characteristic che il valore viene convertito nella rappresentazione a byte attesa dal firmware, rendendo esplicito e localizzato in un solo punto il contratto di formato tra le due parti.

7. Verifica e Validazione

Questo capitolo raccoglie le attività di verifica e validazione condotte sul sistema di telemetria EggLog, con l'obiettivo di dimostrarne sia la correttezza funzionale in condizioni d'uso reali, sia l'adeguatezza prestazionale rispetto ai vincoli del sistema embedded su cui è eseguito.

7.1 Introduzione

Il capitolo è organizzato in due sezioni: la prima presenta i test svolti, distinguendo tra i test su campo, condotti direttamente su motocicletta, e i test su banco, pensati per verificare in modo controllato singoli comportamenti del firmware; la seconda sezione presenta i *benchmark* prestazionali eseguiti sul firmware, relativi al tempo di scrittura su microSD e al carico computazionale imposto al microcontrollore.

7.2 Test

7.2.1 Metodologia dei test su campo

I test su campo sono stati condotti avviando il firmware in modalità *standalone*, così da riprodurre le condizioni d'uso reali del dispositivo: il boot avviene automaticamente non appena il sistema entra in contatto con una sorgente di alimentazione, senza alcun intervento manuale aggiuntivo.

1. **Allestimento del prototipo.** Per ciascun test è stato utilizzato il prototipo saldato (non la versione su breadboard, che ovviamente non avrebbe retto alle sollecitazioni di un utilizzo su strada). Il prototipo è stato fissato con dello scotch al serbatoio di una Yamaha MT-07. Sotto il codino della moto è stato collegato alla porta diagnostica l'EggLog CAN Sniffer, alimentato tramite un powerbank; analogamente, anche EggLog è stato collegato a un powerbank, riposto nella tasca del pilota.
2. **Preparazione della sessione.** Prima di ogni sessione di test è stata eseguita una pulizia completa della microSD, in modo da riportare il dispositivo in una condizione equivalente a quella di *appena acquistato*. È stato inoltre fissato al manubrio un dispositivo mobile con installata l'applicazione EggLog App, collegata a EggLog, ed è stata eseguita la calibrazione dell'IMU per configurare correttamente tutti gli assi. A questo punto è stata avviata una sessione di guida normale su strada, della durata complessiva di 15/20 minuti.
3. **Raccolta e conversione dei dati.** Al termine di ciascuna sessione, il sistema è stato smontato, la microSD estratta e i file binari recuperati. Questi ultimi sono stati convertiti in formato CSV tramite uno script Python sviluppato ad hoc, capace di leggere tutti i file `.bin` generati e produrre i corrispondenti file `.csv` per la successiva analisi.

L'analisi dei dati raccolti è stata condotta incrociando il percorso realmente effettuato (e i relativi timestamp) con l'insieme dei dati registrati, al fine di individuare eventuali bug, buchi nei dati, e per verificare che i valori registrati fossero coerenti con quanto effettivamente accaduto in un dato istante.

Alcuni esempi dei criteri di verosimiglianza adottati sono:

- in ingresso a una rotonda, l'accelerazione frontale deve essere negativa, la velocità deve calare e l'inclinazione frontale deve essere negativa (il posteriore si solleva leggermente);
- durante una curva a sinistra percorsa a velocità costante, l'accelerazione frontale deve essere prossima a zero, mentre l'inclinazione laterale deve avere segno negativo e rientrare in un range verosimile;
- in fase di accelerazione, l'accelerazione frontale deve essere positiva, così come l'inclinazione frontale, segno che la moto tende a impennarsi leggermente per effetto dell'allungamento delle forcelle.

Sono stati condotti tre test su campo, secondo la metodologia appena descritta. I primi due hanno portato all'individuazione di alcune criticità, poi risolte; il terzo si è invece concluso senza criticità: l'analisi incrociata dei dati raccolti con il percorso effettuato non ha evidenziato bug, buchi nei dati o incoerenze rispetto alla dinamica di guida attesa, e il tutor aziendale ha confermato, sulla base di questi risultati, il raggiungimento dell'obiettivo finale.

7.2.2 Test su campo 1 - Calibrazione e posizionamento del sensore

Il primo test ha evidenziato due criticità distinte, accennate anche in [Problematiche riscontrate durante la calibrazione dell'IMU](#).

Il primo step della calibrazione era caratterizzato da una soglia di varianza troppo restrittiva, che non teneva conto delle vibrazioni trasmesse dal motore acceso. Di conseguenza, con la moto accesa, il tremolio impediva il completamento con successo del primo passaggio di calibrazione. Il problema è stato temporaneamente risolto eseguendo la calibrazione a moto spenta.

Il giro di prova è stato completato con successo e la moto 3D dell'applicazione ha replicato fedelmente i movimenti reali del veicolo. È emerso tuttavia un secondo aspetto: essendo il sensore posizionato proprio sopra il serbatoio, l'accelerazione risultava accentuata, venendo talvolta interpretata dal firmware come un'impennata più marcata di quanto realmente avvenuto. La spiegazione, coerente con la posizione di montaggio, è che il sensore, trovandosi a una certa distanza dal centro di rotazione del veicolo, risenta di un effetto di braccio di leva: un lieve sollevamento del frontale, nel punto in cui è montato il prototipo, si traduce in un'inclinazione percepita superiore a quella reale.

In questa fase, il confronto con i dati provenienti dallo sniffer del bus CAN si è rivelato particolarmente utile: trattandosi di dati più affidabili rispetto a quelli restituiti dai sensori del prototipo, hanno permesso di effettuare l'incrocio con maggiore semplicità e sicurezza.

7.2.3 Test su campo 2 - Arrotondamento dei dati di accelerazione

Nel secondo test, la fase di calibrazione (già corretta a seguito del test precedente) è andata a buon fine senza criticità. Un'analisi più approfondita dei dati raccolti ha però permesso di individuare un problema non emerso a una prima ispezione: i valori di accelerazione risultavano sistematicamente arrotondati. La causa è stata ricondotta a un errore di conversione nel firmware, introdotto nel momento in cui i dati venivano inseriti nel superfile ([Sistema di persistenza a superfile](#)).

Il problema è stato corretto e i dati rimanenti sono stati analizzati comunque, confermando la correttezza del comportamento del sistema.

7.2.4 Test su campo 3 - Validazione finale

Il terzo test si è svolto senza che emergessero criticità. L'analisi incrociata dei dati raccolti con il percorso effettuato non ha evidenziato bug, buchi nei dati o incoerenze rispetto alla dinamica di guida attesa. Il tutor aziendale ha confermato, sulla base di questi risultati, il raggiungimento dell'obiettivo finale.

7.2.5 Test su banco 1 - Sospensione e ripresa del logging con fix GPS

Oltre ai test su campo, sono stati condotti due test su banco, con l'obiettivo di verificare in modo controllato la gestione dell'inattività dei sensori ([Gestione dell'inattività dei sensori](#)) e il comportamento del firmware in assenza di movimento, sia con che senza fix GPS.

Per verificare la gestione dell'inattività dei sensori, la procedura seguita è stata la seguente:

1. il firmware viene temporaneamente modificato per impostare a 5 minuti il timer minimo di inattività configurabile;
2. il dispositivo viene portato all'aperto, in attesa dell'acquisizione del fix GPS;
3. avviata una sessione di logging, il dispositivo viene lasciato perfettamente immobile;
4. trascorsi i 5 minuti, il LED della ESP32 inizia a pulsare in verde, a indicare l'ingresso nello stato di sospensione: il test ha quindi avuto esito positivo;
5. il dispositivo viene ripreso in mano e spostato: il LED torna verde fisso, segnalando il rientro nello stato di logging e la ripresa della scrittura dei dati.

7.2.6 Test su banco 2 - *Fallback* sull'IMU senza fix GPS

Lo stesso procedimento è stato ripetuto senza fix GPS, direttamente sulla scrivania in ufficio, per verificare il comportamento di *fallback* basato sull'IMU. Il risultato è stato identico a quello del test precedente: sospensione dopo il timeout configurato e ripresa del logging non appena il dispositivo veniva mosso.

7.2.7 Sintesi dei test

I test sono identificati mediante un codice univoco composto da due elementi: X-N, dove X identifica la modalità di esecuzione del test e N rappresenta il numero progressivo del test.

In particolare, C indica un test condotto su campo, mentre B indica un test condotto su banco.

Codice	Esito	Scoperta / Nota
C-1	Parzialmente positivo	Soglia di varianza della calibrazione troppo restrittiva; effetto di accentuazione dell'accelerazione per posizionamento sopra il serbatoio
C-2	Parzialmente positivo	Arrotondamento dei dati di accelerazione per errore di conversione nel superfile
C-3	Positivo	Nessuna criticità; conferma del raggiungimento degli obiettivi
B-1	Positivo	Sospensione e ripresa del logging corrette, con fix GPS
B-2	Positivo	Sospensione e ripresa del logging corrette, in fallback su IMU senza fix GPS

Tabella 7.1: Sintesi dei test condotti sul firmware

7.3 Benchmark prestazionali

7.3.1 Metodologia

I benchmark relativi al tempo di scrittura sono stati realizzati introducendo un timer attorno a ciascuna operazione che prevedeva l'accesso al modulo di scrittura su microSD. Per ogni operazione veniva avviato immediatamente prima dell'inizio dell'accesso al modulo e arrestato al completamento della relativa procedura di scrittura.

Il tempo così misurato veniva quindi passato a un oggetto statico dedicato alla raccolta delle statistiche. Attraverso metodi specifici, tale oggetto manteneva e aggiornava progressivamente il tempo minimo, medio e massimo di scrittura osservati durante l'esecuzione del benchmark.

Le operazioni sottoposte a misurazione sono state progettate appositamente per avere lo stesso peso computazionale e le stesse caratteristiche in entrambe le versioni del firmware. Ciò ha permesso di applicare la medesima metodologia sia all'implementazione precedente, basata sulla gestione di un file per ciascun *chunk*, sia alla nuova implementazione basata sul superfile, rendendo i risultati direttamente confrontabili.

7.3.2 Tempo di scrittura su microSD

7.3.2.0.1 Il problema della cluster size (superato)

Nell'approccio precedente, basato su un file per *chunk* ([Situazione di partenza](#)), il dimensionamento della cluster size rappresentava un problema di non semplice soluzione: bisognava gestire il rischio di frammentazione. Ogni nuovo file creato richiedeva infatti l'allocazione di nuovi cluster, con un relativo *overhang* ogni volta che il *chunk* non riempiva completamente l'ultimo cluster allocato.

Con l'introduzione del superfile ([Sistema di persistenza a superfile](#)), un unico file preallocato per sensore, mai chiuso né ricreato per l'intera durata del logging, ha eliminato alla radice questo problema: l'area dati viene allocata una sola volta per l'intera retention, e tutte le scritture successive avvengono all'interno di cluster già assegnati. Non è quindi più necessario gestire l'*overhang chunk* per *chunk*, né si introduce frammentazione a runtime.

7.3.2.0.2 Confronto misurato: un file per chunk vs superfile

Per quantificare il beneficio introdotto dal superfile, è stato misurato il tempo (in μs) dell'intero blocco di scrittura su un campione di 8 ore di logging:

	Un file per chunk	superfile	Differenza
avg	150.045 μs	83.330 μs	−44%
min	70.547 μs	53.447 μs	−24%
max	665.726 μs	131.747 μs	−80%

Tabella 7.2: Confronto dei tempi di scrittura su microSD

I risultati mostrano un netto miglioramento su tutte le metriche considerate, in particolare sul valore massimo, dove il superfile riduce dell'80% il caso peggiore osservato. È inoltre

da notare che, aumentando la dimensione del campione, il tempo medio della versione di partenza tende a peggiorare ulteriormente, per effetto della crescente frammentazione e del tempo di ricerca dello spazio libero, mentre quello del superfile rimane stabile: un comportamento coerente con l'assenza, in quest'ultimo, di allocazione di nuovi cluster a runtime.

7.3.3 Occupazione di CPU

Oltre al tempo di scrittura su microSD, sono stati misurati anche l'occupazione di CPU nei diversi stati del firmware e l'occupazione di RAM effettivamente utilizzata dai buffer dei sensori, per entrambe le versioni.

Poiché il logging veniva avviato automaticamente non appena il dispositivo veniva collegato a una fonte di alimentazione ([Situazione di partenza](#)), non esisteva uno stato di attesa distinto dallo stato di logging: l'occupazione di CPU restava quindi costantemente attorno al 25%.

Con l'introduzione del protocollo di avvio e arresto della sessione ([Protocollo di avvio e arresto della sessione](#)), il dispositivo, appena collegato all'alimentazione e senza alcuna sessione avviata, presenta un'occupazione di CPU di circa il 3%. All'avvio effettivo di una sessione, il consumo sale a circa il 23%, un valore inferiore anche a quello della versione di partenza. L'introduzione della task dedicata alla segnalazione tramite LED ([Segnalazione di stato tramite LED](#)), assente nella versione originale, aggiunge un piccolo *overhead* rispetto alle sole task di producer e consumer già presenti; tale *overhead* risulta però più che compensato dalla configurazione attuale dei sensori, in cui il magnetometro è disabilitato: una task producer in meno comporta un risparmio di cicli di CPU superiore al costo della nuova task LED. Il firmware attuale tende inoltre a risparmiare più cicli di CPU possibile rispetto alla versione di partenza grazie all'*inactivity* dei sensori implementata.

La Tabella 7.3 riassume i valori osservati nei diversi stati.

Stato del dispositivo	Prima	Dopo
Prima di una sessione	25%	3%
Durante una sessione	25%	23%

Tabella 7.3: Occupazione di CPU nei diversi stati del firmware

7.3.4 Occupazione della RAM

7.3.4.0.1 Calcolo della RAM occupata

Entrambe le versioni assegnano a ogni producer due buffer alternati (*ping-pong*, si veda [Situazione di partenza](#)), quindi la RAM effettivamente occupata da ciascun sensore è pari al doppio della dimensione del singolo buffer.

7.3.4.0.2 Confronto per sensore

La Tabella 7.4 riporta il confronto tra le due versioni, entrambe i buffer inclusi. Gli aumenti più significativi riguardano i sensori GPS e IMU: la relativa giustificazione è approfondita nel paragrafo seguente. Il buffer del barometro resta invece sostanzialmente invariato. Il magnetometro, il cui buffer nella versione attuale sarebbe stato dimensionato per circa

31.8 KB, risulta invece disabilitato: l'occupazione di RAM ad esso associata è quindi nulla (si veda il paragrafo successivo).

Sensore	Versione di partenza	Versione attuale	Differenza
Barometro	5.0 KB	4.7 KB	−0.3 KB
GPS	4.0 KB	39.4 KB	+35.4 KB
IMU	123.0 KB	130.3 KB	+7.3 KB
Magnetometro	32.0 KB	Disabilitato	−32.0 KB
Totale	164.0 KB	174.4 KB	+10.4 KB

Tabella 7.4: RAM occupata dai buffer dei producer (ping-pong incluso)

7.3.4.0.3 Variazione percentuale tra le due versioni

Esprese in termini percentuali rispetto alla versione di partenza, le variazioni per ciascun sensore sono riportate in Tabella 7.5.

Sensore	Variazione
Barometro	−6.0%
GPS	+885.0%
IMU	+5.9%
Magnetometro	Disabilitato
Totale	+6.3%

Tabella 7.5: Variazione percentuale della RAM occupata dai buffer dei producer

7.3.4.0.4 Il ruolo del formato di serializzazione

Il confronto in termini di *kilobyte* occupati, per quanto corretto, non racconta però l'intera storia: nella versione di partenza ogni campione veniva serializzato in formato testuale CSV direttamente all'interno del buffer del producer, mentre nella versione attuale ogni campione occupa uno slot a dimensione fissa pari a `RECORD_SIZE` in un formato binario compatto. Stimando la lunghezza di una riga CSV a partire dai formati di conversione impiegati (`snprintf`) e dai relativi range di valore dei singoli campi, si ottiene una dimensione per campione sensibilmente superiore a quella del corrispondente record binario, come riportato in Tabella 7.6.

Sensore	CSV (B/record)	Binario (B/record)
Barometro	32	20
GPS	75	42
IMU	56	20
Magnetometro	40	21

Tabella 7.6: Confronto tra la dimensione stimata di un record in formato CSV e la dimensione del record binario attuale

Applicando queste dimensioni alla capacità del singolo buffer riportata in Tabella 7.4, emerge che il numero di campioni effettivamente contenibili in un buffer è cresciuto mol-

to più di quanto suggerito dal solo aumento in kilobyte, ed è aumentato persino per il barometro, il cui buffer si è invece *ridotto* in termini assoluti (Tabella 7.5).

Sensore	Record/buffer (prima)	Record/buffer (ora)	Fattore
Barometro	≈ 80	120	$\times 1,5$
GPS	≈ 27	480	$\times 17,8$
IMU	≈ 1124	3336	$\times 3,0$

Tabella 7.7: Numero stimato di campioni contenuti in un singolo buffer, prima e dopo il passaggio al formato binario

In altre parole, l'incremento di RAM riportato in Tabella 7.5 rappresenta solo una parte del guadagno complessivo: una quota rilevante dell'aumento di capacità di ritenzione dei buffer deriva dal passaggio da una rappresentazione testuale, verbosa e a lunghezza variabile, a una rappresentazione binaria compatta a lunghezza fissa, indipendente dall'eventuale ridimensionamento dei buffer stessi.

7.3.4.0.5 Un compromesso tra RAM e frequenza di scrittura

L'aumento dei buffer di GPS e IMU deriva dalla scelta di privilegiare un minor numero di operazioni di I/O sulla microSD, a fronte di un maggiore utilizzo della RAM. Le operazioni di I/O sono infatti sensibilmente più lente rispetto all'accesso alla memoria principale; accumulare più campioni prima di ogni scrittura consente quindi di ridurre la frequenza e il relativo costo prestazionale. Tale dimensionamento è indipendente dalla finestra di retention del superfile ([Capacità del file](#)) e dipende dalla frequenza di trasferimento dei campioni (`FLUSH_INTERVAL_S`, Tabella 6.2).

La scelta è stata applicata a GPS e IMU, considerati più critici rispetto al barometro, per i quali si è preferito sostenere una maggiore occupazione della RAM pur di ridurre gli accessi alla microSD. Per evitare la perdita dei campioni ancora presenti nei buffer al termine della sessione, la sequenza di arresto ([Protocollo di avvio e arresto della sessione](#)) esegue uno svuotamento forzato di tutti i buffer prima della chiusura dei superfile.

7.3.4.0.6 Disabilitazione del magnetometro

Il magnetometro, i cui dati non trovano al momento alcun utilizzo concreto nel sistema, è stato invece disabilitato in questa versione del firmware: il suo buffer, presente nella versione di partenza con una dimensione di circa 32.0 KB, non viene più allocato, azzerandone completamente l'occupazione di RAM.

7.3.4.0.7 Il momento dell'allocazione conta più della dimensione

Il dato più rilevante non riguarda quindi la dimensione dei singoli buffer, quanto il momento in cui vengono allocati. Nella versione di partenza, il logging iniziava automaticamente all'accensione e tutti e quattro i buffer restavano allocati permanentemente, per un totale di circa 164 KB. Nella versione attuale, invece, i buffer dei sensori attivi (barometro, GPS e IMU) vengono allocati solo all'avvio effettivo di una sessione, grazie al protocollo di avvio ([Protocollo di avvio e arresto della sessione](#)) e ai producer inattivi fino alla ricezione di un

comando esplicito. Nello stato di attesa risultano quindi liberi circa 174 KB, contribuendo al netto calo dell'occupazione di CPU, dal 25% al 3% (Tabella 7.3).

Durante il logging, l'occupazione dei buffer aumenta invece a circa 174 KB, contro i 164 KB della versione di partenza. L'incremento, pari a circa il 6%, è un compromesso deliberato: si è preferito utilizzare una maggiore quantità di RAM per accumulare più campioni e ridurre la frequenza delle operazioni di I/O sulla microSD, più lente rispetto all'accesso alla memoria principale.

8. Conclusioni

In questo capitolo si presenta il consuntivo finale del lavoro svolto, valutando il grado di raggiungimento degli obiettivi e di soddisfazione dei requisiti di progetto, e si propone, oltre alle conoscenze acquisite, una riflessione personale sul percorso intrapreso.

8.1 Consuntivo finale

La Tabella 8.1 riporta la ripartizione delle 320 ore di stage tra le fasi in cui il lavoro è stato organizzato, descritte nella [Sezione Pianificazione del Capitolo](#).

Attività principali	Ore
Onboarding e analisi del progetto	16
Studio delle tecnologie e analisi del firmware	56
Analisi e progettazione delle modifiche	64
Sviluppo e ottimizzazione dello storage	24
Gestione dei dati e comunicazioni	40
Analisi tecnica CAN sniffer e integrazione firmware	56
Testing e validazione	48
Documentazione finale	16
Totale	320

Tabella 8.1: Ripartizione delle ore di stage per fase

8.2 Raggiungimento degli obiettivi

Questa sezione mette a confronto gli obiettivi definiti dall'azienda in fase di pianificazione (Sezione [Obiettivi](#)) con quanto effettivamente realizzato.

8.2.1 Obiettivi obbligatori

Codice	Obiettivo	Raggiunto
OO-1	<i>Refactoring</i> e ottimizzazione del sistema di lettura e scrittura su scheda SD	Sì
OO-2	Completamento del sistema di rotazione dei file mediante l'utilizzo di ring-buffer	Sì
OO-3	Implementare la scrittura e salvataggio dei dati raccolti direttamente in formato binario	Sì
Continua nella prossima pagina...		

Tabella 8.2 - Continua dalla pagina precedente

Codice	Obiettivo	Raggiunto
OO-4	Implementazione di un sistema di <i>recovery</i> che permette di riprendere la scrittura su SD in caso di guasti o rimozione della stessa dal suo <i>slot</i>	Sì
OO-5	Miglioramento dei servizi BLE e introduzione di nuovi comandi firmware	Sì
OO-6	Aggiunta di metriche di confronto per poter stimare il miglioramento rispetto alla versione precedente del firmware	Sì
OO-7	Implementare un protocollo di calibrazione dei sensori	Sì
OO-8	Aggiungere una logica di sessionamento, attivando i sensori solamente all'avvio della sessione e disattivandoli quando termina la sessione	Sì
OO-9	Produzione della documentazione tecnica relativa al firmware sviluppato	Sì

Tabella 8.2: Raggiungimento degli obiettivi obbligatori

8.2.2 Obiettivi desiderabili

Codice	Obiettivo	Raggiunto
OD-1	Permettere la visualizzazione dei movimenti della moto in tempo reale con un modellino 3D	Sì
OD-2	Integrazione dell'analisi e gestione dei dati provenienti dalla ECU	Sì
OD-3	Implementare un sistema di rilevamento di assenza della scheda microSD	Sì

Tabella 8.3: Raggiungimento degli obiettivi desiderabili

8.2.3 Obiettivi facoltativi

Codice	Obiettivo	Raggiunto
OF-1	Implementazione di un sistema di indicizzazione delle sessioni memorizzate sulla scheda SD	Sì
OF-2	Implementazione di un sistema di <i>inactivity</i> per permettere la disattivazione dei sensori quando non viene rilevato alcun movimento per un tempo configurabile dall'utente	Sì

Tabella 8.4: Raggiungimento degli obiettivi facoltativi

8.2.4 Requisiti raggiunti

Sulla base delle scelte progettuali e implementative descritte nei capitoli precedenti, risultano raggiunti complessivamente **37** requisiti sui **38** tracciati in [Sezione Tracciamento dei](#)

requisiti. La Tabella 8.5 riporta, per ciascuna categoria e urgenza, il numero di requisiti raggiunti sul totale tracciato.

	Obbligatori	Desiderabili	Opzionali	TOTALE
RF	14/14	7/7	1/1	22/22
RN	3/4	3/3	1/1	7/8
RV	8/8	0/0	0/0	8/8
TOTALE	25/26	10/10	2/2	37/38

Tabella 8.5: Requisiti raggiunti, per categoria e urgenza

L'unico requisito non pienamente raggiunto è **R-N-Ob-1**, che riguarda la riduzione del consumo di RAM rispetto alla versione precedente del firmware. La motivazione dietro al parziale raggiungimento è stata approfondita maggiormente nella sezione sulla [Occupazione della RAM](#).

8.3 Conoscenze acquisite

Nel corso del tirocinio sono state acquisite e consolidate competenze tecniche e metodologiche. In particolare, è stata maturata esperienza nello sviluppo e nel refactoring di firmware embedded in C++ su ESP32, con gestione della concorrenza tramite FreeRTOS e variabili atomiche, nella progettazione di formati binari a basso overhead con meccanismi di recovery e nella realizzazione di servizi BLE, sia in modalità connessa sia broadcast. Sono inoltre state sviluppate competenze nell'analisi di protocolli CAN proprietari tramite tecniche di sniffing, nella valutazione di alternative hardware secondo criteri di costo e rischio e nello sviluppo mobile con React Native/TypeScript, con attenzione all'ottimizzazione del rendering. Le scelte implementative sul firmware sono state inoltre valutate tramite benchmark comparativi, misurando quantitativamente l'impatto su CPU e RAM.

Dal punto di vista metodologico, il tirocinio ha permesso di consolidare la capacità di lavorare su componenti differenti di uno stesso sistema, coordinandone le interfacce, di operare secondo una metodologia Agile e di condurre analisi preventive dei rischi. È stata inoltre sviluppata una maggiore capacità di motivare le scelte progettuali e di affrontare attività di debug in ambito embedded, ricercando le cause dei problemi anche a livello fisico ed elettrico.

8.4 Valutazione personale

L'esperienza di tirocinio svolta presso Eggon è stata, nel complesso, **estremamente positiva**.

Un aspetto che ho apprezzato particolarmente è stato lo sviluppo congiunto di firmware e applicazione insieme alla mia collega tirocinante: questo lavoro parallelo, ma costantemente allineato, ci ha tenuti uniti per l'intera durata del tirocinio ed è stato per me un'occasione importante di crescita nella capacità di comunicare in modo chiaro ed efficace il proprio lavoro, soprattutto durante gli *stand-up meeting* giornalieri.

Mi sono inoltre trovato molto bene con il team di Eggon, che ringrazio per il supporto costante, per aver stimolato in me la ricerca di soluzioni sempre migliori e per la disponibilità all'ascolto anche nei momenti di difficoltà, specialmente durante le fasi di implementazione più complesse e ricche di bug.

Nel complesso, ritengo questa esperienza di tirocinio molto formativa, sia dal punto di vista tecnico sia da quello relazionale.

Glossario

ACK Acronimo di *Acknowledgement*. Messaggio di conferma inviato dal destinatario per indicare che un dato, un pacchetto o un comando è stato ricevuto correttamente.. 80

advertising Meccanismo BLE mediante il quale un dispositivo trasmette periodicamente pacchetti che ne permettono il rilevamento da parte di altri dispositivi.. 53

Metodologia Agile La metodologia Agile è un approccio alla gestione e allo sviluppo di progetti software basato su iterazioni brevi, collaborazione continua e capacità di adattarsi rapidamente ai cambiamenti dei requisiti. Le attività vengono organizzate in cicli di sviluppo incrementali, durante i quali il team pianifica, realizza e verifica progressivamente le funzionalità del sistema, favorendo il confronto tra i membri del gruppo e il miglioramento continuo del prodotto.. 10

Arduino Arduino è una piattaforma hardware e software open-source basata su microcontrollori, progettata per facilitare lo sviluppo e la prototipazione di sistemi elettronici e applicazioni embedded.. 2, 3

automotive Il termine automotive si riferisce al settore industriale e tecnologico dedicato alla progettazione, allo sviluppo, alla produzione e alla gestione dei veicoli a motore e delle tecnologie ad essi associate.. 2

Bluetooth Low Energy Bluetooth Low Energy è una tecnologia di comunicazione wireless a basso consumo energetico basata sullo standard Bluetooth. È progettata per applicazioni che richiedono lo scambio periodico di piccole quantità di dati mantenendo un ridotto consumo energetico, risultando particolarmente adatta per dispositivi embedded, sensori e sistemi IoT.. 6

buffer circolare Struttura dati di dimensione fissa in cui, una volta raggiunta la capacità massima, la scrittura riprende dall'inizio sovrascrivendo i dati meno recenti (*wraparound*), anziché richiedere l'allocazione di nuovo spazio. È utilizzata per gestire archivi di dimensione limitata che devono conservare solo la porzione più recente di un flusso continuo di dati.. 7

bus CAN Il bus Controller Area Network (CAN) è una rete di comunicazione seriale progettata per consentire lo scambio affidabile di dati tra dispositivi elettronici all'interno di un sistema distribuito. È ampiamente utilizzato in ambito automotive per la comunicazione tra le centraline elettroniche del veicolo, grazie alla sua robustezza, alla gestione degli errori integrata e alla capacità di operare in ambienti caratterizzati da elevati livelli di disturbo elettromagnetico.. 6

CAN sniffing Il CAN sniffing è una tecnica di analisi del traffico presente su una rete Controller Area Network (CAN) che consiste nell'acquisizione e registrazione dei messaggi scambiati tra i dispositivi collegati al bus. Questa attività permette di analizzare gli identificativi dei messaggi (*CAN ID*), i dati trasmessi e la frequenza di comunicazione, al fine di individuare la struttura e il significato delle informazioni scambiate tra le centraline elettroniche. In ambito automotive viene utilizzata, ad esempio, per attività di diagnostica, sviluppo di sistemi telemetrici e analisi del comportamento delle ECU.. [6](#)

characteristic Elemento di un Service BLE che rappresenta un dato o una funzionalità accessibile tramite operazioni di lettura, scrittura o notifica.. [53](#)

CID *Card Identification* è un identificativo univoco associato a una scheda SD. La sua lettura consente di verificare che la scheda sia fisicamente presente e correttamente operativa.. [94](#)

cluster Unità minima di allocazione dello spazio su una memoria di massa gestita da un file system, costituita da uno o più settori contigui. Ogni file occupa un numero intero di cluster anche quando i suoi dati non ne riempiono completamente l'ultimo, generando uno spazio inutilizzato noto come *overhang*.. [91](#)

C++ C++ è un linguaggio di programmazione general-purpose derivato dal linguaggio C, che integra il supporto alla programmazione orientata agli oggetti e a diversi paradigmi di sviluppo. Grazie alle sue elevate prestazioni e alla possibilità di interagire direttamente con le risorse hardware, viene frequentemente utilizzato nello sviluppo di sistemi embedded e applicazioni dove sono richiesti efficienza e controllo delle risorse.. [6](#)

CPU La CPU (*Central Processing Unit*) è l'unità di elaborazione centrale di un sistema di calcolo. Ha il compito di eseguire le istruzioni dei programmi, effettuare operazioni aritmetiche e logiche e coordinare il funzionamento delle altre componenti hardware del dispositivo.. [11](#)

CSV CSV (*Comma-Separated Values*) è un formato di file testuale utilizzato per rappresentare dati tabellari attraverso una sequenza di valori separati da un delimitatore, generalmente una virgola. Questo formato è ampiamente utilizzato per lo scambio e l'archiviazione di dati strutturati grazie alla sua semplicità e alla compatibilità con numerosi strumenti di elaborazione e analisi dei dati.. [12](#)

debounce Tecnica utilizzata per evitare che variazioni brevi e ripetute di un segnale vengano interpretate come eventi distinti. Nel sistema, consente di confermare il rilevamento del movimento solo quando la condizione permane per un determinato intervallo di tempo, riducendo i falsi rilevamenti dovuti a variazioni momentanee delle misure GPS o IMU.. [67](#)

Digital Health Con Digital Health si intende l'insieme di tecnologie e soluzioni digitali applicate al settore sanitario, che mirano a migliorare la gestione della salute, la

prevenzione delle malattie e l'efficienza dei servizi sanitari. Include strumenti come dispositivi indossabili, applicazioni mobili, telemedicina, intelligenza artificiale e analisi dei dati per supportare la diagnosi, il monitoraggio e il trattamento dei pazienti.. [1](#)

Electronic Control Unit Una Electronic Control Unit (ECU) è una centralina elettronica presente nei veicoli moderni incaricata di controllare e gestire specifiche funzionalità del mezzo, come il controllo del motore, la gestione dell'iniezione o l'acquisizione di dati provenienti dai sensori. Le ECU comunicano frequentemente tra loro attraverso reti interne del veicolo, come il bus CAN, per lo scambio delle informazioni necessarie al funzionamento dei sistemi elettronici.. [6](#)

embedded Un sistema embedded è un sistema hardware-software dedicato all'esecuzione di funzioni specifiche all'interno di un dispositivo elettronico, spesso caratterizzato da vincoli di prestazioni, memoria e consumo energetico.. [1](#)

FAT Con File Allocation Table (FAT) si indica il sistema di gestione dello spazio di archiviazione utilizzato da un file system per organizzare e tenere traccia dei file presenti su una memoria. La FAT mantiene una struttura di informazioni che associa i file alle unità di allocazione del dispositivo, permettendo al sistema di individuare i dati che li compongono e gestire lo spazio disponibile.. [52](#)

Fintech Con Fintech si intende l'insieme di tecnologie e soluzioni digitali applicate al settore finanziario, che mirano a migliorare l'efficienza dei processi, a ridurre i costi e a offrire servizi più personalizzati ai clienti.. [1](#)

firmware Software a basso livello che viene eseguito direttamente su un dispositivo hardware, permettendone il controllo e la gestione delle sue componenti elettroniche.. [1](#)

fix GPS Stato nel quale un ricevitore GPS ha acquisito e utilizzato un numero sufficiente di segnali satellitari per determinare in modo affidabile la propria posizione. Il mantenimento del fix consente di continuare a fornire informazioni di posizione senza dover effettuare una nuova fase di acquisizione dei satelliti.. [67](#)

GAP Profilo Bluetooth che definisce le modalità con cui i dispositivi vengono rilevati, identificati e connessi, nonché i relativi ruoli nella comunicazione.. [53](#)

hook Meccanismo che consente di collegare un componente o una funzione a funzionalità e comportamenti forniti da un framework o da una libreria. In React, gli hook permettono ai componenti funzionali di utilizzare funzionalità quali la gestione dello stato, degli effetti collaterali e del contesto.. [85](#)

hPa Ettopascal, unità di misura della pressione pari a 100 pascal (Pa), comunemente utilizzata per esprimere la pressione atmosferica.. [106](#)

Industrial Internet of Things Con Industrial Internet of Things (IIoT) si intende l'insieme di tecnologie e dispositivi connessi a Internet che vengono utilizzati in ambito

industriale per migliorare l'efficienza, la produttività e la sicurezza dei processi produttivi. L'IIoT consente la raccolta e l'analisi dei dati in tempo reale, permettendo alle aziende di prendere decisioni più informate e ottimizzare le operazioni.. [1](#), [3](#)

IMU IMU (*Inertial Measurement Unit*) è un dispositivo elettronico composto generalmente da accelerometri e giroscopi, in grado di misurare rispettivamente l'accelerazione lineare e la velocità angolare di un oggetto. Le IMU vengono utilizzate per rilevare il movimento, l'orientamento e le variazioni di posizione di un dispositivo, risultando fondamentali in numerose applicazioni come sistemi embedded, robotica, veicoli e sistemi di telemetria.. [12](#)

Industria 5.0 Con Industria 5.0 si intende la quinta generazione di industrializzazione, caratterizzata dall'integrazione di tecnologie digitali, l'Intelligenza Artificiale e l'Automazione, per migliorare la produttività e l'efficienza dei processi produttivi.. [1](#)

Insurtech Con Insurtech si intende l'insieme di tecnologie e soluzioni digitali applicate al settore assicurativo, che mirano a migliorare l'efficienza dei processi, a ridurre i costi e a offrire servizi più personalizzati ai clienti.. [1](#)

Internet of Things Insieme di dispositivi fisici connessi alla rete in grado di raccogliere, elaborare e scambiare dati, permettendo il monitoraggio e il controllo remoto delle informazioni.. [2](#)

isolamento galvanico Separazione elettrica tra due circuiti che impedisce il passaggio diretto di corrente continua tra essi, pur consentendo, quando previsto, il trasferimento di energia o di informazioni.. [78](#)

LED Il LED (*Light Emitting Diode*) è un componente elettronico a semiconduttore in grado di emettere luce quando è attraversato da una corrente elettrica. Grazie al basso consumo energetico, alla lunga durata e all'elevata efficienza luminosa, è ampiamente utilizzato come indicatore di stato e per applicazioni di illuminazione in dispositivi elettronici ed embedded.. [v](#), [22](#)

multipath Fenomeno di propagazione dei segnali satellitari nel quale lo stesso segnale raggiunge il ricevitore attraverso più percorsi, ad esempio a causa di riflessioni su edifici, terreno o altre superfici. La sovrapposizione del segnale diretto con le sue copie riflesse può introdurre errori nella stima della posizione e della qualità del segnale GNSS.. [67](#)

ODR Con Output Data Rate (ODR) si indica la frequenza con cui un sensore acquisisce e rende disponibili nuovi dati in uscita. Generalmente è espressa in hertz (Hz) e determina il numero di campioni prodotti nell'unità di tempo. La scelta dell'ODR influisce sulla quantità di dati raccolti, sulla precisione temporale delle misurazioni e sul consumo energetico del dispositivo.. [59](#), [90](#)

ping-pong buffering Il ping-pong buffering è una tecnica di gestione della memoria utilizzata nei sistemi embedded e nelle applicazioni che richiedono un'acquisizione

continua di dati. Consiste nell'utilizzo di due buffer distinti: mentre un buffer viene utilizzato per la scrittura dei nuovi dati acquisiti, l'altro viene elaborato o trasferito dal sistema. Al termine di un ciclo di acquisizione, i due buffer vengono scambiati, permettendo di ridurre i tempi di attesa e garantire un flusso continuo di dati senza interrompere le operazioni di lettura o elaborazione.. [12](#)

Producer-Consumer Pattern di programmazione concorrente in cui uno o più task produttori (*producer*) generano dati resi disponibili, tipicamente tramite una coda o un buffer condiviso, a uno o più task consumatori (*consumer*) incaricati di elaborarli, disaccoppiando la velocità di produzione da quella di elaborazione.. [53](#)

RAM La RAM (*Random Access Memory*) è una memoria volatile utilizzata dal sistema per memorizzare temporaneamente dati e istruzioni necessari all'esecuzione dei programmi. Il suo contenuto viene perso allo spegnimento del dispositivo, ma consente un accesso rapido alle informazioni durante il funzionamento del sistema.. [11](#)

record Con record si indica una singola unità di dati strutturati, costituita da un insieme di valori associati a una specifica osservazione o acquisizione. In una sequenza di misure, ciascun record rappresenta una singola rilevazione effettuata in un determinato istante.. [57](#)

Scrum Scrum è un framework appartenente alla metodologia Agile utilizzato per organizzare il lavoro attraverso iterazioni chiamate sprint. Prevede ruoli, eventi e strumenti specifici per pianificare le attività, monitorare l'avanzamento del progetto e favorire la collaborazione tra i membri del team.. [10](#)

service Raggruppamento logico di funzionalità e dati esposti da un dispositivo BLE.. [53](#)

SPI *Serial Peripheral Interface* è un protocollo di comunicazione seriale sincrona utilizzato per lo scambio di dati tra un dispositivo master e una o più periferiche.. [94](#)

stand-up meeting Lo stand-up meeting è una breve riunione periodica utilizzata nelle metodologie Agile per favorire l'allineamento tra i membri di un team di sviluppo. Durante l'incontro ogni partecipante condivide lo stato delle attività svolte, gli obiettivi da completare e gli eventuali problemi o impedimenti riscontrati. L'obiettivo principale è garantire una comunicazione efficace, individuare tempestivamente eventuali criticità e coordinare il lavoro del gruppo.. [10](#)

transceiver Dispositivo elettronico in grado di svolgere sia la funzione di trasmettitore sia quella di ricevitore, consentendo la trasmissione e la ricezione di segnali attraverso un determinato mezzo di comunicazione.. [78](#)

transienti Variazioni rapide e di breve durata di una grandezza fisica, in particolare elettrica, che si verificano durante il passaggio da una condizione di funzionamento a un'altra.. [78](#)

TVS Transient Voltage Suppressor, dispositivo di protezione utilizzato per limitare sovratensioni transitorie ai capi di un circuito elettrico.. [78](#)

TWAI Periferica integrata nei microcontrollori ESP32 per la comunicazione tramite bus CAN.. [79](#)

Sitografia

- arduino-esp32*. URL: <https://github.com/espressif/arduino-esp32> (cit. a p. 44).
- C++20 – cppreference.com*. URL: <https://en.cppreference.com/w/cpp/20> (cit. a p. 44).
- C4 model*. URL: <https://c4model.com/> (cit. a p. 51).
- Datasheet di NUP2105LT1G*. URL: <https://www.onsemi.cn/pdf/datasheet/nup21051-d.pdf> (cit. a p. 79).
- Datasheet di SN65HVD230*. URL: <https://files.waveshare.com/upload/3/3a/SN65HVD230-CAN-Board-Datasheets.pdf> (cit. a p. 79).
- ESP32-S3-DevKitC-1*. URL: <https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/index.html> (cit. alle pp. 3, 42).
- ESP32-WROOM-32 Series*. URL: <https://www.espressif.com/en/products/modules/esp32> (cit. a p. 43).
- Expo Documentation*. URL: <https://expo.dev/> (cit. a p. 49).
- FreeRTOS – Real-time operating system for microcontrollers*. URL: <https://www.freertos.org/> (cit. a p. 45).
- Git*. URL: <https://git-scm.com/> (cit. a p. 50).
- LaTeX – A document preparation system*. URL: <https://www.latex-project.org/> (cit. a p. 51).
- PlantUML*. URL: <https://plantuml.com/> (cit. a p. 51).
- PlatformIO*. URL: <https://platformio.org/> (cit. a p. 50).
- React Native*. URL: <https://reactnative.dev/> (cit. a p. 49).
- TypeScript*. URL: <https://www.typescriptlang.org/> (cit. a p. 49).